线性表

线性表

线性表的定义与特点

线性表的顺序表示与实现

线性表的链式表示与实现

线性表的应用

线性表

由 n（$n \geqslant 0$）个数据特性相同的元素构成的有限序列，称为线性表。

线性表是 n 个数据元素的有限序列，其中 n 个数据是相同数据类型的。

线性表

线性表中元素的个数 n（$n \geqslant 0$）定义为线性表的长度，当 n=0 时称之为空表。

对于非空的线性表或线性结构，其特点是：

存在唯一的一个被称作“第一个”的数据元素；

存在唯一的一个被称作“最后一个”的数据元素；

除第一个元素外，结构中的每个数据元素均只有一个前驱；

除最后一个元素外，结构中的每个数据元素均只有一个后继。

案例2.3：图书信息管理系统

查找：根据指定的 ISBN 或书名查找相应图书的有关信息。

插入：插入一条新的图书信息。

删除：删除一条图书信息。

ISBN	书名	定价
98856432334	程序设计基础	25
98676454354	单片机技术及应用	32
98745354322	编译原理	46
96467688768	汇编语言程序设计教程	21

修改：根据指定的 ISBN ， 修改该图书的价格。

排序：将图书按照价格由低到高进行排序。

计数：统计文件中的图书数量。

线性表 ADT

ADT List {

数据对象：D={a_i|a_i ∈ ElemSet,i=1,2,…,n,$n \geqslant 0$}

数据关系：S={<a_{i-1},a_i>|a_{i-1},a_i ∈ D,i=2,…,n}

基本操作：

InitList(&L)

操作结果：构造一个空的线性表L。

DestroyList(&L)

初始条件：线性表L已存在。

操作结果：销毁线性表L。

……

} ADT List

顺序表

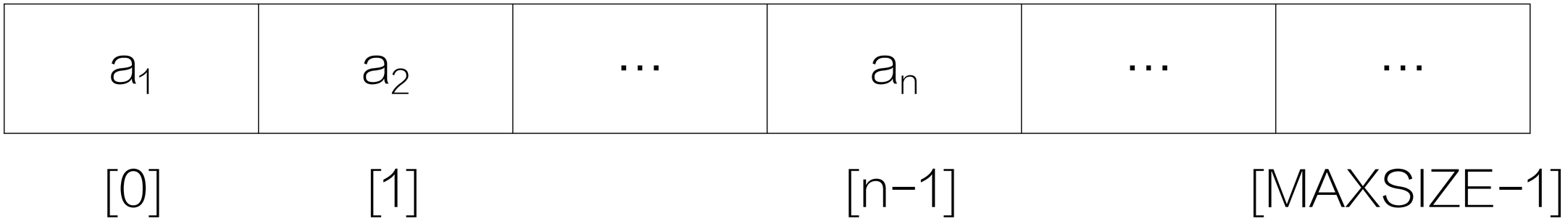
用一组连续的内存单元依次存储线性表的各个元素，也就是说，逻辑上相邻的元素，实际的物理存储空间也是连续的。

顺序表 – 存储结构

高内聚 低耦合

```
#define MAXSIZE 100
typedef int ElemType;


typedef struct{
    ElemType data[MAXSIZE];
    int length;
}SeqList;
```



顺序表 – 初始化

```
#define MAXSIZE 100
typedef int ElemType;

typedef struct{
    ElemType data[MAXSIZE];
    int length;
}SeqList;

void initList(SeqList *L)
{
    L->length = 0;
}
```

案例：线性表/顺序表/test1.c

顺序表 – 在尾部添加元素

```
int appendElem(SeqList *L, ElemType e)
{
    if (L->length>=MAXSIZE)
    {
        printf("顺序表已满\n");
        return 0;
    }
    L->data[L->length] = e;
    L->length++;
    return 1;
}
```

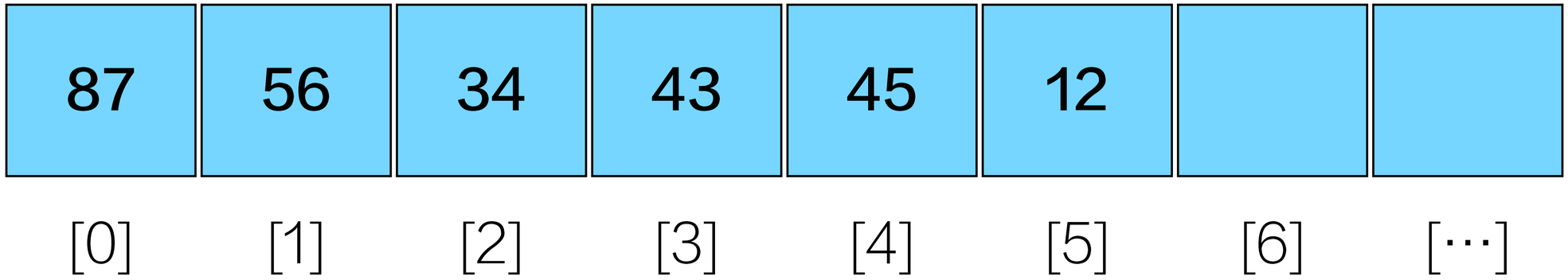
案例：线性表/顺序表/test2.c

顺序表 – 遍历

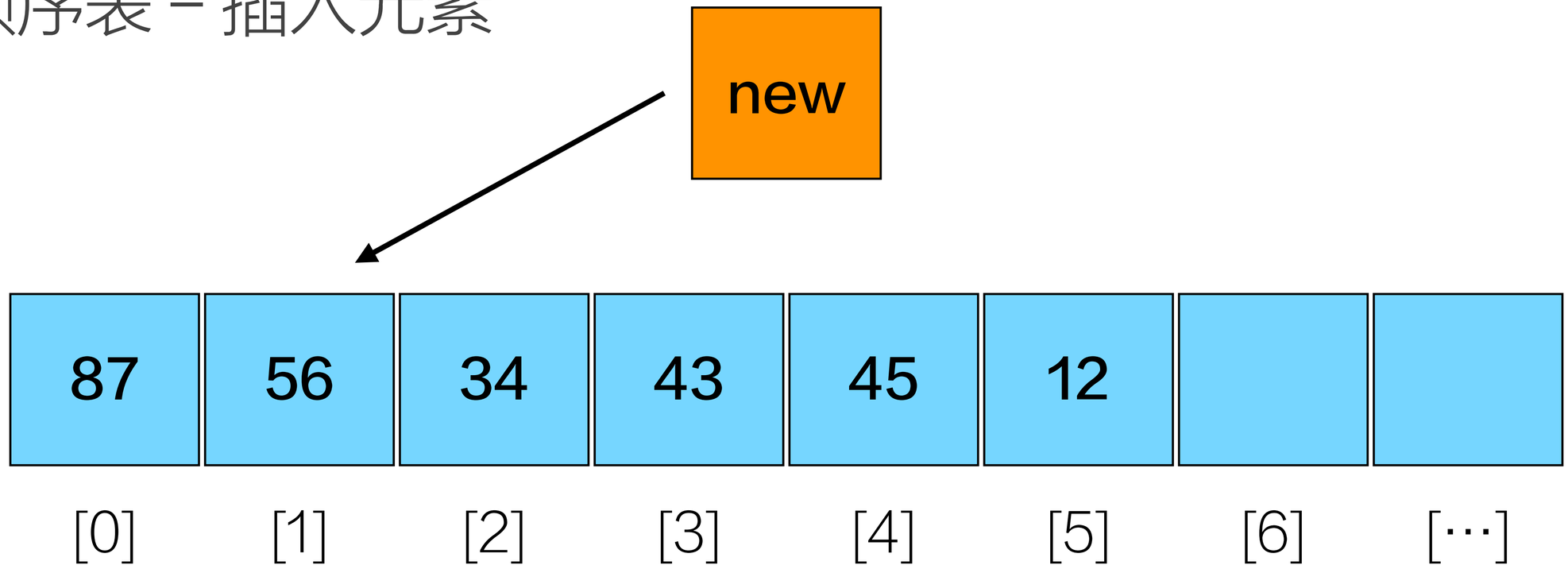
```
void listElem(SeqList *L)
{
    for (int i = 0; i < L->length; i++)
    {
        printf("%d ", L->data[i]);
    }
    printf("\n");
}
```

案例：线性表/顺序表/test3.c

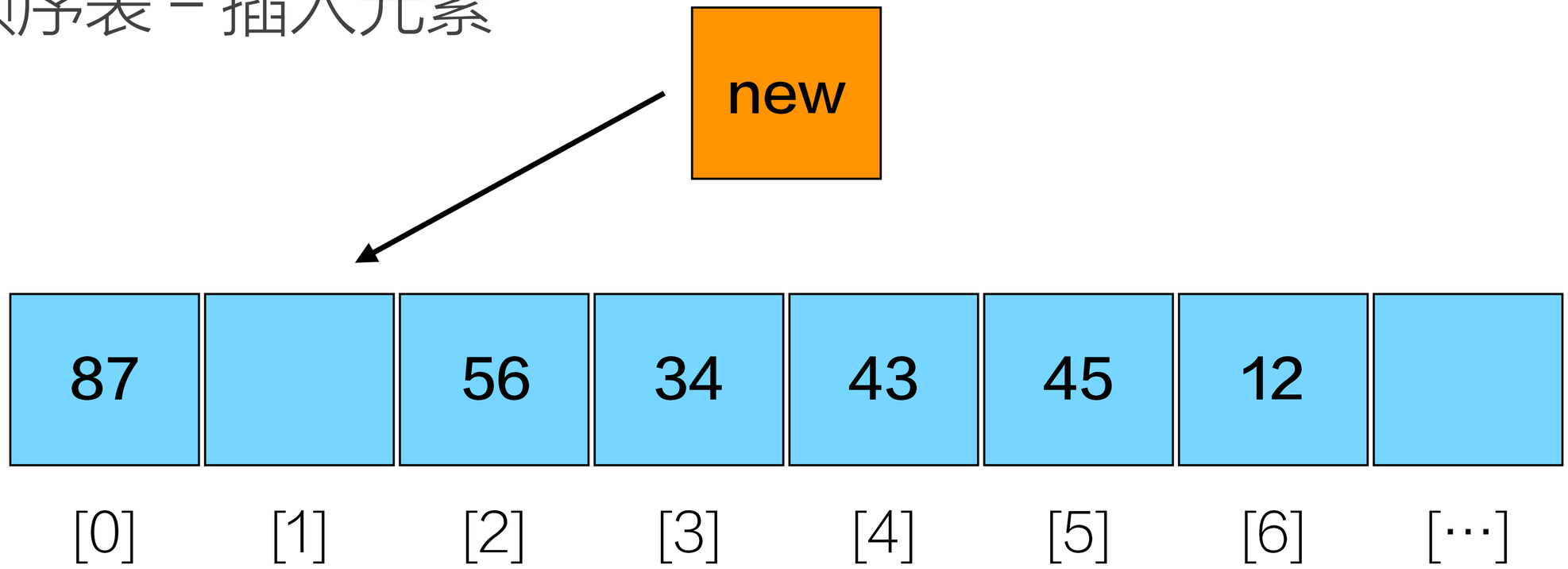
顺序表 – 插入元素



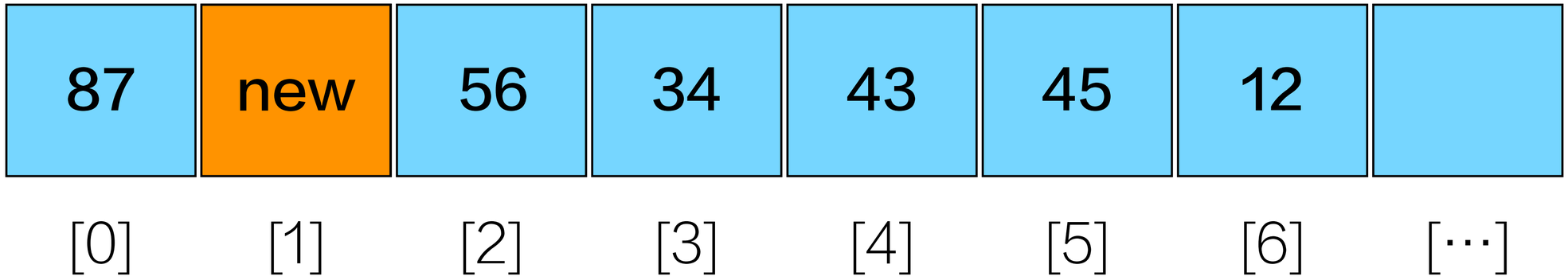
顺序表 – 插入元素



顺序表 – 插入元素



顺序表 – 插入元素



顺序表 – 插入元素

```
int insertElem(SeqList *L, int pos, ElemType e)
{
    if (pos <= L->length)
    {
        for (int i = L->length-1; i >= pos-1; i--)
        {
            L->data[i+1] = L->data[i];
        }
        L->data[pos-1] = e;
        L->length++;
    }
    return 1;
}
```

案例：线性表/顺序表/test4.c

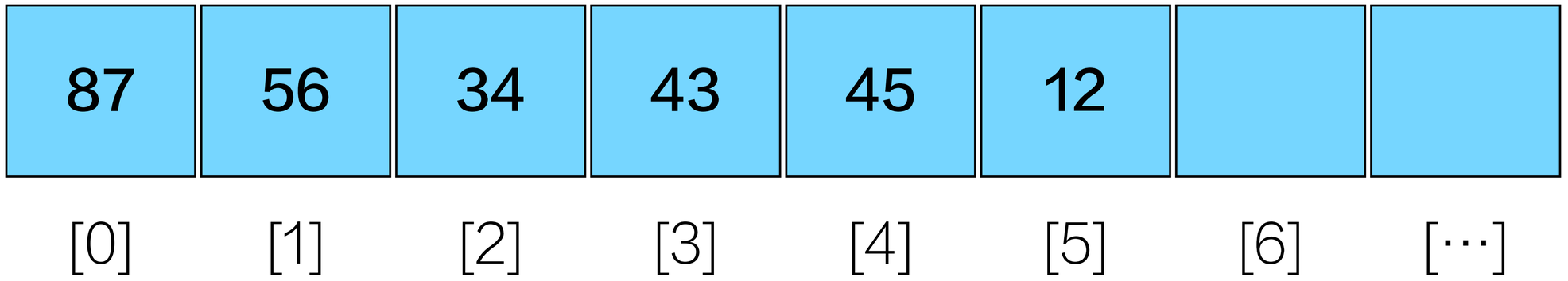
顺序表插入数据的最好时间复杂度是?

O(1)

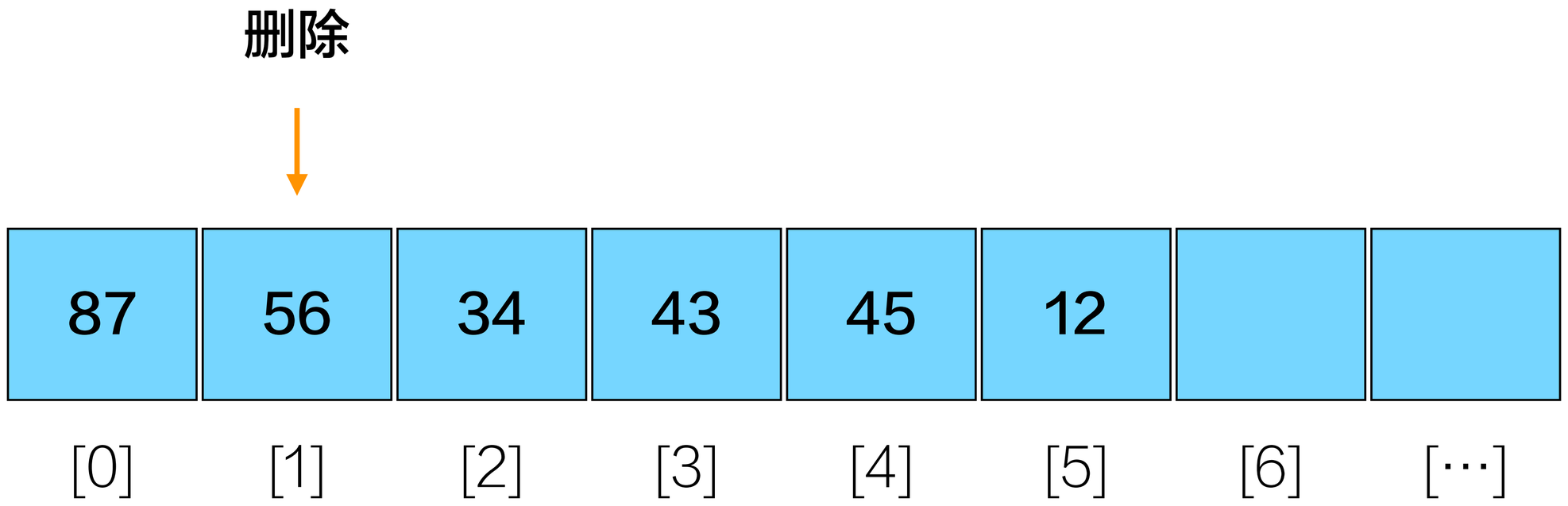
顺序表插入数据的最坏时间复杂度是?

O(n)

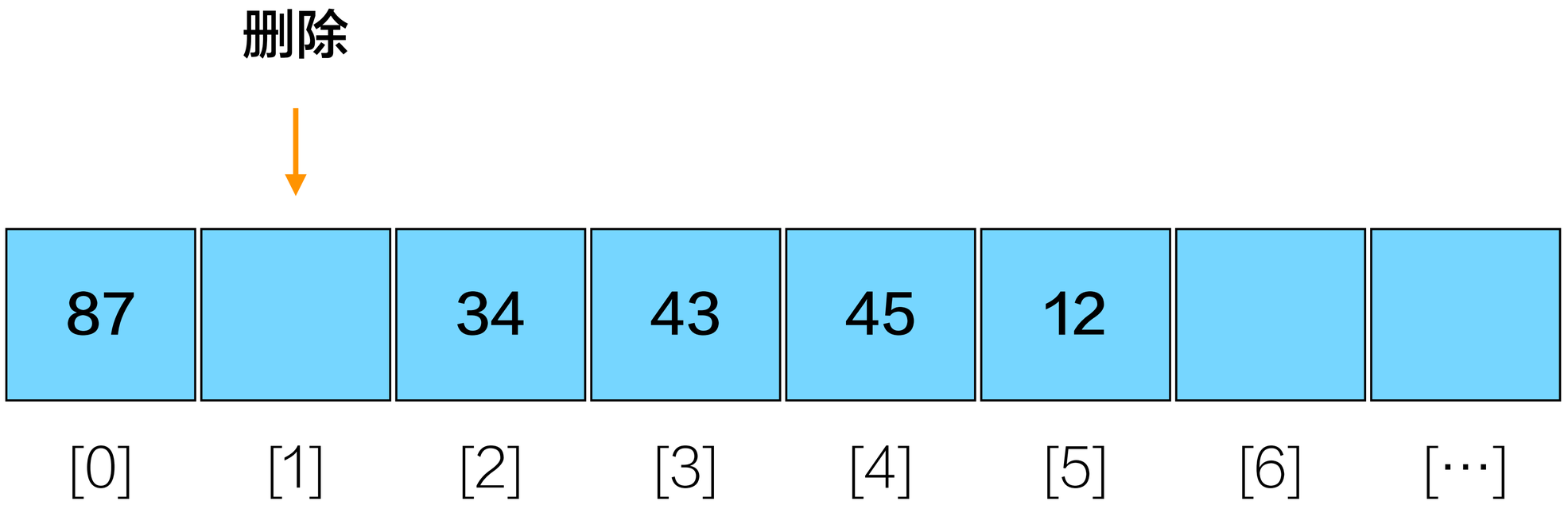
顺序表 – 删除元素



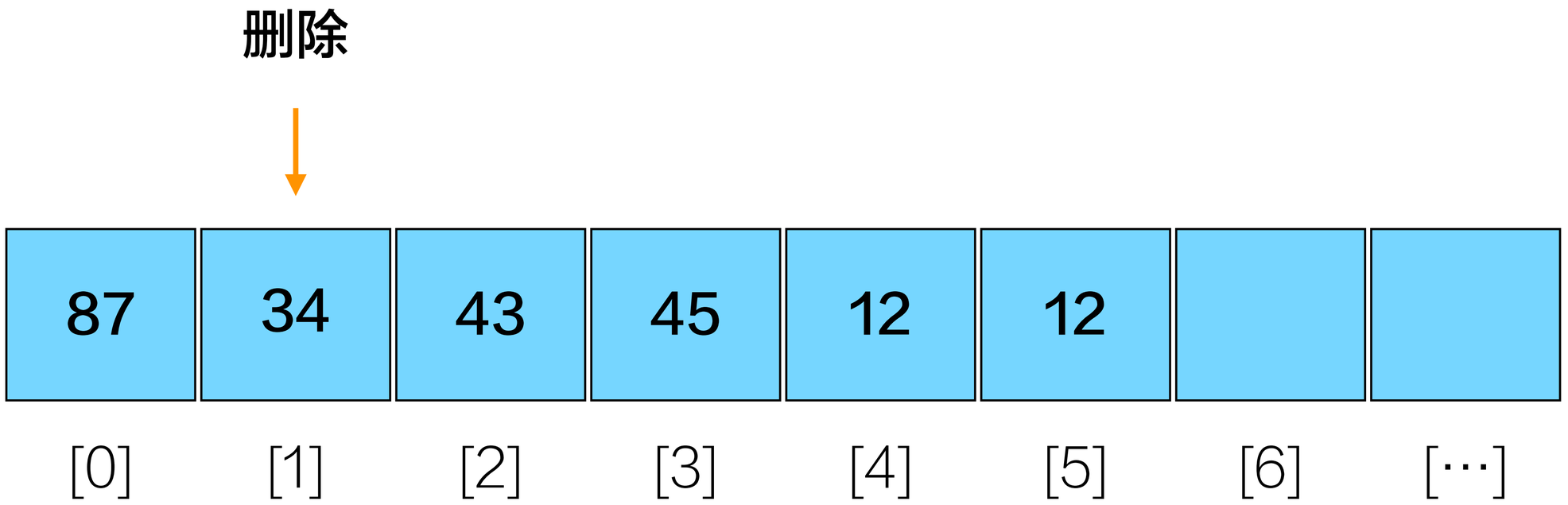
顺序表 – 删除元素



顺序表 – 删除元素



顺序表 – 删除元素



顺序表 - 删除元素

```
int deleteElem(SeqList *L, int pos, ElemType *e)
{
    *e = L->data[pos-1];
    if (pos < L->length)
    {
        for (int i = pos; i < L->length; i++)
        {
            L->data[i-1] = L->data[i];
        }
    }
    L->length--;
    return 1;
}
```

案例：线性表/顺序表/test5.c

顺序表 - 查找

```
int findElem(SeqList *L, ElemType e)
{
    for (int i = 0; i < L->length; i++)
    {
        if(L->data[i] == e)
        {
            return i + 1;
        }
    }
    return 0;
}
```

案例：线性表/顺序表/test6.c

顺序表 – 动态分配内存地址初始化

```
typedef struct{
    ElemType *data;
    int length;
}SeqList;

SeqList* initList()
{
    SeqList *L = (SeqList*)malloc(sizeof(SeqList));
    L->data = (ElemType*)malloc(sizeof(ElemType) * MAXSIZE);
    L->length = 0;
    return L;
}
```

案例：线性表/顺序表/test7.c

线性表的顺序存储形式（顺序表）有哪些点让你觉得很麻烦?

链表

线性表链式存储结构的特点是：用一组任意的存储单元存储线性表的数据元素（这组存储单元可以是连续的，也可以是不连续的）。

为了表示每个数据元素 a_i 与其直接后继数据元素 a_{i+1} 之间的逻辑关系，对数据元素 a_i 来说，除了其本身的信息之外，还需要存储一个指示其直接后继的信息（直接后继的存储位置）。这两部分信息组成数据元素 a_i 的存储映像，称为**节点（node）**。

结点包括两个域：其中存储数据元素信息的称为**数据域**；存储直接后继存储位置的域称为**指针域**。指针域中存储的信息称作指针或链。

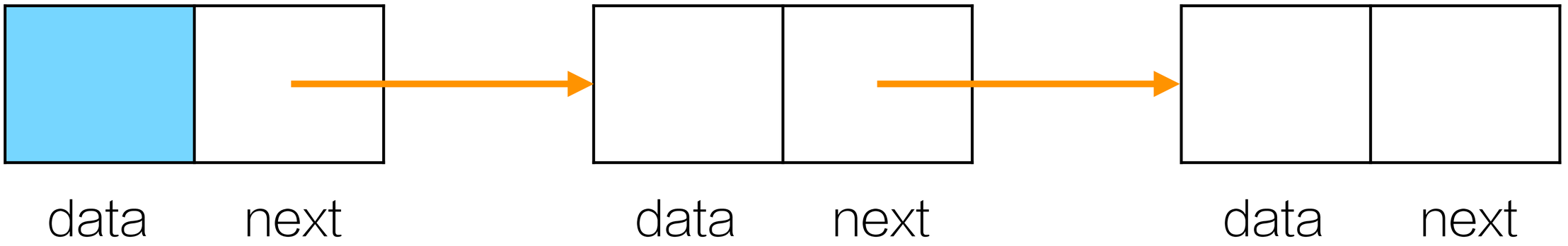
n 个结点 [a_i（$1 \leqslant i \leqslant n$）的存储映像] 链接成一个链表，即为线性表（$a_1, a_2, \cdots, a_n$）。

链表存储结构

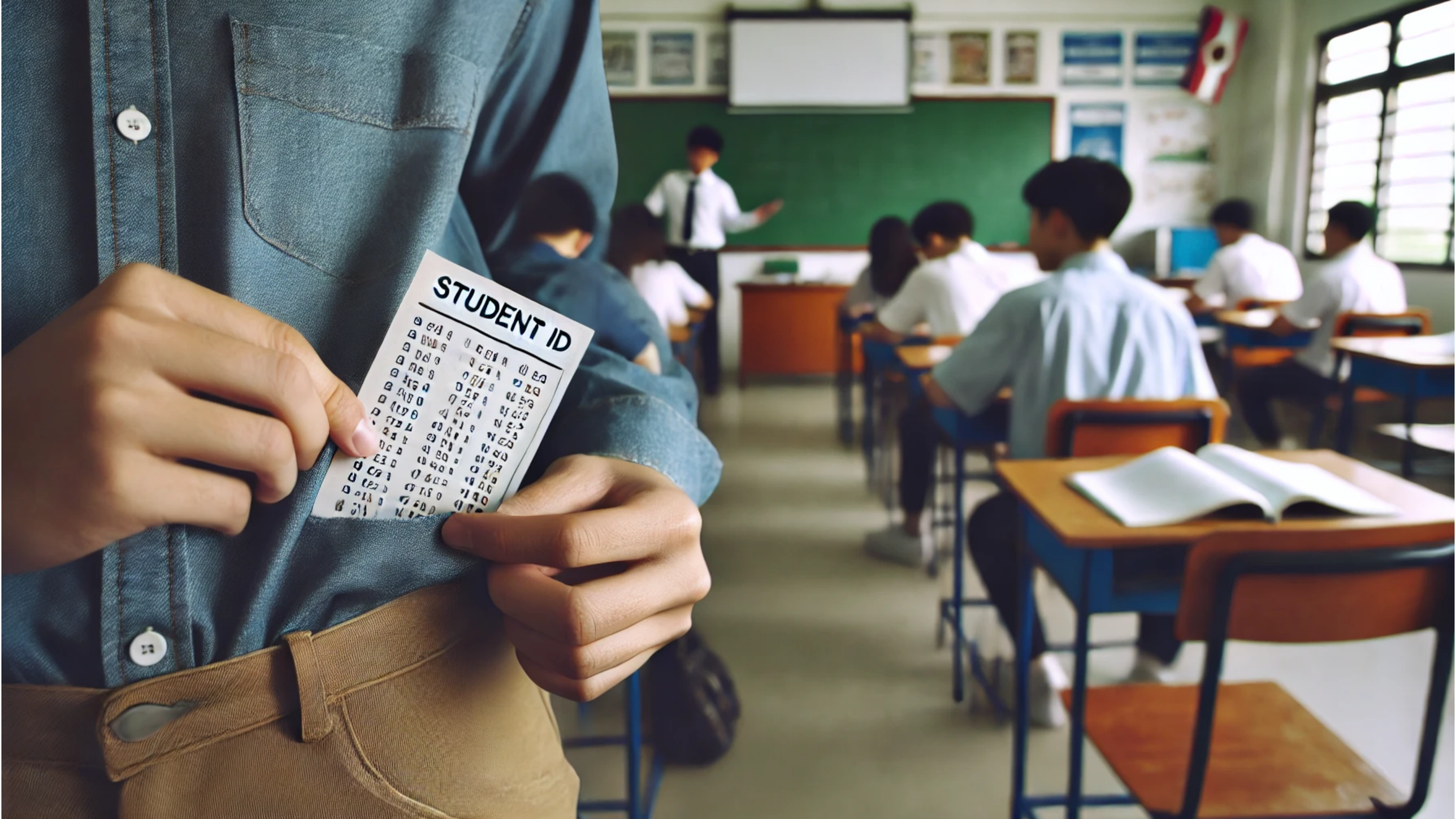
```
typedef int ElemType;


typedef struct node{
    ElemType data;
    struct node *next;
}Node;
```

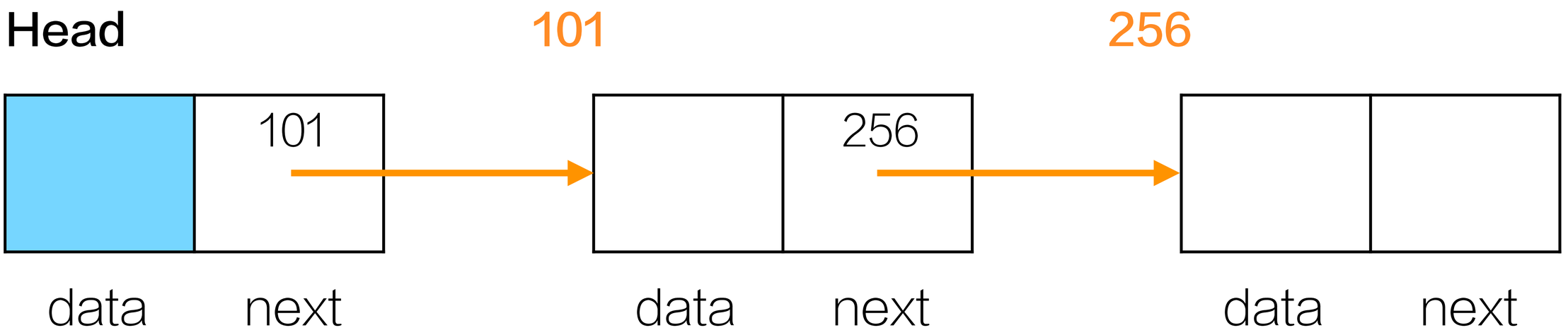







STUDENT ID

单链表

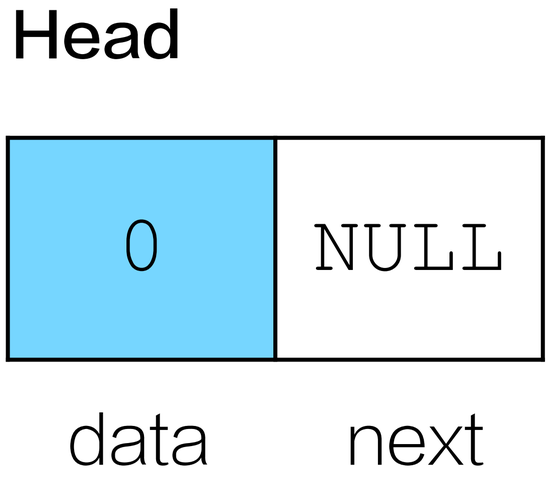


单链表 – 初始化

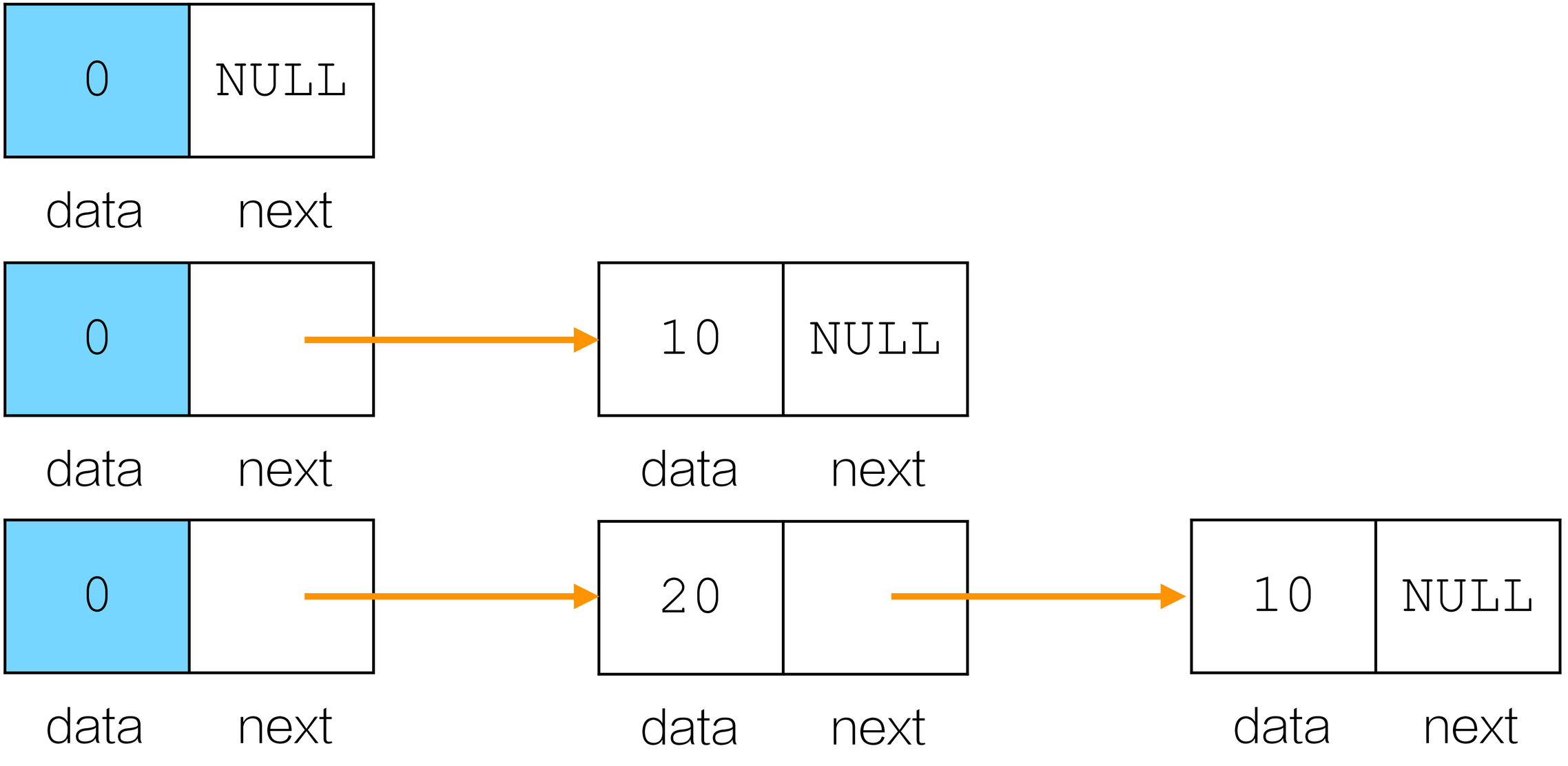
```
Node* initList()
{
    Node *head = (Node*)malloc(sizeof(Node));
    head->data = 0;
    head->next = NULL;
    return head;
}

int main()
{
    Node *list = initList();
    return 1;
}
```

案例：线性表/链表/test1.c



单链表 – 头插法


0
NULL
data
next
0
10
NULL
data
next
data
next
0
20
10
NULL
data
next
data
next
data
next

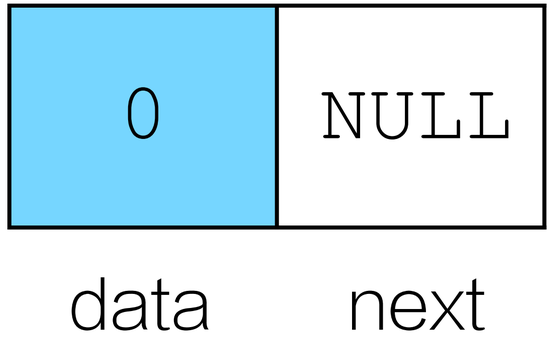
单链表 – 头插法

```
int insertHead(Node* L, ElemType e)
{
    Node *p = (Node*)malloc(sizeof(Node));
    p->data = e;
    p->next = L->next;
    L->next = p;
}
int main()
{
    Node *list = initList();
    insertHead(list, 10);
    insertHead(list, 20);
}
```

案例：线性表/链表/test2.c

单链表 – 头插法

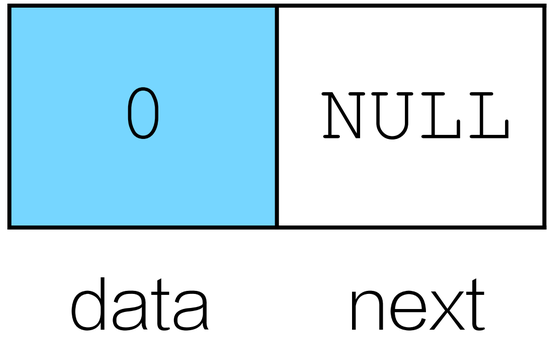
```
int insertHead(Node* L, ElemType e)
{
    Node *p = (Node*)malloc(sizeof(Node));
    p->data = e;
    p->next = L->next;
    L->next = p;
}
int main()
{
    Node *list = initList();
    insertHead(list, 10);
    insertHead(list, 20);
}
```



案例：线性表/链表/test2.c

单链表 – 头插法

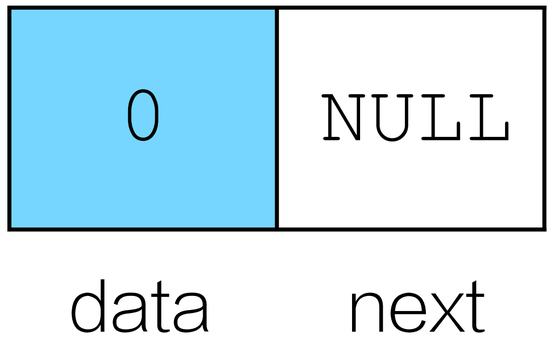
```
int insertHead(Node* L, ElemType e)
{
    Node *p = (Node*)malloc(sizeof(Node));
    p->data = e;
    p->next = L->next;
    L->next = p;
}
int main()
{
    Node *list = initList();
    insertHead(list, 10);
    insertHead(list, 20);
}
```

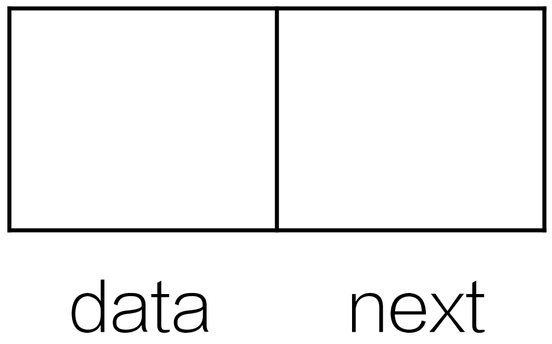


案例：线性表/链表/test2.c

单链表 – 头插法

```
int insertHead(Node* L, ElemType e)
{
    Node *p = (Node*)malloc(sizeof(Node));
    p->data = e;
    p->next = L->next;
    L->next = p;
}
int main()
{
    Node *list = initList();
    insertHead(list, 10);
    insertHead(list, 20);
}
```

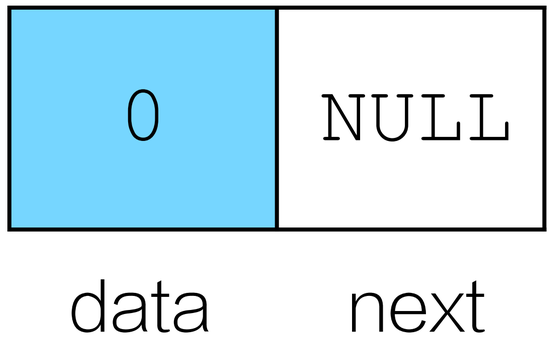


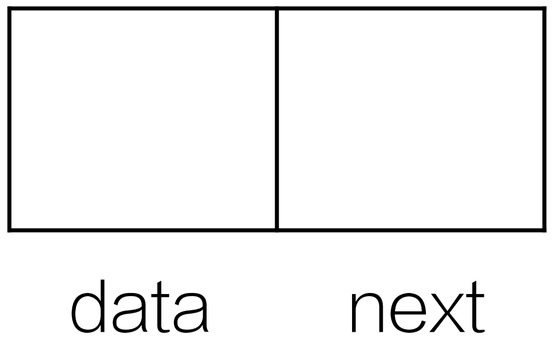


案例：线性表/链表/test2.c

单链表 – 头插法

```
int insertHead(Node* L, ElemType e)
{
    Node *p = (Node*)malloc(sizeof(Node));
    p->data = e;
    p->next = L->next;
    L->next = p;
}
int main()
{
    Node *list = initList();
    insertHead(list, 10);
    insertHead(list, 20);
}
```

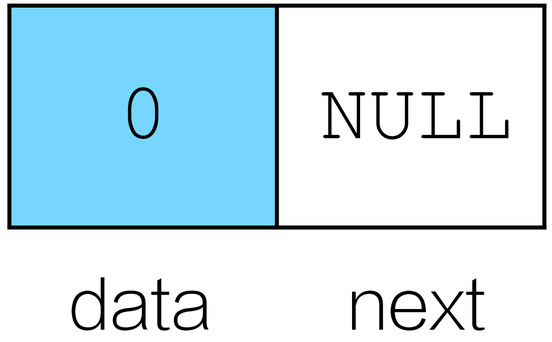


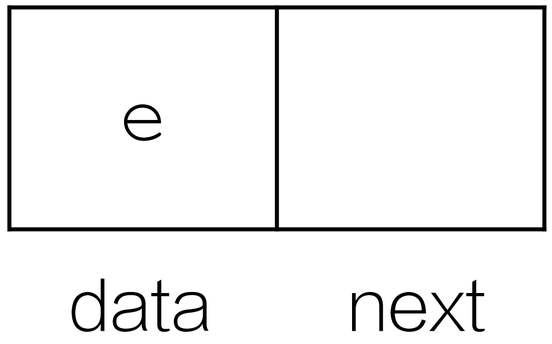


案例：线性表/链表/test2.c

单链表 – 头插法

```
int insertHead(Node* L, ElemType e)
{
    Node *p = (Node*)malloc(sizeof(Node));
    p->data = e;
    p->next = L->next;
    L->next = p;
}
int main()
{
    Node *list = initList();
    insertHead(list, 10);
    insertHead(list, 20);
}
```

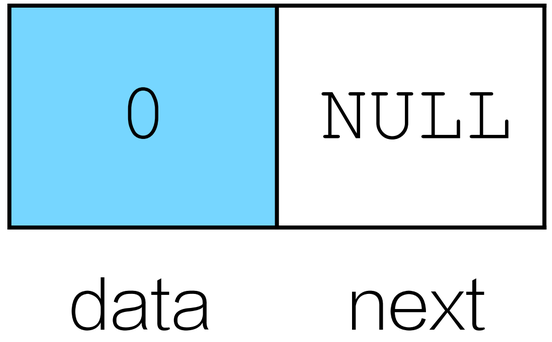


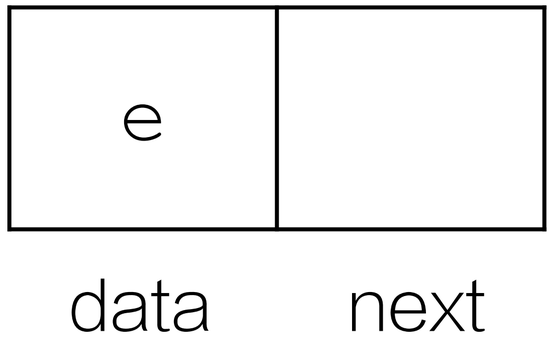


案例：线性表/链表/test2.c

单链表 – 头插法

```
int insertHead(Node* L, ElemType e)
{
    Node *p = (Node*)malloc(sizeof(Node));
    p->data = e;
    p->next = L->next;
    L->next = p;
}
int main()
{
    Node *list = initList();
    insertHead(list, 10);
    insertHead(list, 20);
}
```

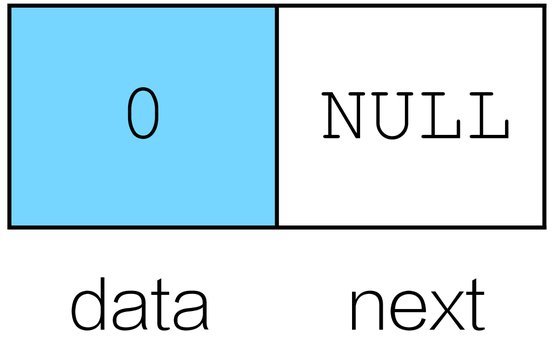


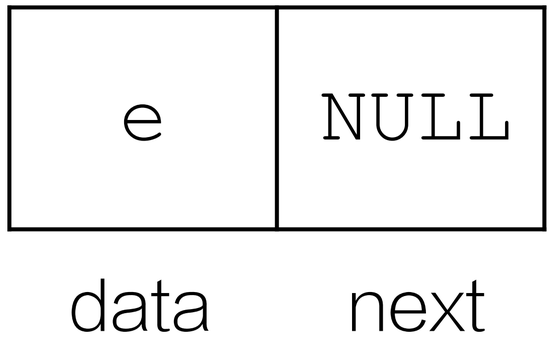


案例：线性表/链表/test2.c

单链表 – 头插法

```
int insertHead(Node* L, ElemType e)
{
    Node *p = (Node*)malloc(sizeof(Node));
    p->data = e;
    p->next = L->next;
    L->next = p;
}
int main()
{
    Node *list = initList();
    insertHead(list, 10);
    insertHead(list, 20);
}
```

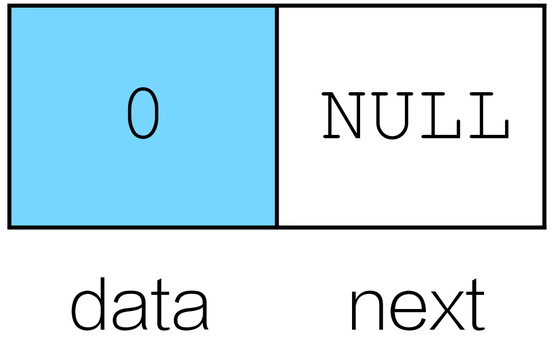


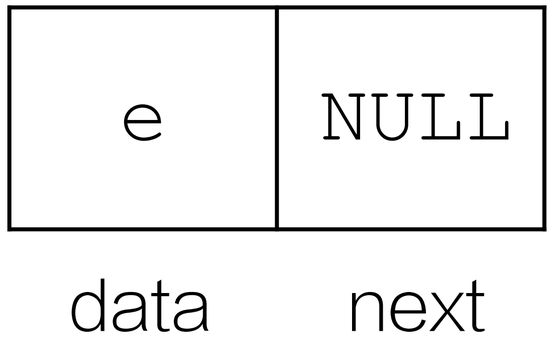


案例：线性表/链表/test2.c

单链表 – 头插法

```
int insertHead(Node* L, ElemType e)
{
    Node *p = (Node*)malloc(sizeof(Node));
    p->data = e;
    p->next = L->next;
    L->next = p;
}
int main()
{
    Node *list = initList();
    insertHead(list, 10);
    insertHead(list, 20);
}
```

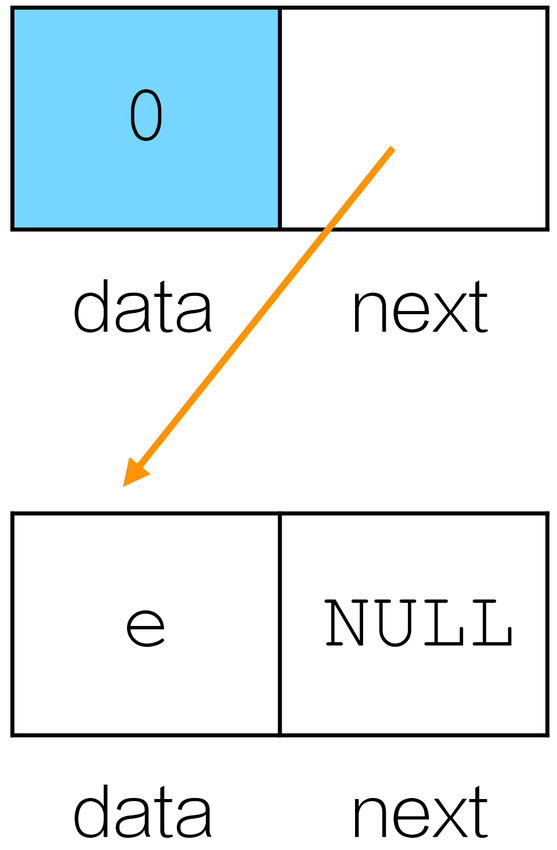




案例：线性表/链表/test2.c

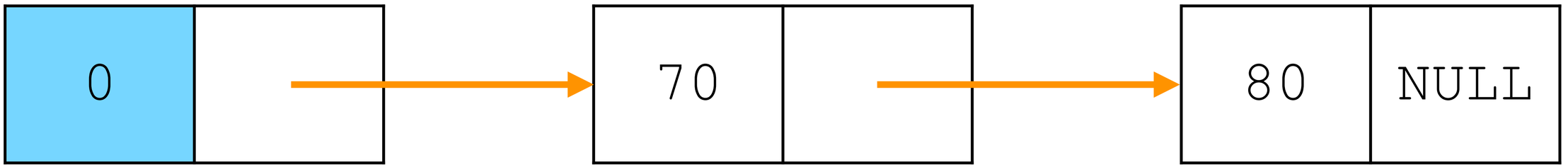
单链表 – 头插法

```
int insertHead(Node* L, ElemType e)
{
    Node *p = (Node*)malloc(sizeof(Node));
    p->data = e;
    p->next = L->next;
    L->next = p;
}
int main()
{
    Node *list = initList();
    insertHead(list, 10);
    insertHead(list, 20);
}
```

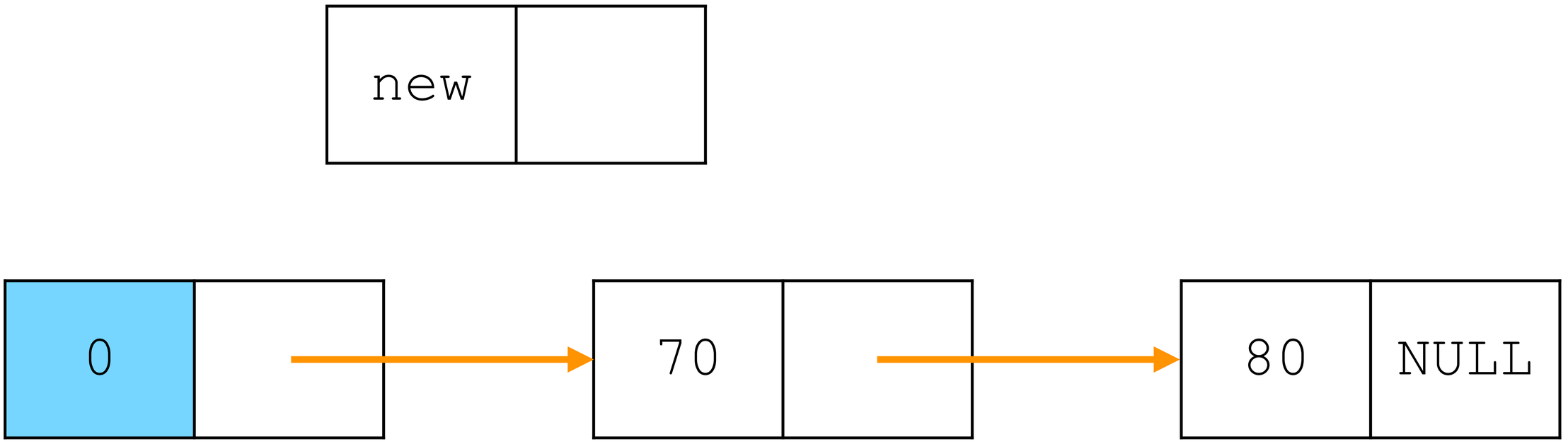


案例：线性表/链表/test2.c

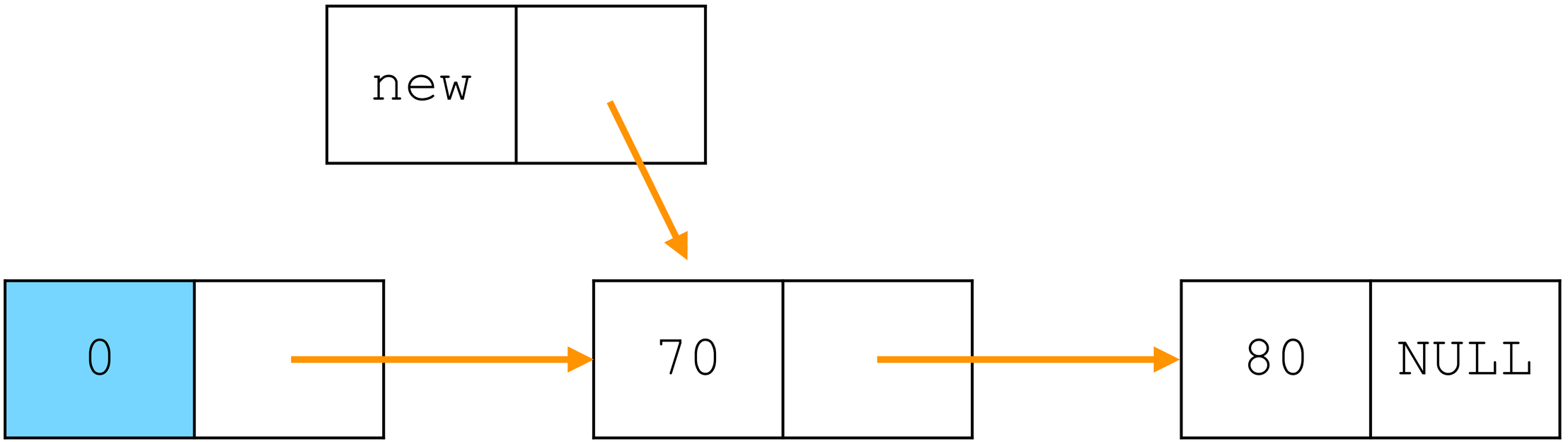
单链表 – 头插法



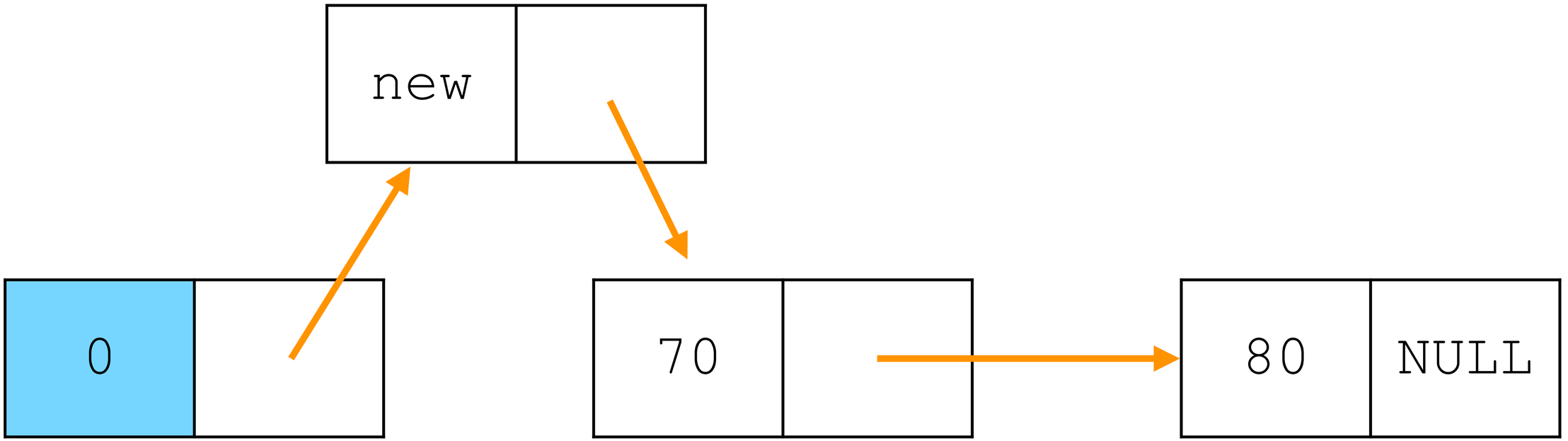
单链表 – 头插法



单链表 – 头插法



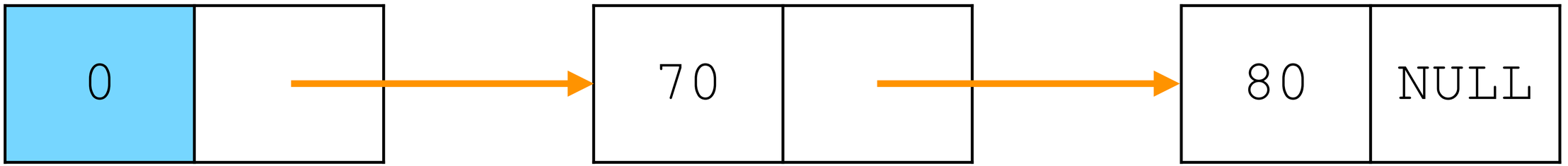
单链表 – 头插法



单链表 – 遍历

```
void listNode(Node* L)
{
    Node *p = L->next;
    while(p != NULL)
    {
        printf("%d\n", p->data);
        p = p->next;
    }
    printf("\n");
}
```

案例：线性表/链表/test3.c



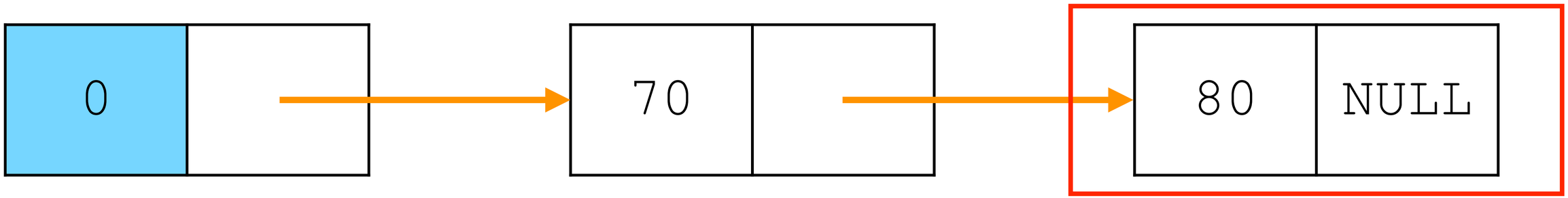
头插法的顺序和排列的顺序是相反的

单链表 - 尾插法

```
Node*  get_tail(Node  *L)
{
    Node *p = L;
    while(p->next != NULL)
    {
        p = p->next;
    }
    return p;
}
```

案例：线性表/链表/test4.c

先获取尾节点地址

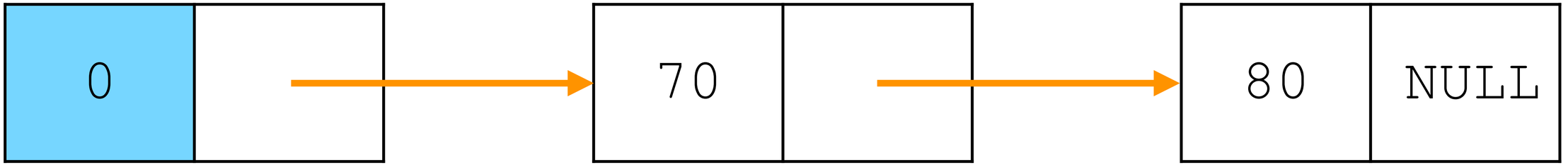


单链表 – 尾插法

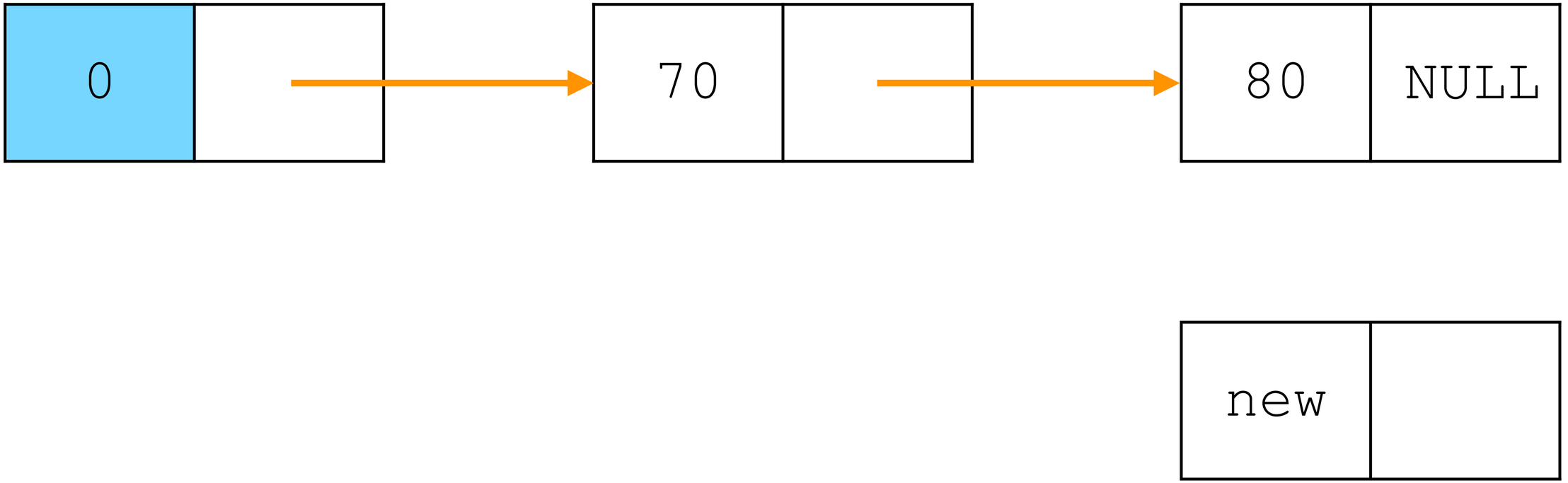
```
Node* insertTail(Node *tail, ElemType e)
{
    Node *p = (Node*)malloc(sizeof(Node));
    p->data = e;
    tail->next = p;
    p->next = NULL;
    return p;
}
```

案例：线性表/链表/test4.c

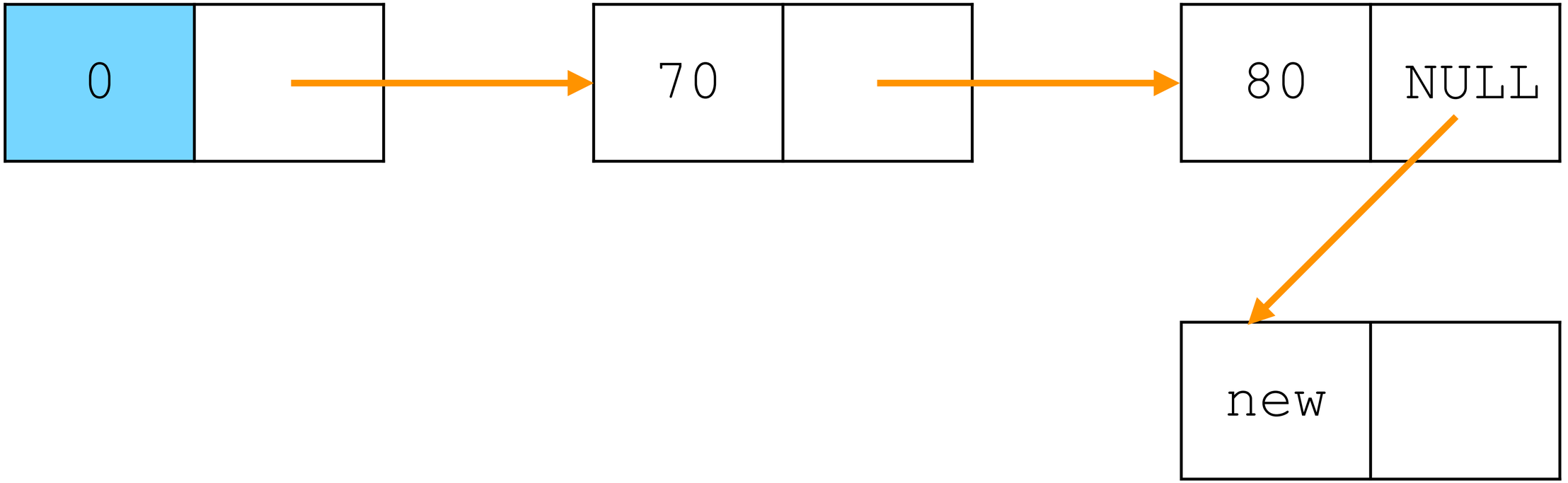
单链表 – 尾插法



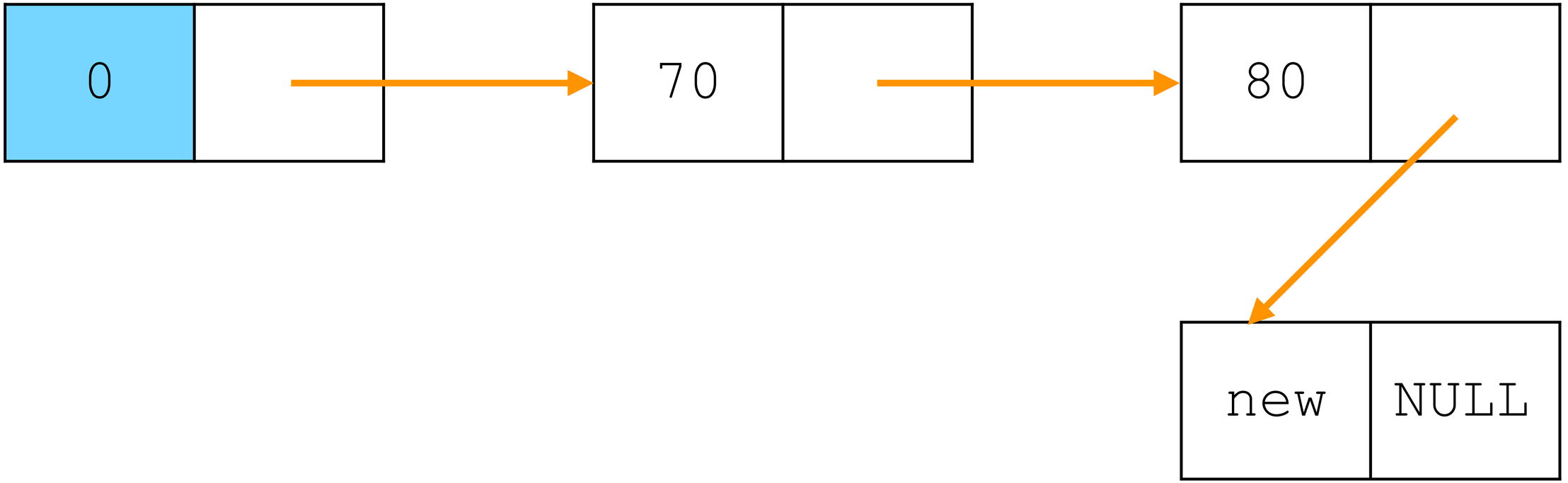
单链表 – 尾插法



单链表 – 尾插法

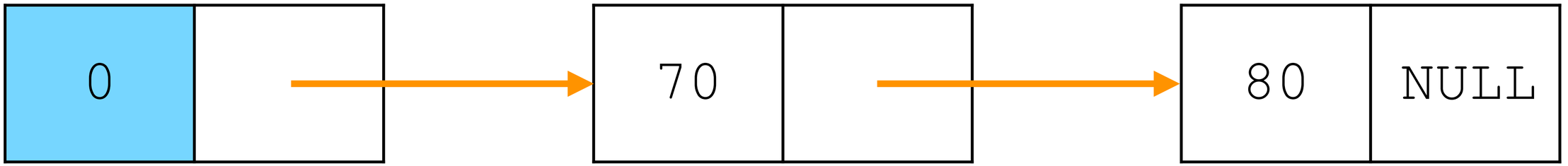


单链表 – 尾插法

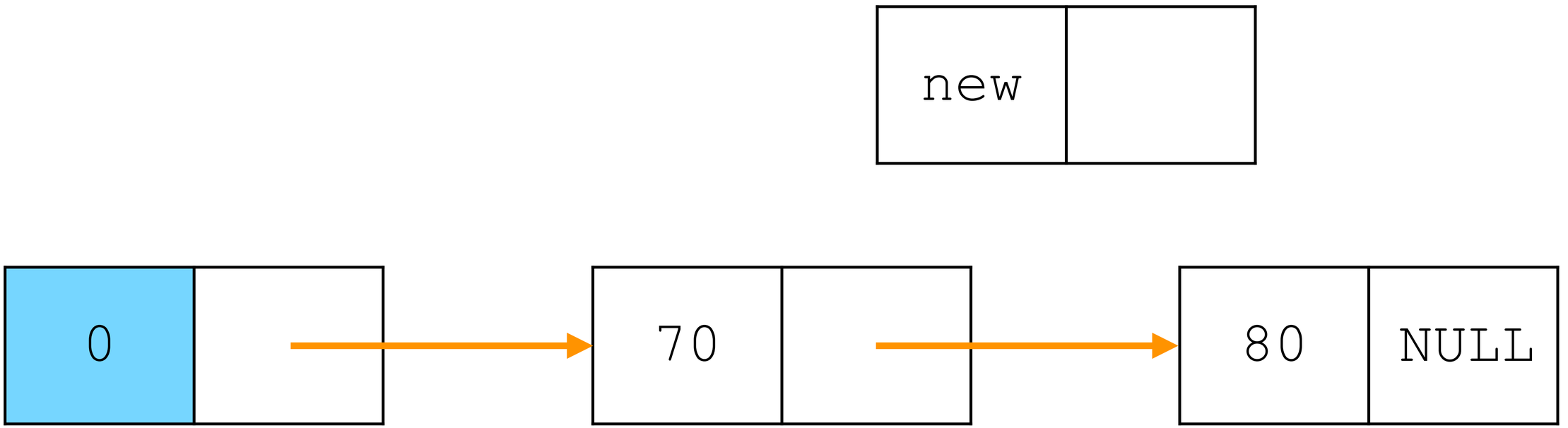


单链表 – 在指定位置插入数据

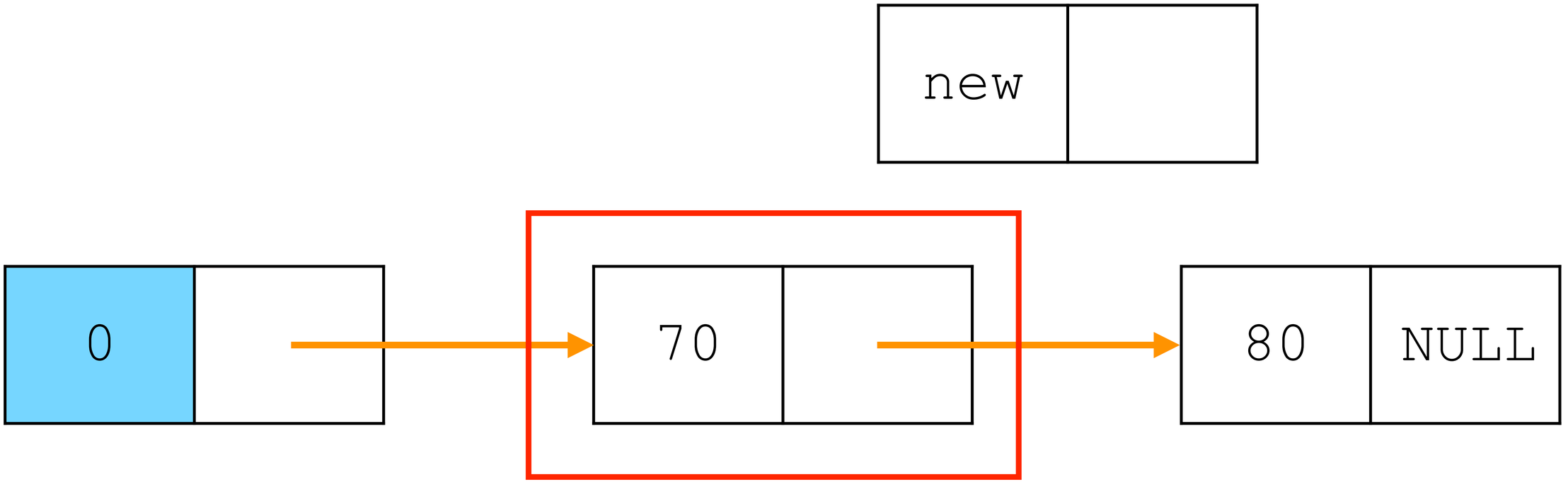
单链表 – 在指定位置插入数据



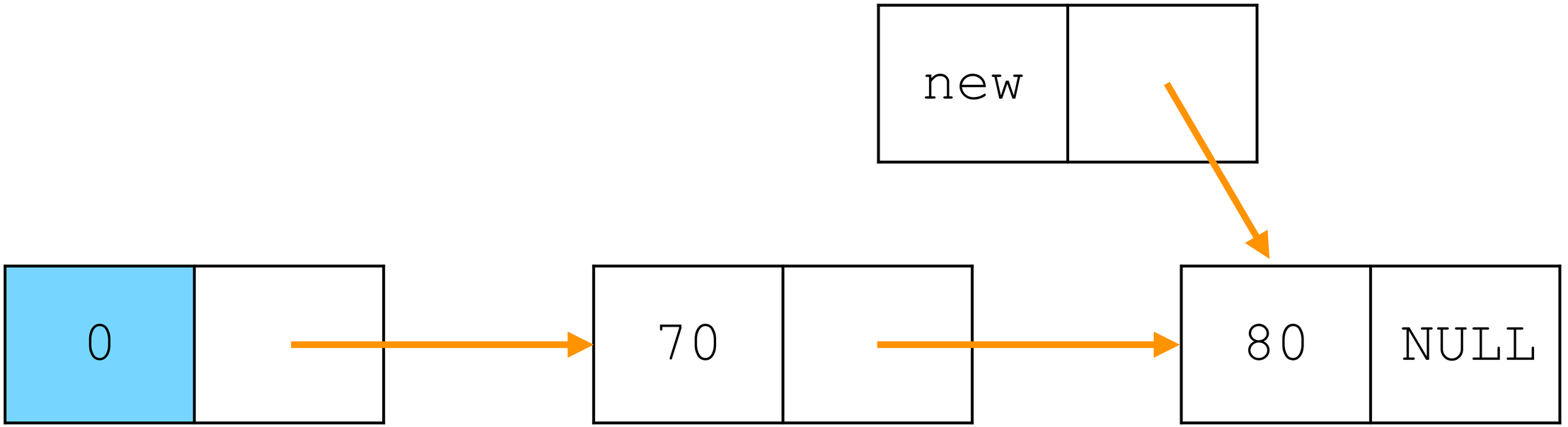
单链表 – 在指定位置插入数据



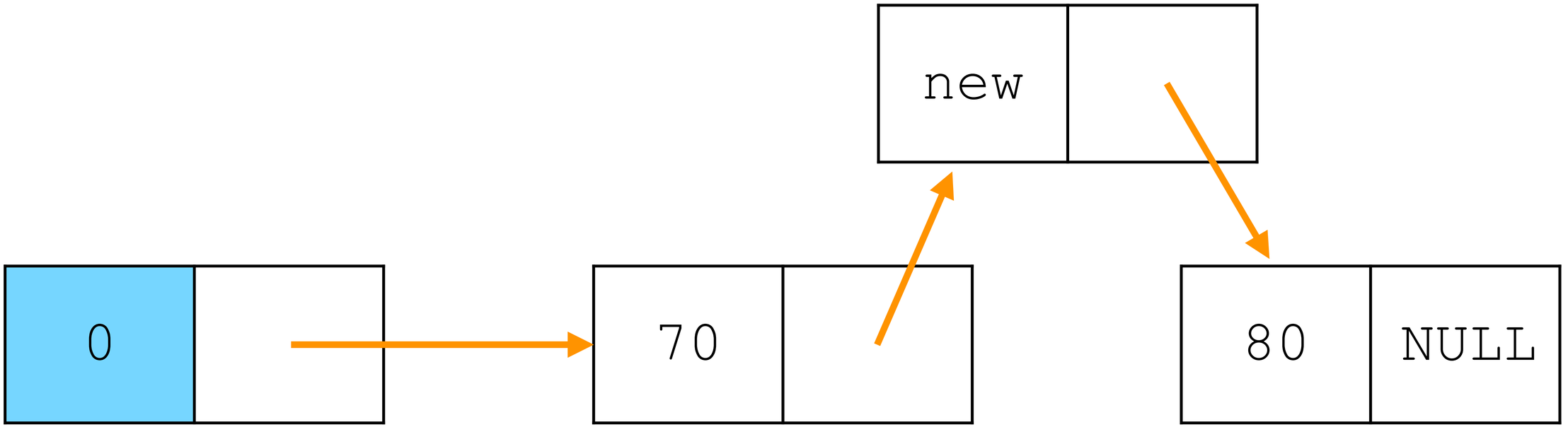
单链表 – 在指定位置插入数据



单链表 – 在指定位置插入数据



单链表 – 在指定位置插入数据



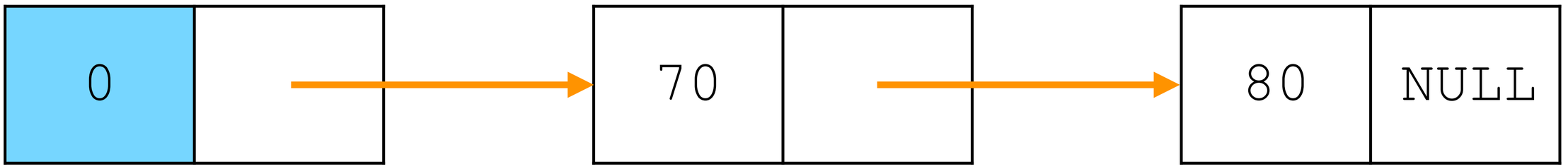
单链表 - 在指定位置插入数据

```c
int insertNode(Node *L, int pos, ElemType e)
{
    Node *p = L;
    int i = 0;
    while(i < pos-1)
    {
        p = p->next;
        i++;
        if (p == NULL)
        {
            return 0;
        }
    }

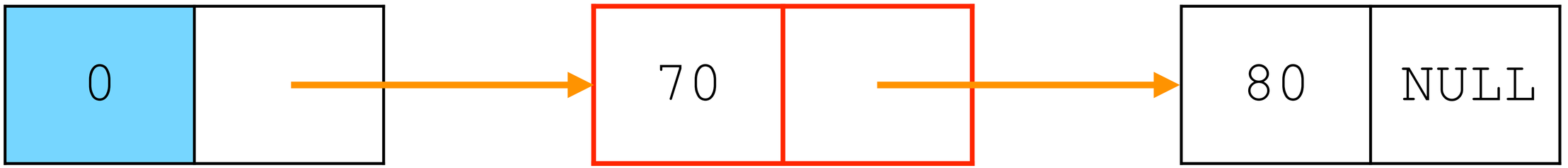

    Node *q = (Node*)malloc(sizeof(Node));
    q->data = e;
    q->next = p->next;
    p->next = q;
    return 1;
}
```

案例：线性表/链表/test5.c

单链表 – 删除节点

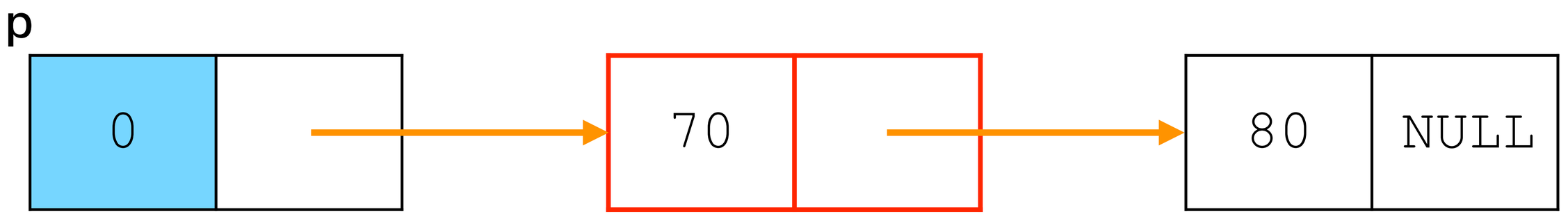


单链表 – 删除节点



单链表 – 删除节点

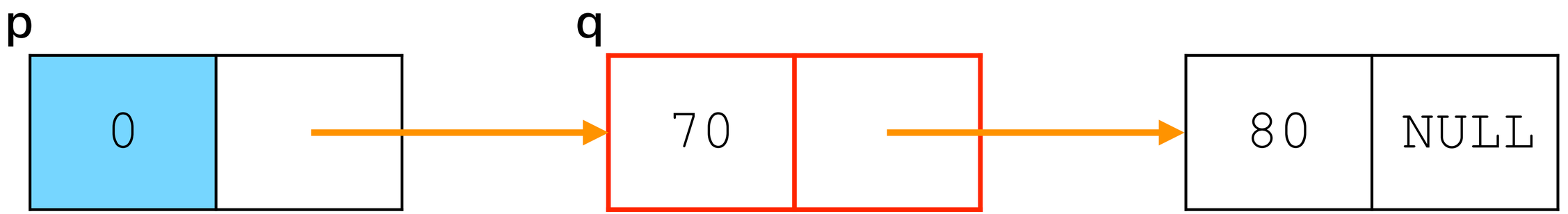
找到要删除节点的前置节点 p



单链表 – 删除节点

找到要删除节点的前置节点 p

用指针 q 记录要删除的节点

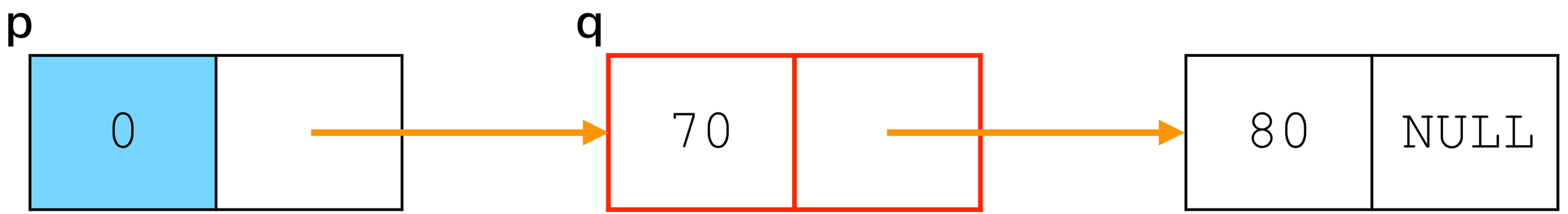


单链表 – 删除节点

找到要删除节点的前置节点 p

用指针 q 记录要删除的节点

通过改变 p 的后继节点实现删除

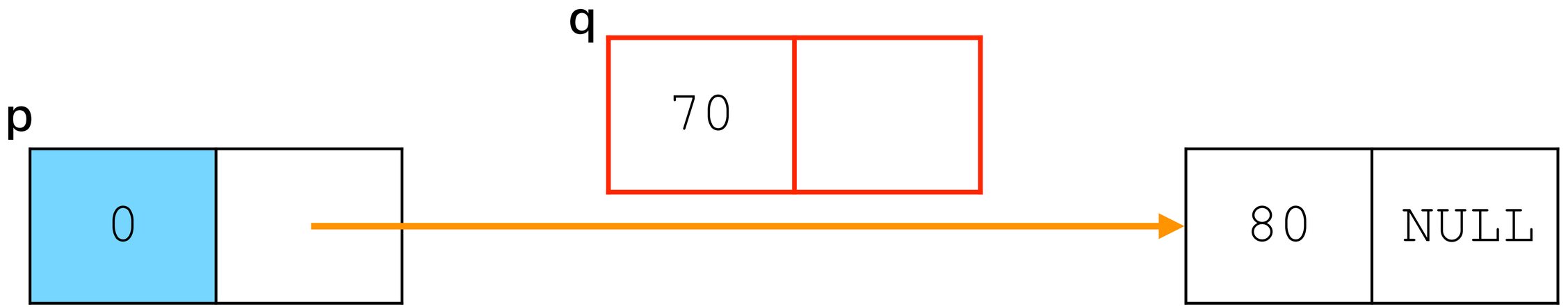


单链表 – 删除节点

找到要删除节点的前置节点 p

用指针 q 记录要删除的节点

通过改变 p 的后继节点实现删除



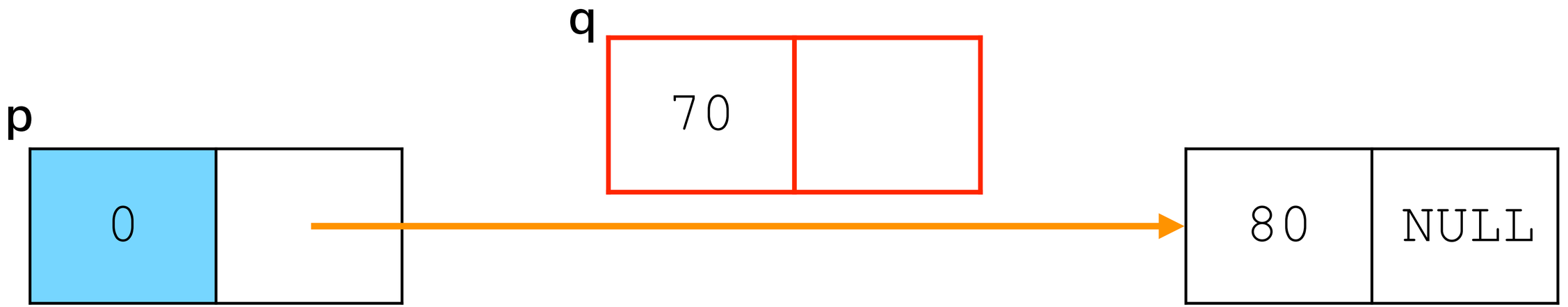
单链表 – 删除节点

找到要删除节点的前置节点 p

用指针 q 记录要删除的节点

通过改变 p 的后继节点实现删除

释放删除节点的空间



单链表 - 删除节点

```c
int deleteNode(Node *L, int pos)
{
    Node *p = L;
    int i = 0;
    while(i < pos-1)
    {
        p = p->next;
        i++;
        if (p == NULL)
        {
            return 0;
        }
    }

    if(p->next == NULL)
    {
        printf("要删除的位置错误\n");
        return 0;
    }

    Node *q = p->next;
    p->next = q->next;
    free(q);
    return 1;
}
```

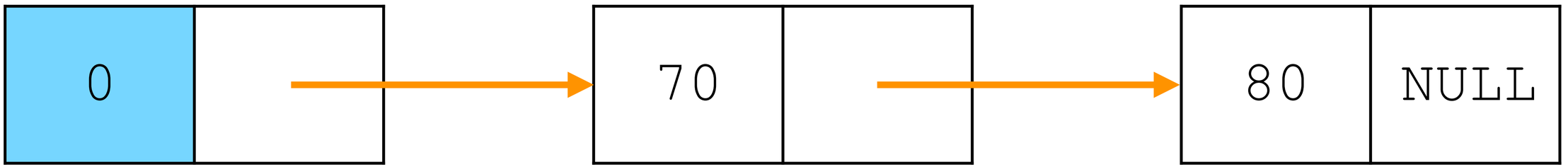
案例：线性表/链表/test6.c

单链表 – 获取链表长度

```
int listLength(Node *L)
{
    Node *p = L;
    int len = 0;
    while(p != NULL)
    {
        p = p->next;
        len++;
    }
    return len;
}
```

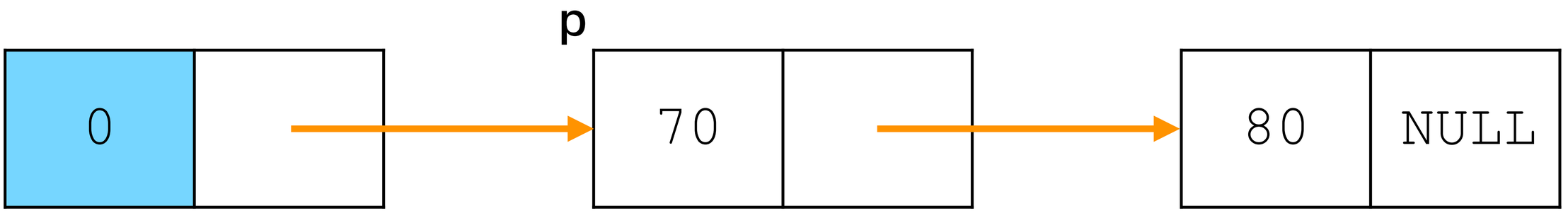
案例：线性表/链表/test7.c

单链表 – 释放链表



单链表 – 释放链表

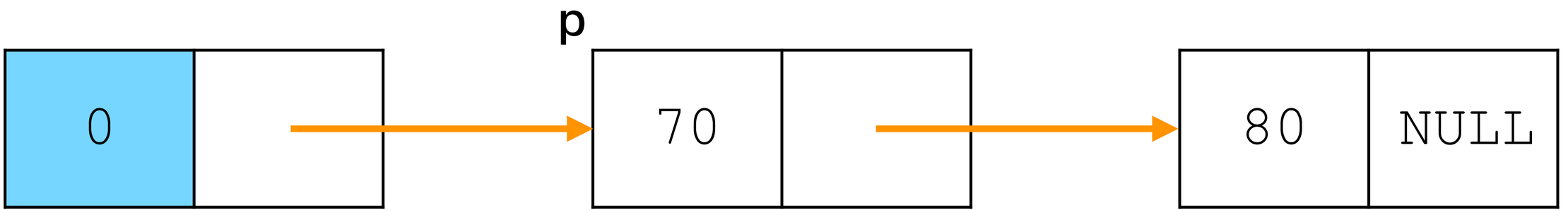
指针 p 指向头节点后的第一个节点



单链表 – 释放链表

指针 p 指向头节点后的第一个节点

判断指针 p 是否指向空节点

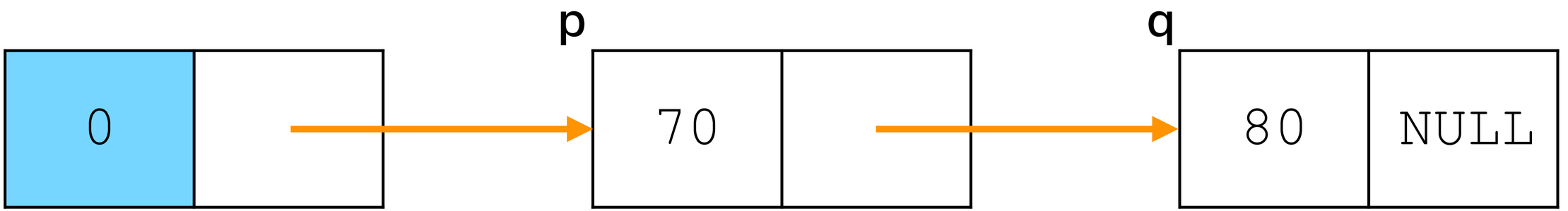


单链表 – 释放链表

指针 p 指向头节点后的第一个节点

判断指针 p 是否指向空节点

如果 p 不为空，用指针 q 记录指针 p 的后继节点。



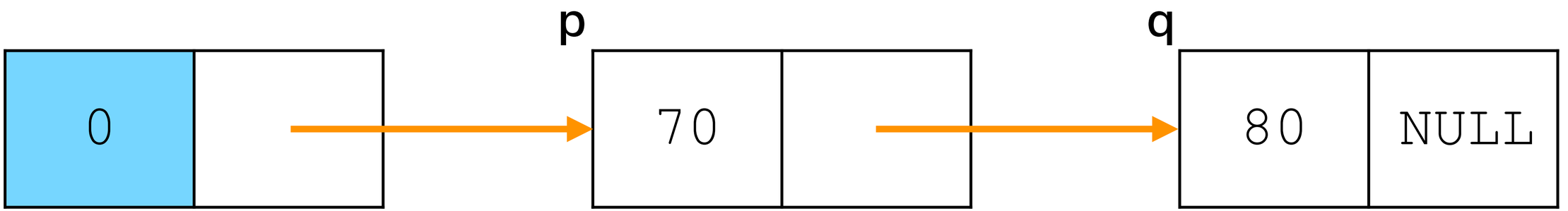
单链表 – 释放链表

指针 p 指向头节点后的第一个节点

判断指针 p 是否指向空节点

如果 p 不为空，用指针 q 记录指针 p 的后继节点。

释放指针 p 指向的节点



单链表 – 释放链表

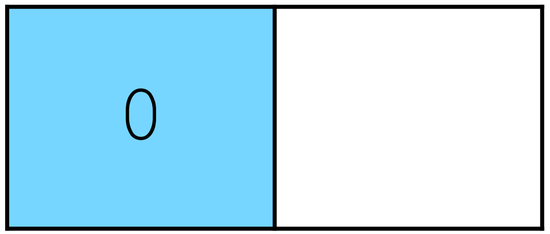
指针 p 指向头节点后的第一个节点

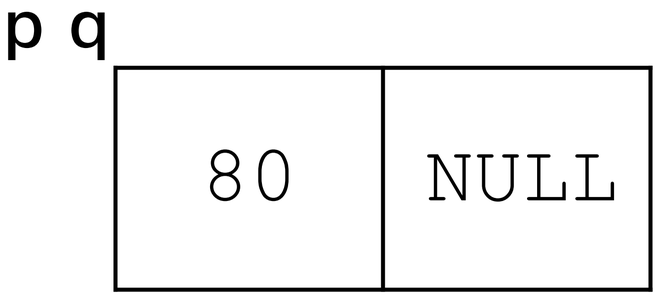
判断指针 p 是否指向空节点

如果 p 不为空，用指针 q 记录指针 p 的后继节点。

释放指针 p 指向的节点

指针 p 和指针 q 指向同一个节点，循环上面的操作。





单链表 – 释放链表

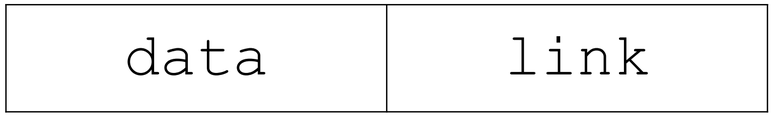
```
void freeList(Node *L)
{
    Node *p = L->next;
    Node *q;

    while(p != NULL)
    {
        q = p->next;
        free(p);
        p = q;
    }
    L->next = NULL;
}
```

案例：线性表/链表/test8.c

单链表应用

[2019]　（ 15分 ）已知一个带有表头节点的单链表，节点结构为



假设该链表只给出了头指针 list 。在不改变链表的前提下，请设计一个尽可能高效的算法，查找链表中倒数第 k 个位置上的结点（ k 为正整数 ）。若查找成功，算法输出该节点的 data 域的值，并返回 1 ；否则，只返回 0 。要求：

1 ）描述算法的基本思想；

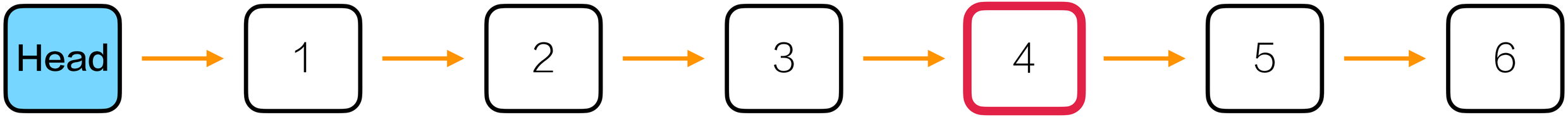
2 ）描述算法的详细实现步骤；

3 ）根据设计思想和实现步骤，采用程序设计语言描述算法（ 使用 C 、C++、或 Java 语言实现 ），关键之处请给出简要注释

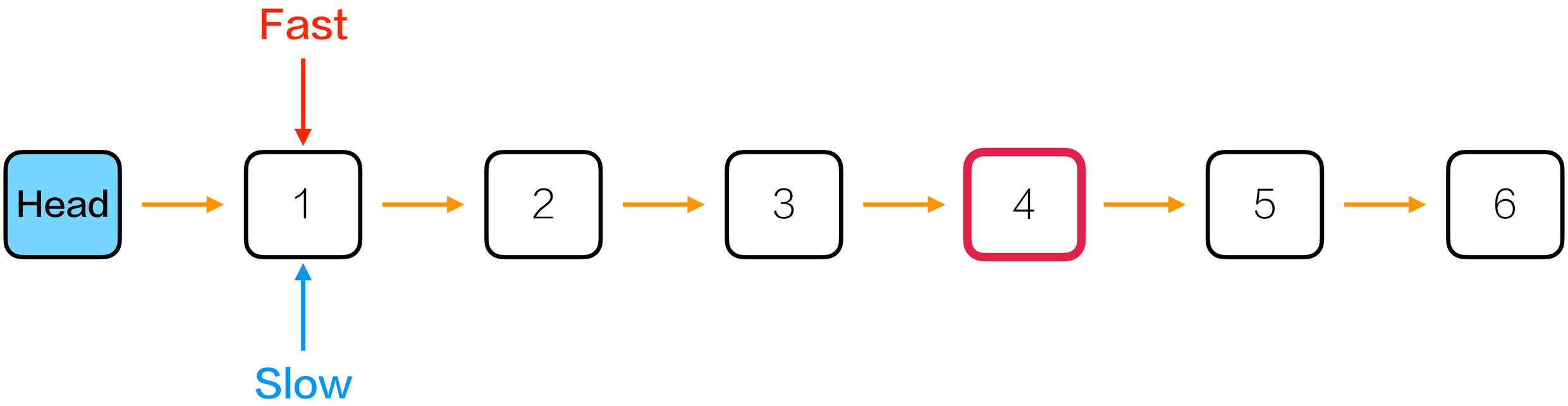
查找倒数第 k 个节点



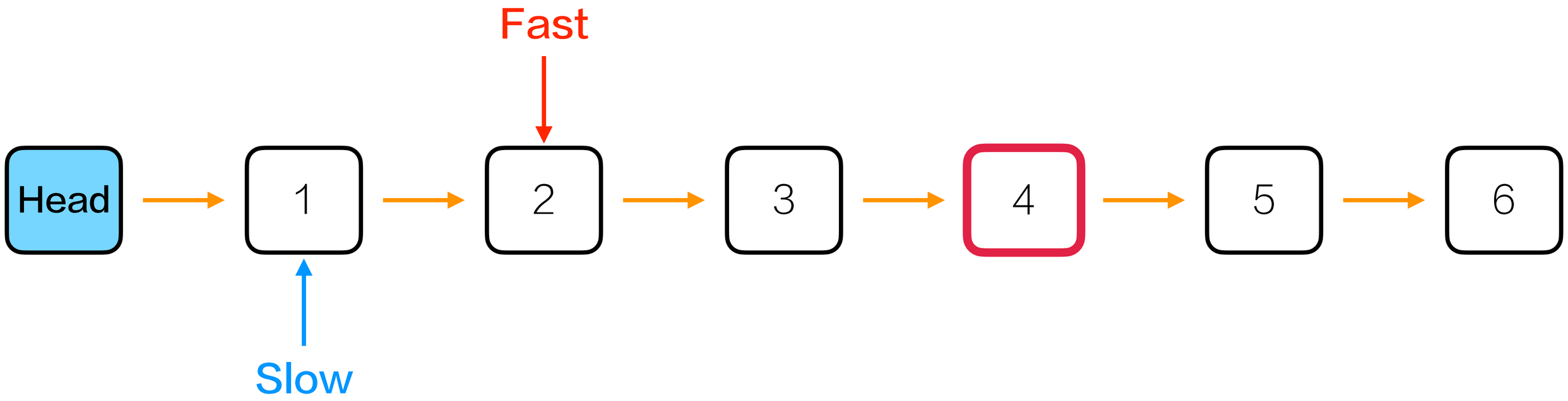
假设倒数第 3 个节点



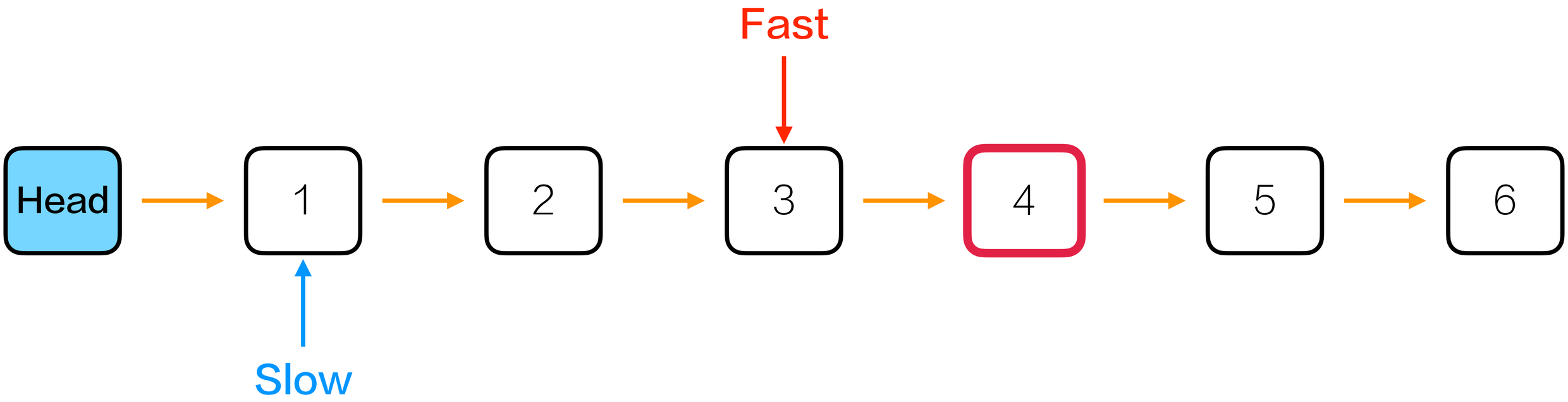
双指针（快慢指针）



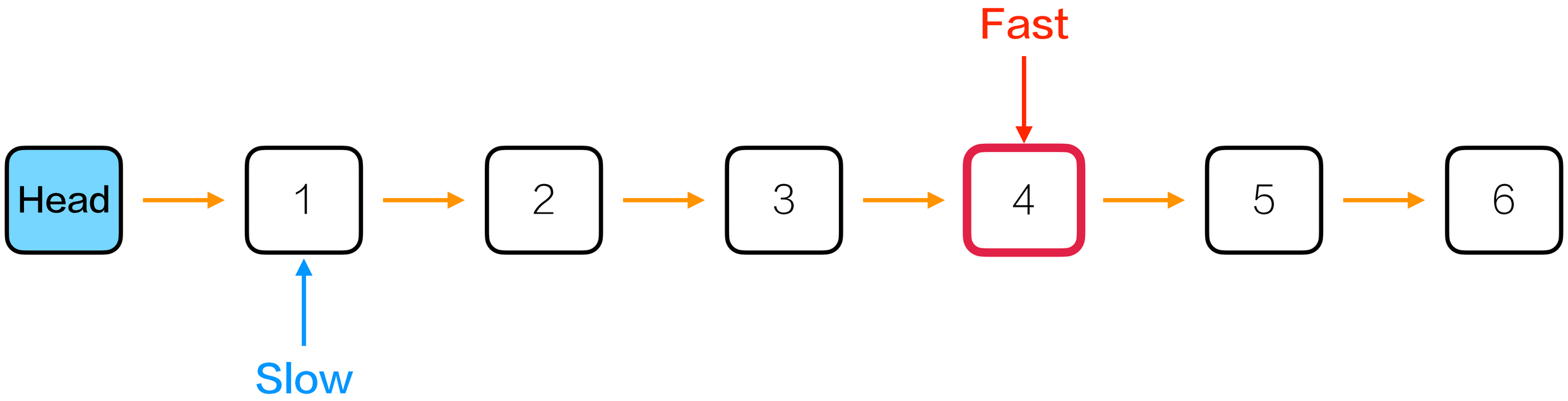
双指针（快慢指针）



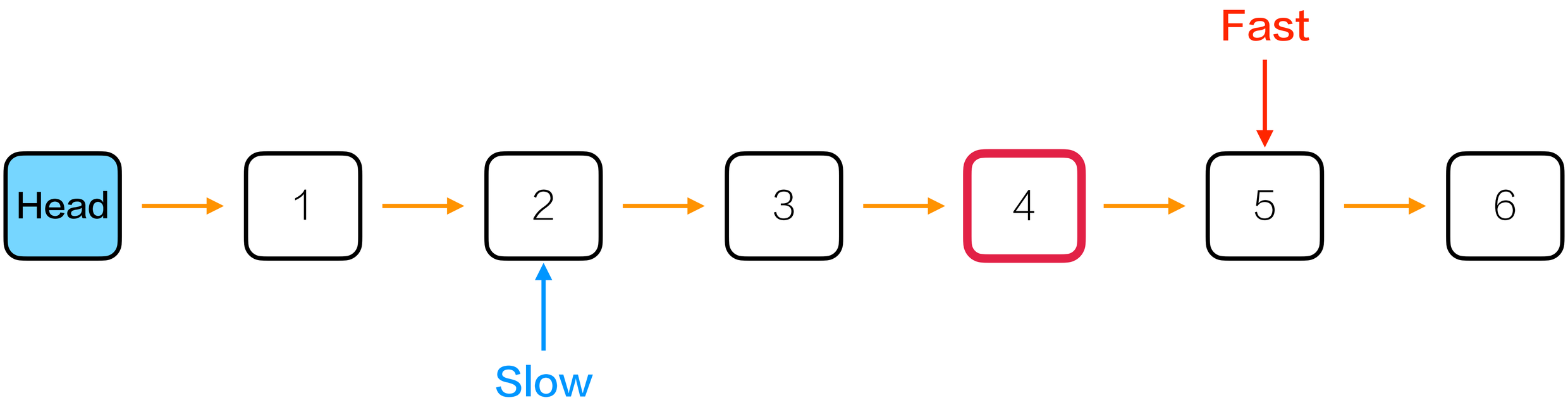
双指针（快慢指针）



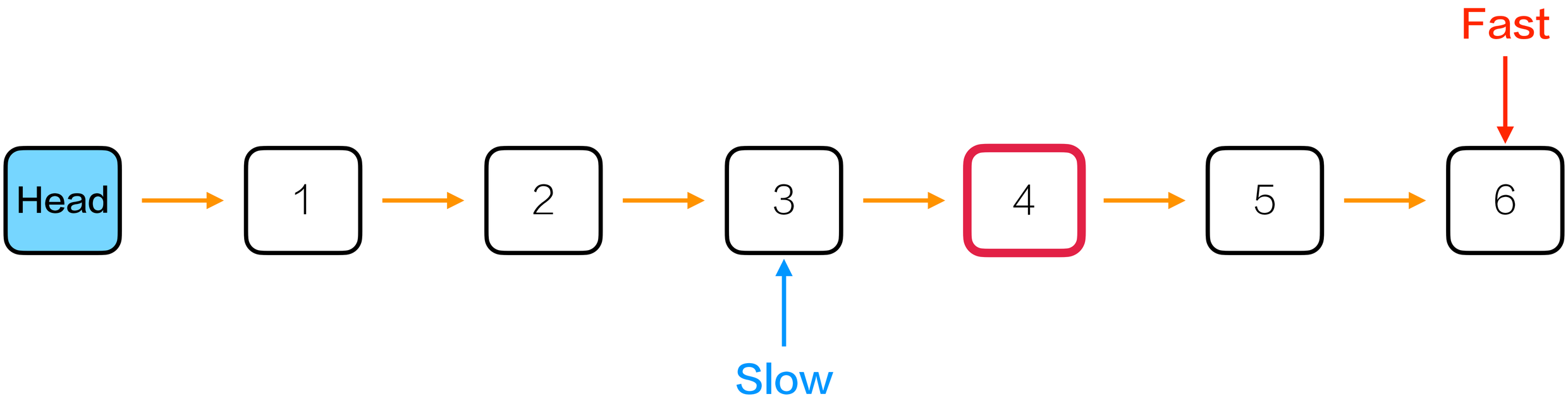
双指针（快慢指针）



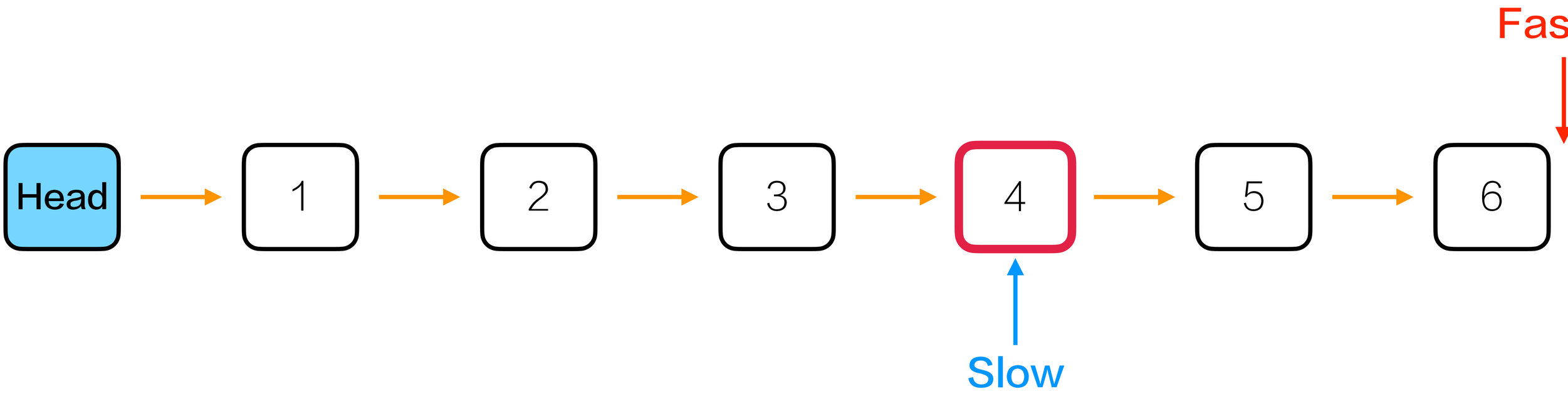
双指针（快慢指针）



双指针（快慢指针）



双指针（快慢指针）



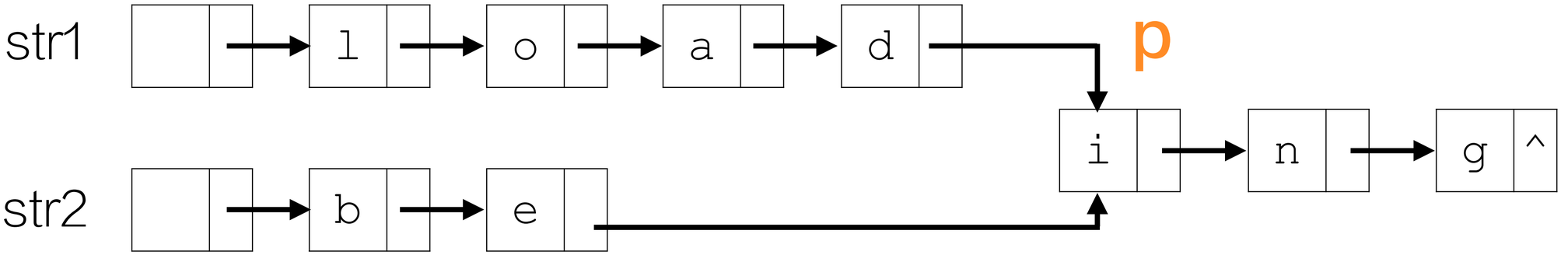
使用双指针找到倒数第 k 个节点

案例：线性表/链表/test9.c

```
int findNodeFS(Node *L, int k)
{
    Node *fast = L->next;
    Node *slow = L->next;
    for (int i = 0; i < k; i++)
    {
        fast = fast->next;
    }
    while(fast != NULL)
    {
        fast = fast->next;
        slow = slow->next;
    }
    printf("倒数第%d个节点值为：%d\n", k, slow->data);
}
```

单链表应用

[2012] 假定采用带头节点的单链表保存单词，当两个单词有相同的后缀时，则可共享相同的后缀存储空间，例如，“loading”和“being”的存储映像如下图所示。



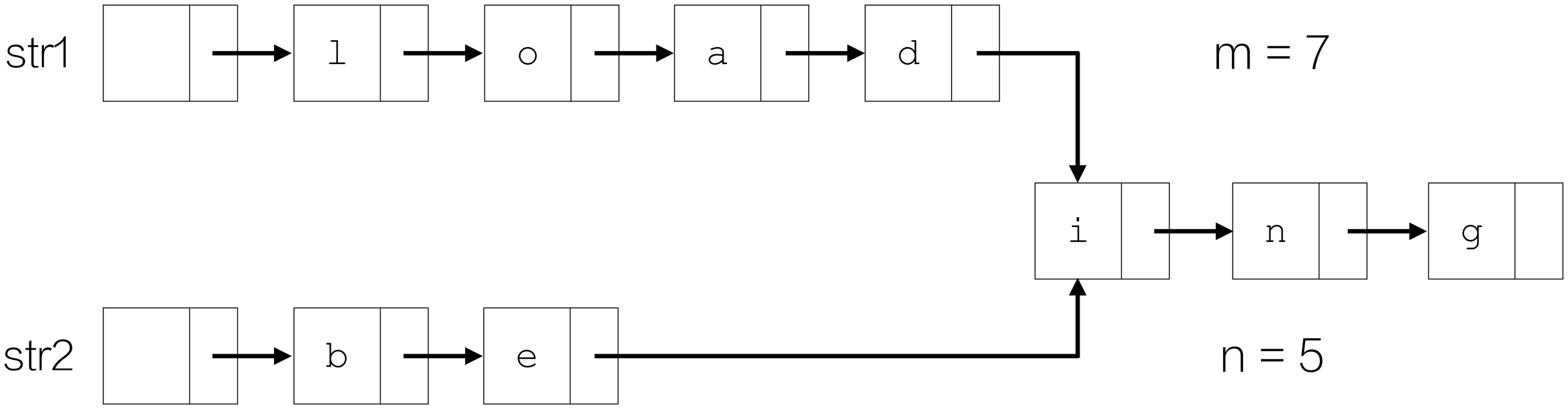
设 str1 和 str2 分别指向两个单词所在单链表的头节点，链表结点结构为 | data | next |

请设计一个时间上尽可能高效的算法，找出由 str1 和 str2 所指向两个链表共同后缀的起始位置（如图字符 i 所在结点的位置 p）。要求：

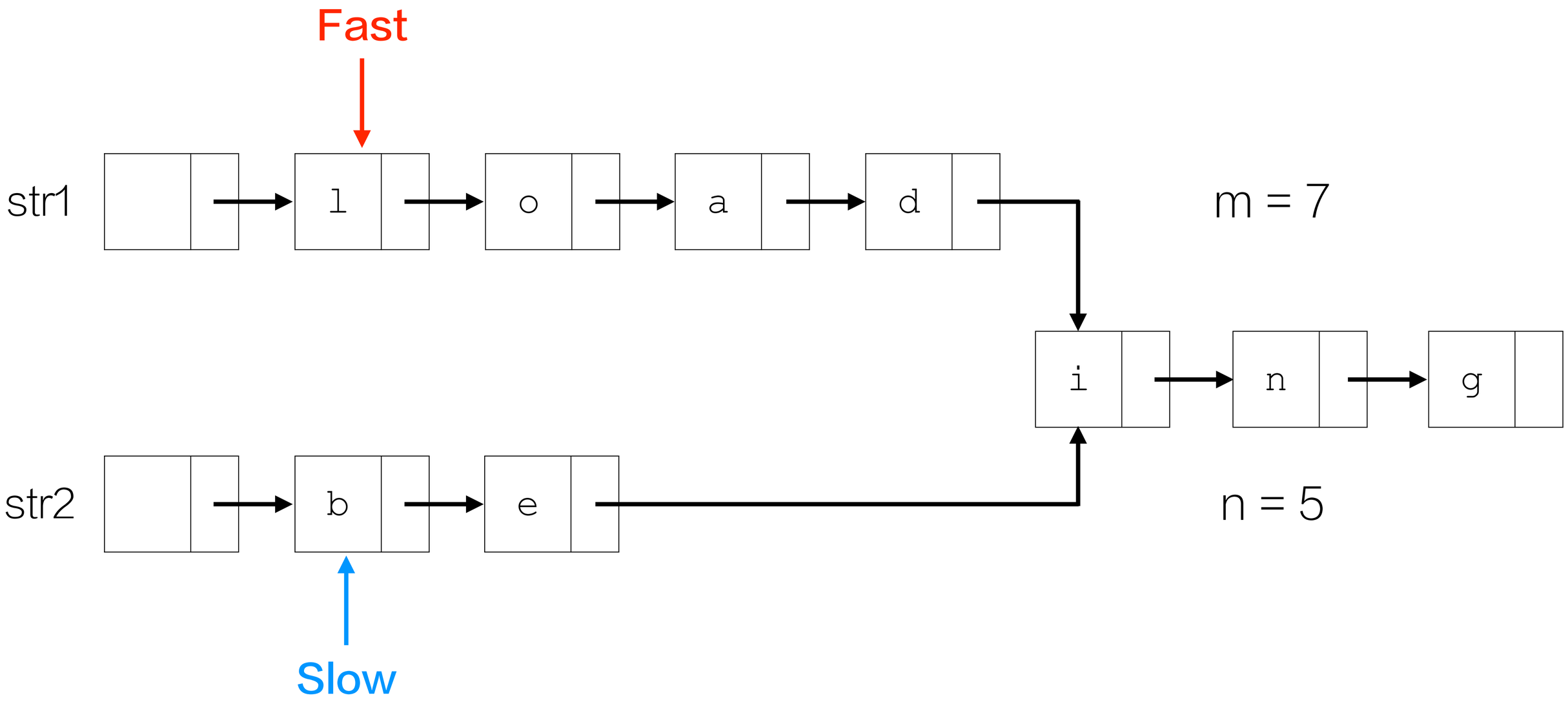
1）描述算法的基本设计思想；

2）根据设计思想，采用 C 或 C++ 或 Java 语言描述，关键之处给出注释

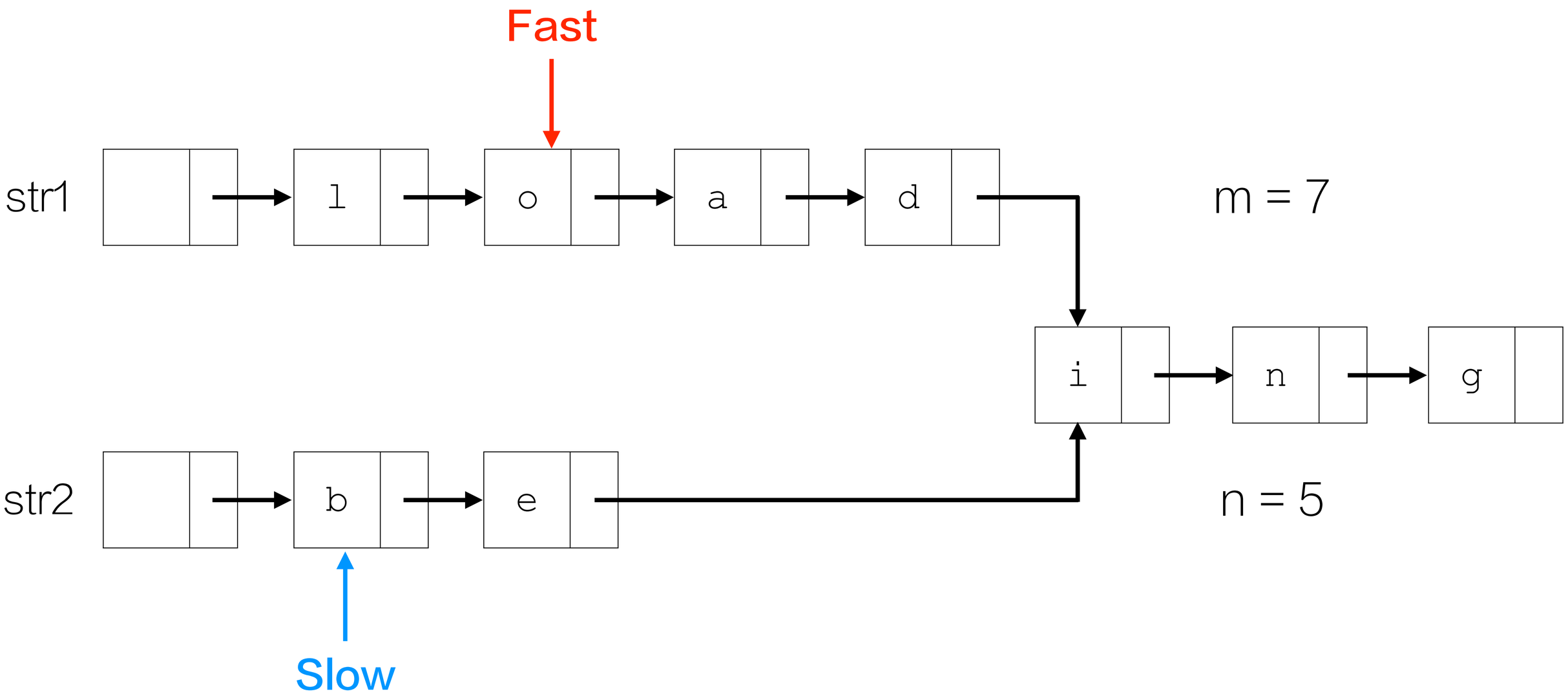
3）说明你所设计算法的时间复杂度



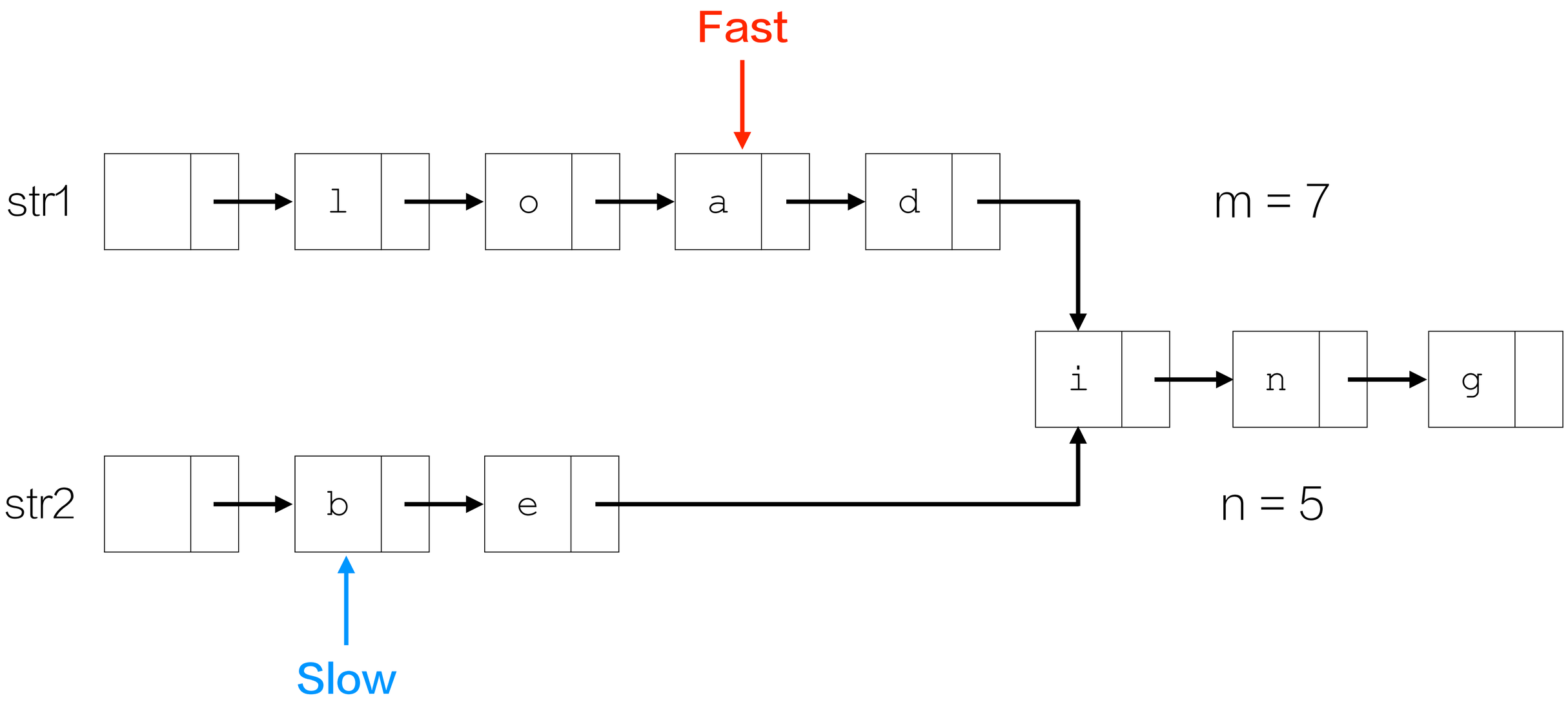
第一步：分别求出两个链表的长度 m 和 n



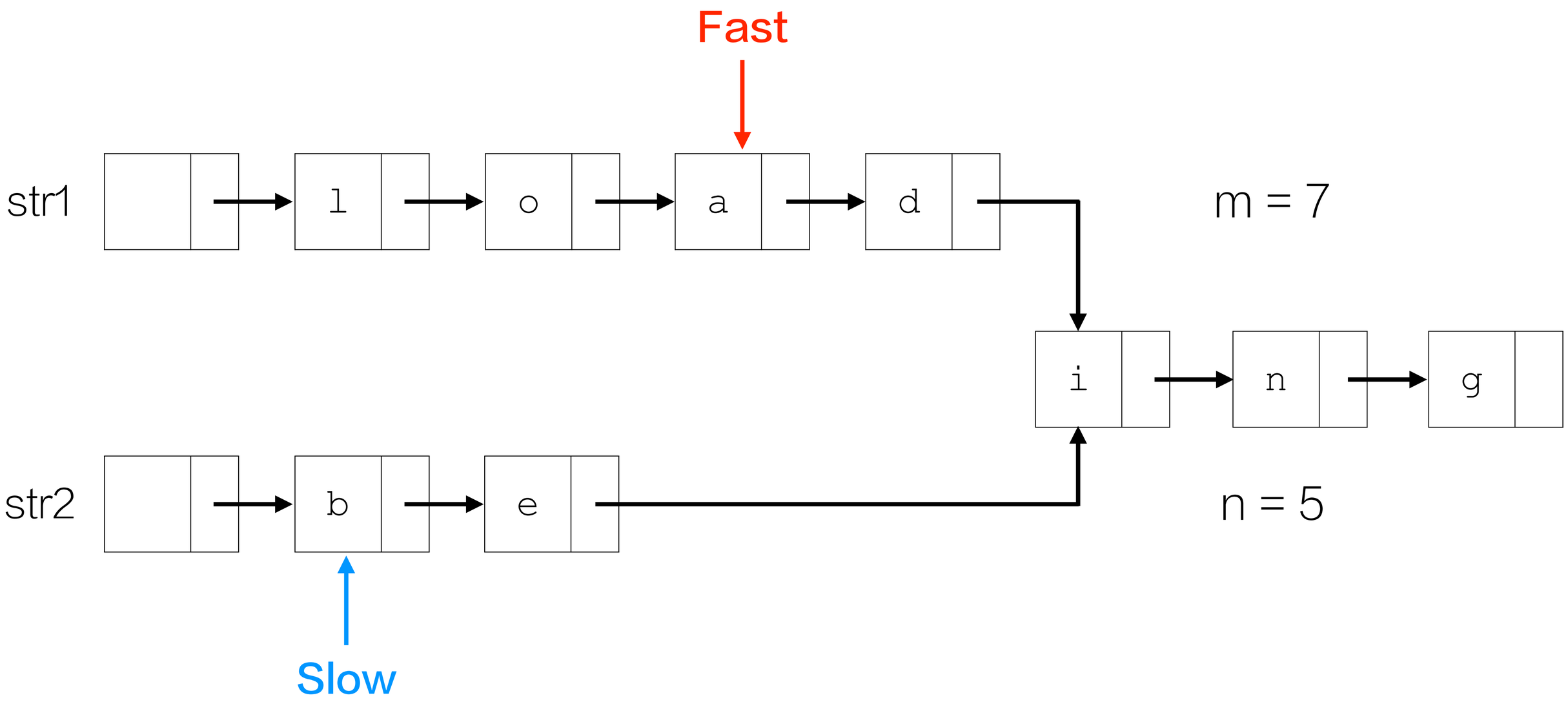
第二步：Fast 指针指向较长的链表，先走 m-n 或 n-m 步。



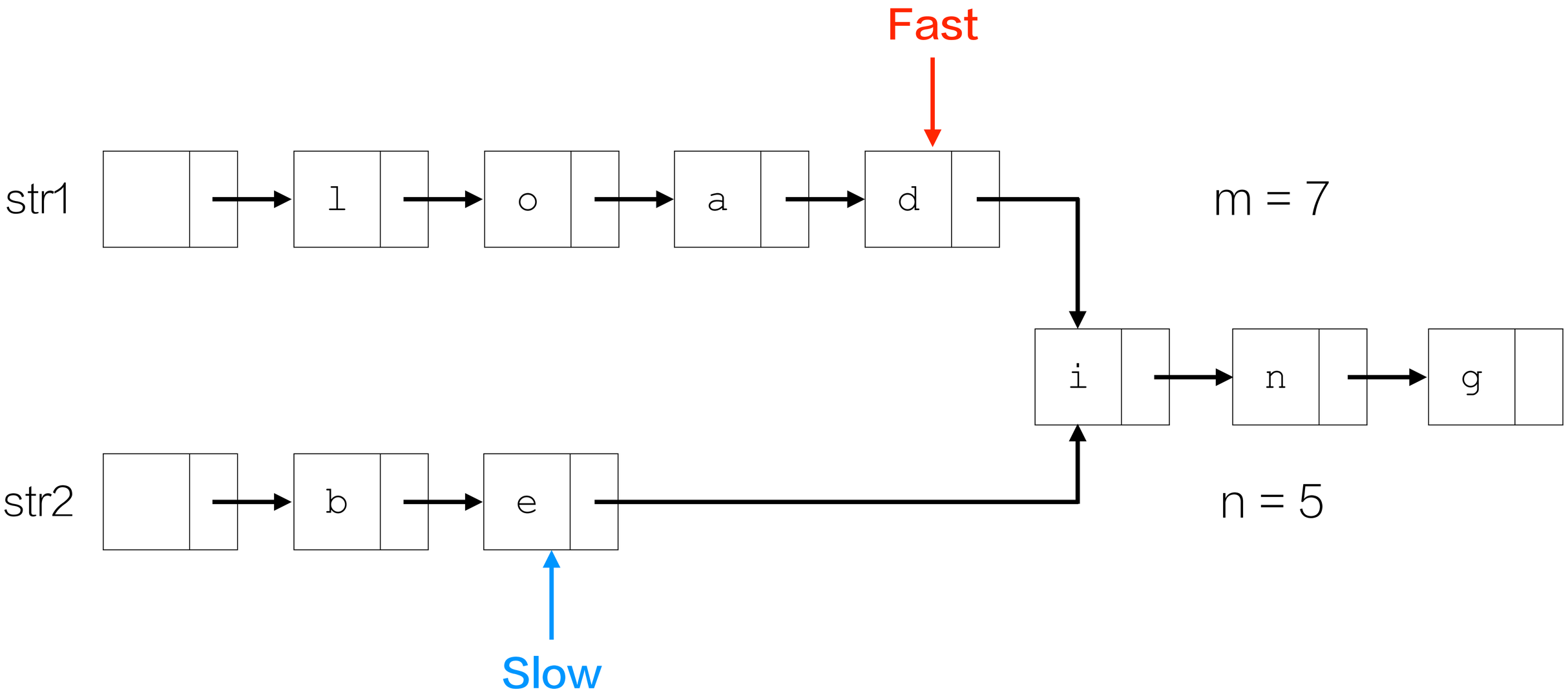
第二步：Fast 指针指向较长的链表，先走 m-n 或 n-m 步。



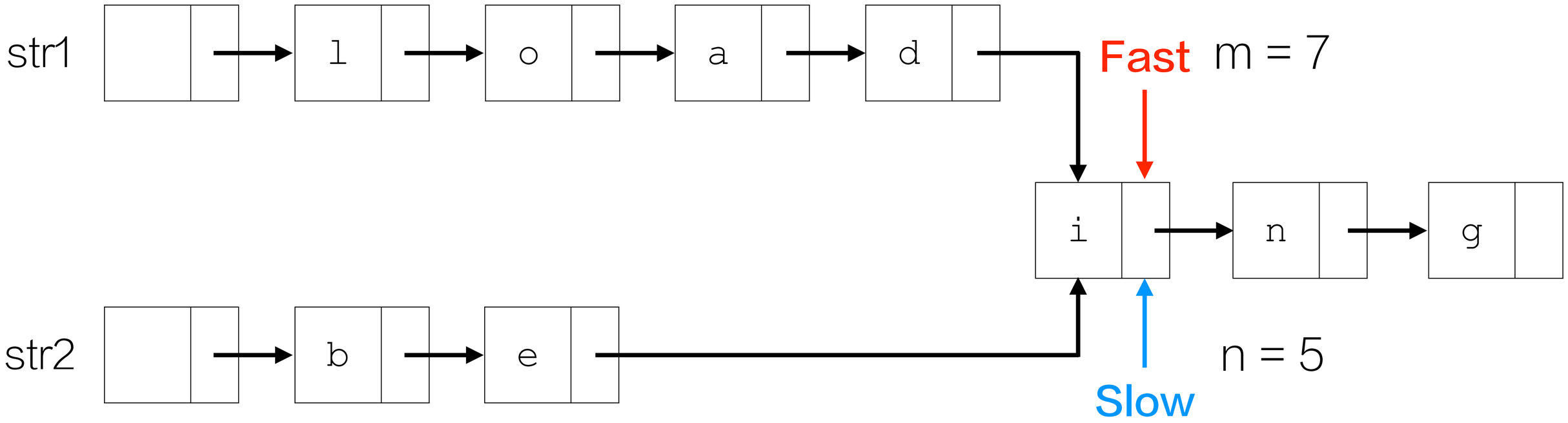
第二步：Fast 指针指向较长的链表，先走 m-n 或 n-m 步。



第三步：同步移动指针，判断它们是否指向同一个节点



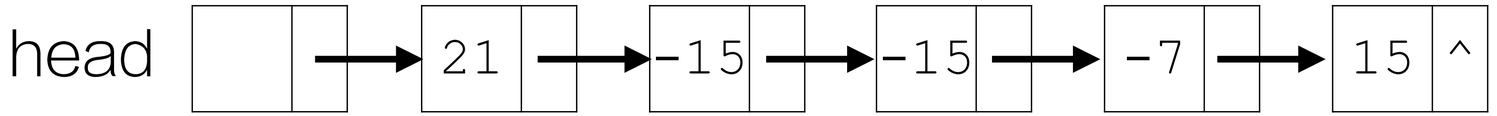
第三步：同步移动指针，判断它们是否指向同一个节点



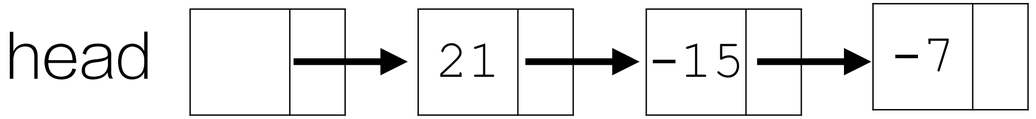
第三步：同步移动指针，判断它们是否指向同一个节点

单链表应用

[2015]（15分）用单链表保存 n 个整数，结点的结构为 [data][link]，且|data|≤n(n 为正整数)。现要求设计一个时间复杂度尽可能高效的算法，对于链表中 data 的绝对值相等的节点，仅保留第一次出现的节点而删除其余绝对值相等的结点。例如，若给定单链表 head 如下：



则删除节点后的 head 为



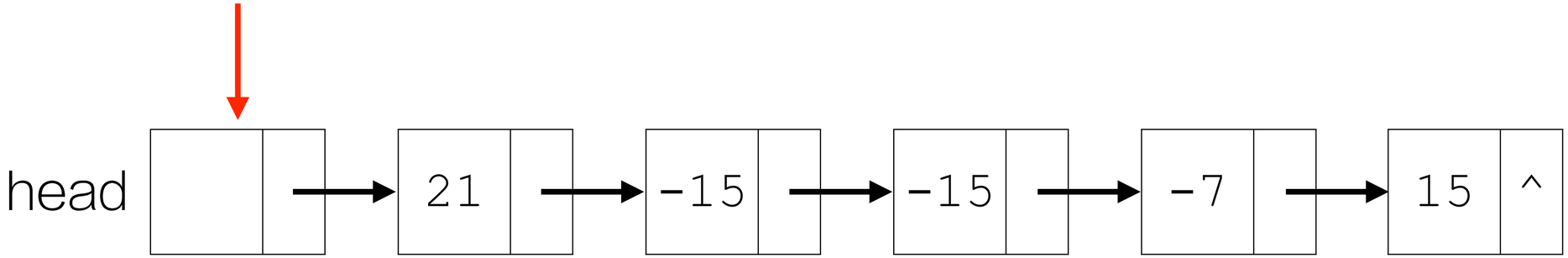
要求：

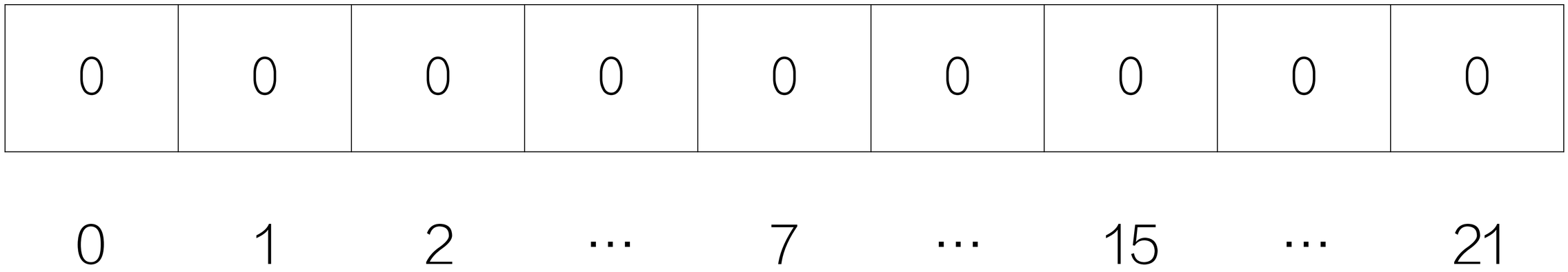
1）给出算法的基本设计思想。

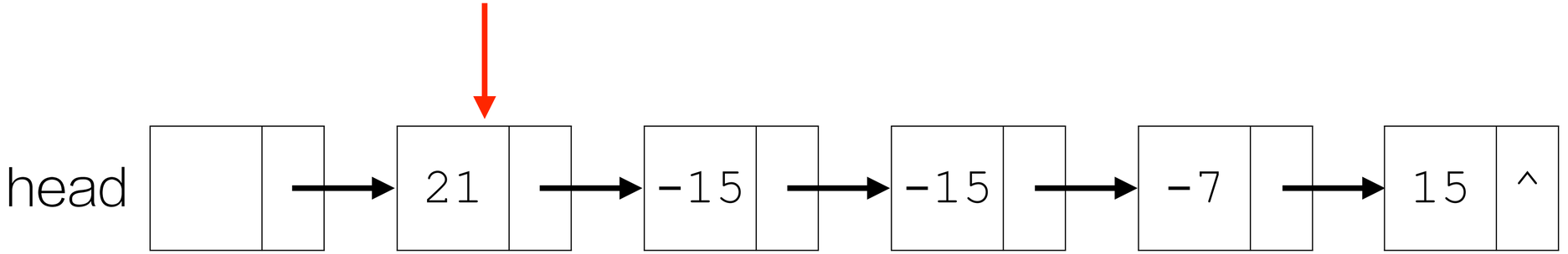
2）使用 C 或 C++ 语言，给出单链表节点的数据类型定义。

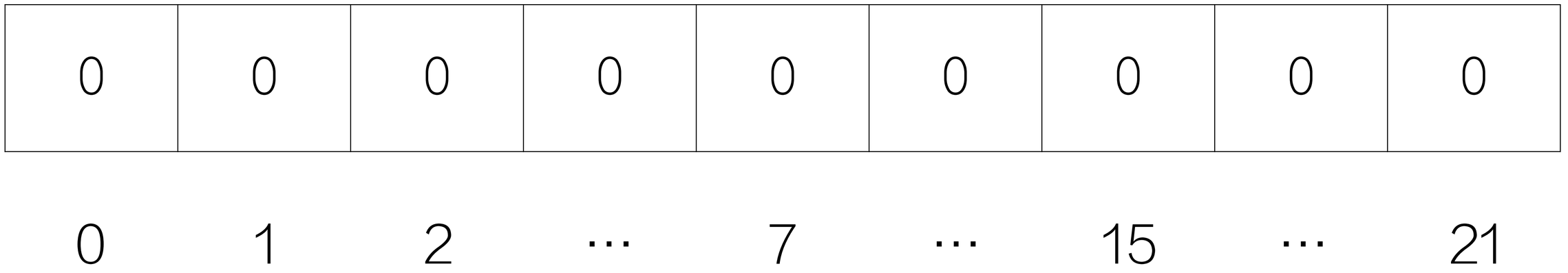
3）根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

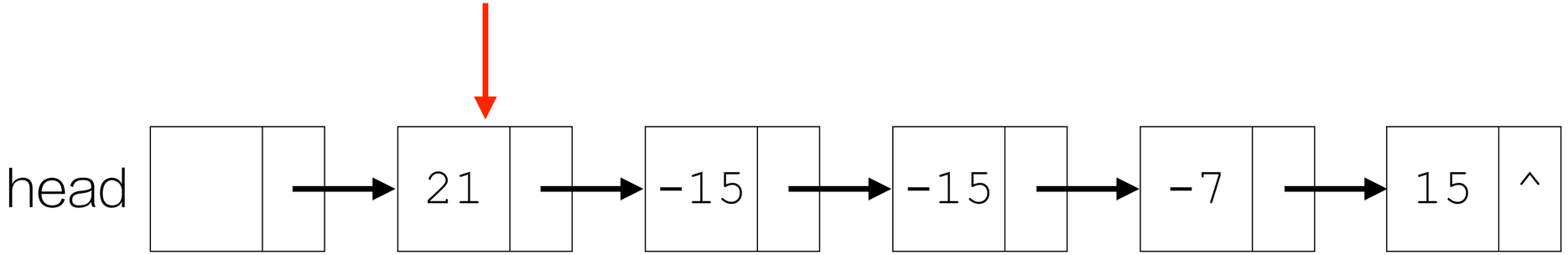
4）说明你所设计算法的时间复杂度和空间复杂度。

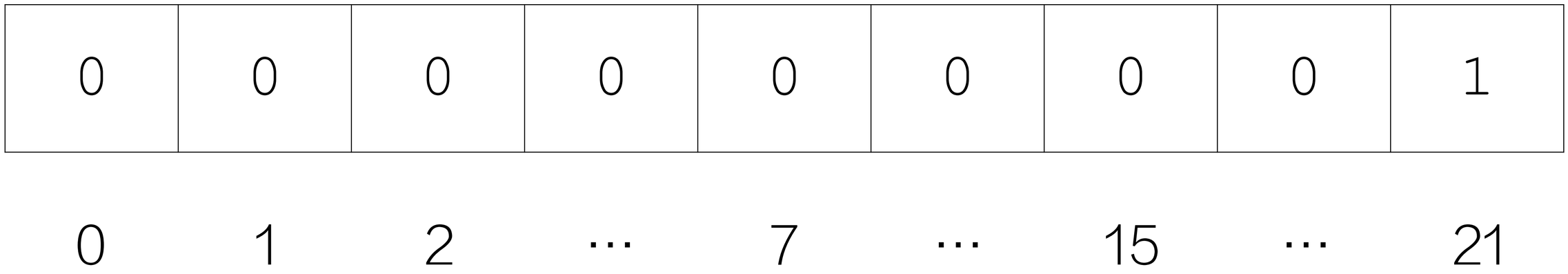

head
21
-15
-15
-7
15
^

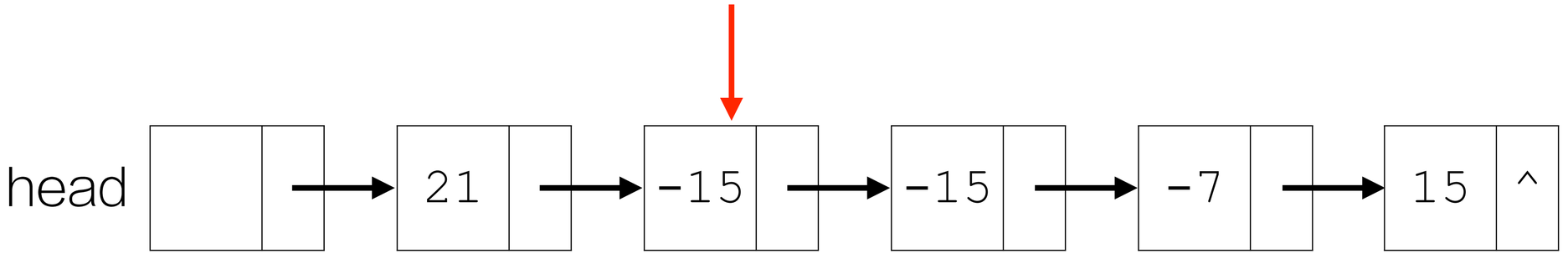

0
0
0
0
0
0
0
0
0
0
1
2
...
7
...
15
...
21

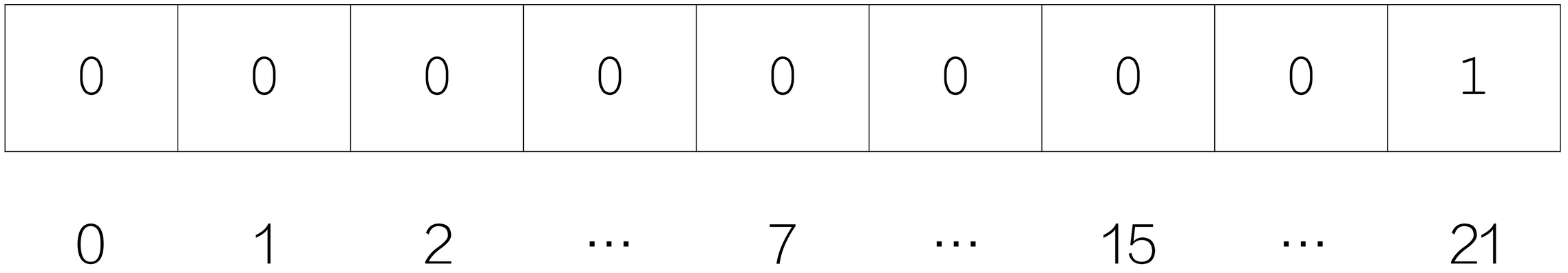

head
21
-15
-15
-7
15
^

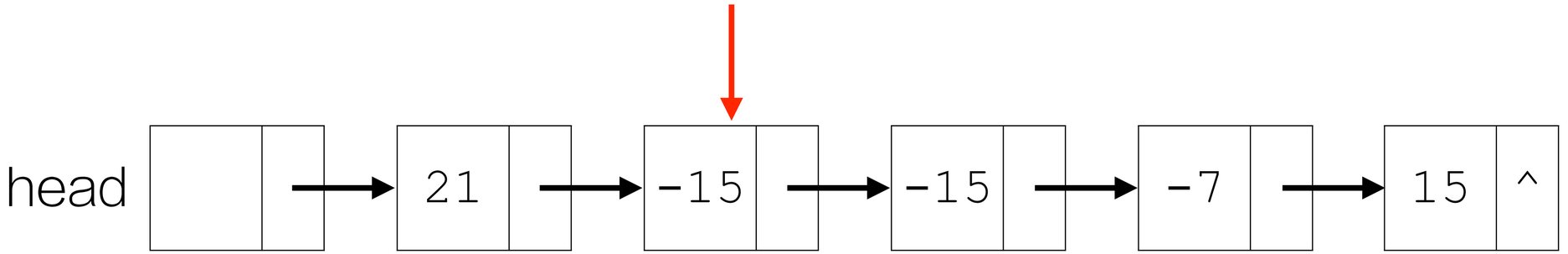

0
0
0
0
0
0
0
0
0
0
1
2
...
7
...
15
...
21

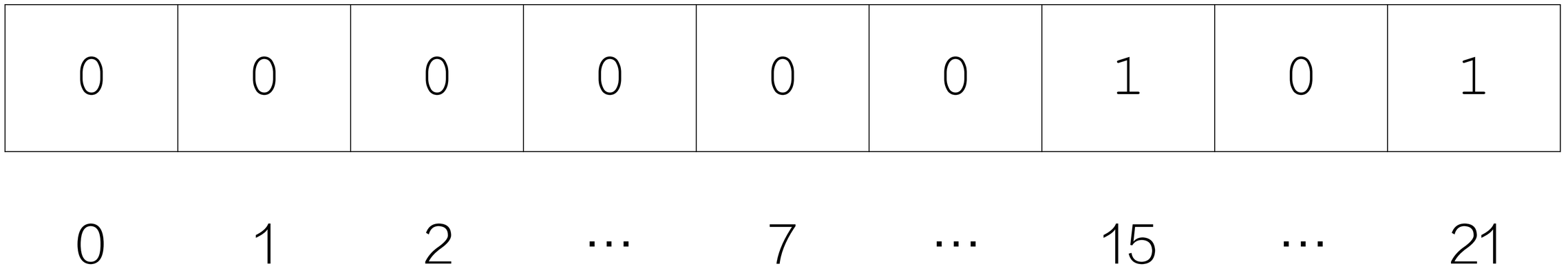

head
21
-15
-15
-7
15
^

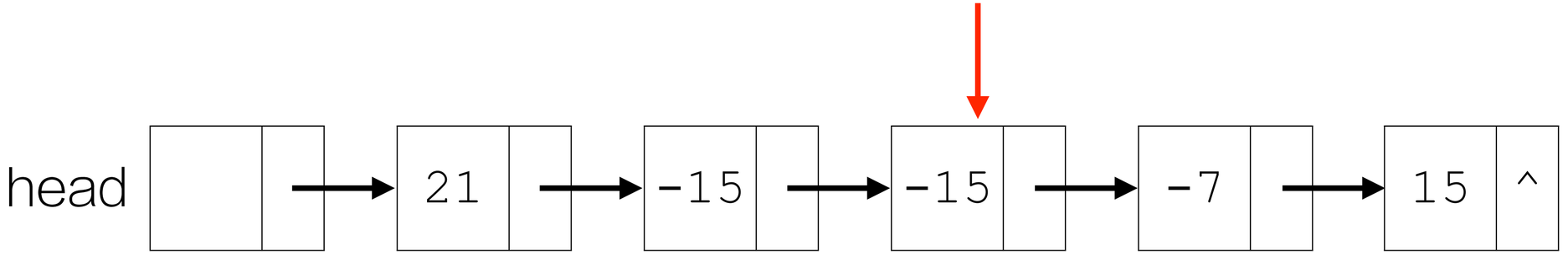

0
0
0
0
0
0
0
0
1
0
1
2
...
7
...
15
...
21

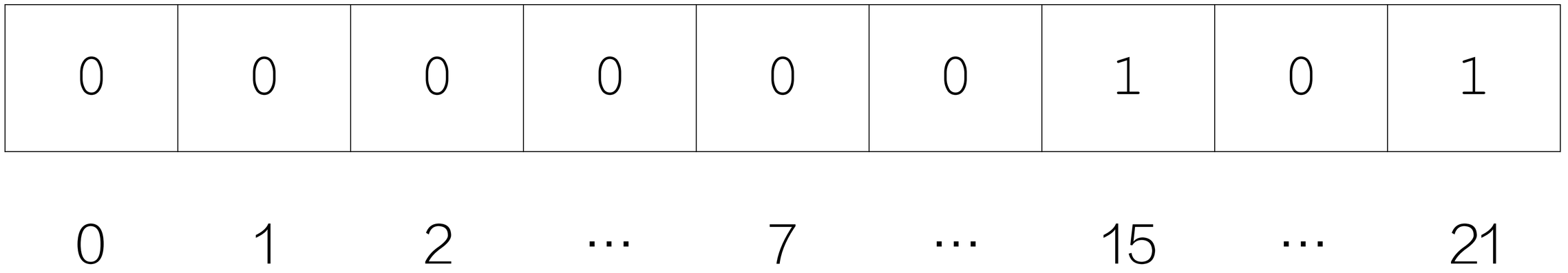

head
21
-15
-15
-7
15
^

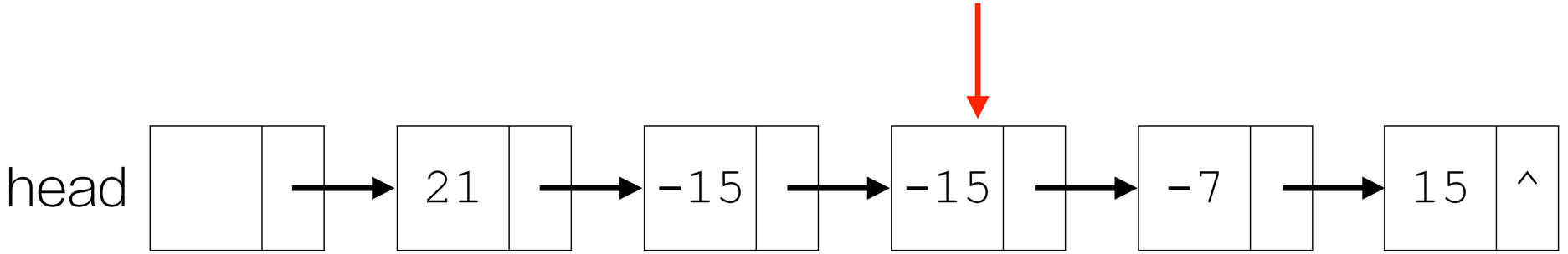

0
0
0
0
0
0
0
0
1
0
1
2
...
7
...
15
...
21

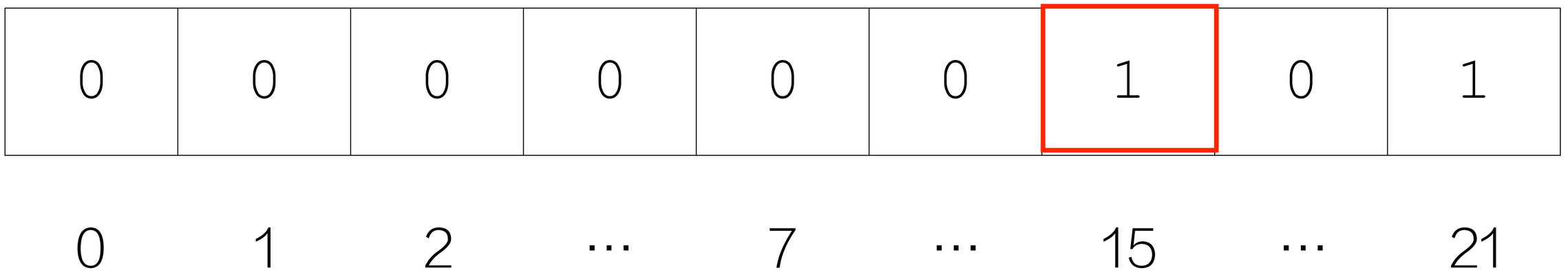

head
21
-15
-15
-7
15
^

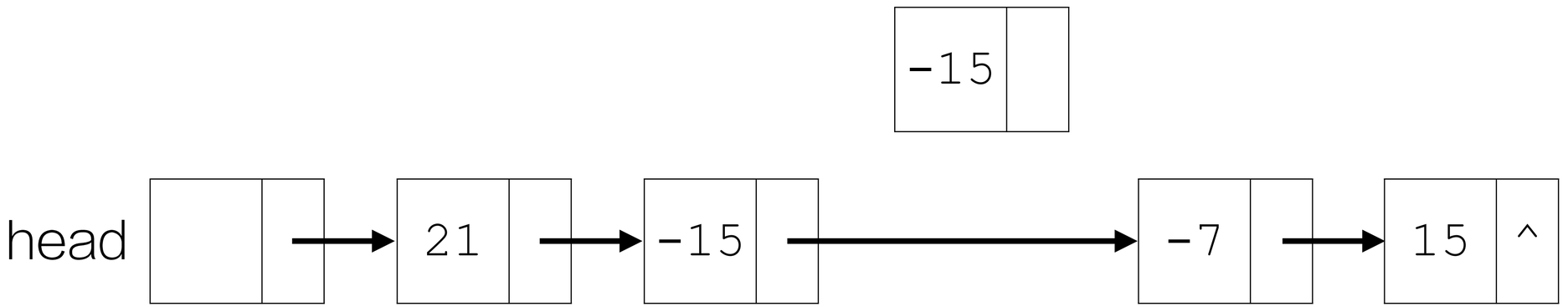

0
0
0
0
0
0
1
0
1
0
1
2
...
7
...
15
...
21

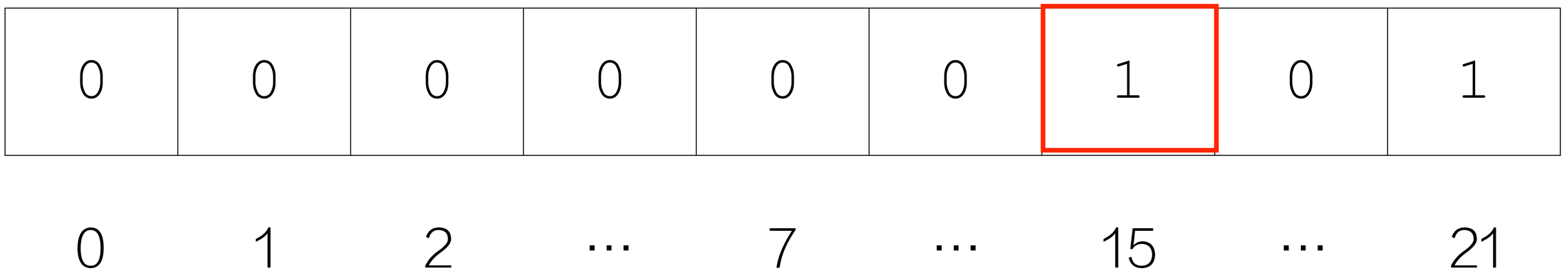

head
21
-15
-15
-7
15
^

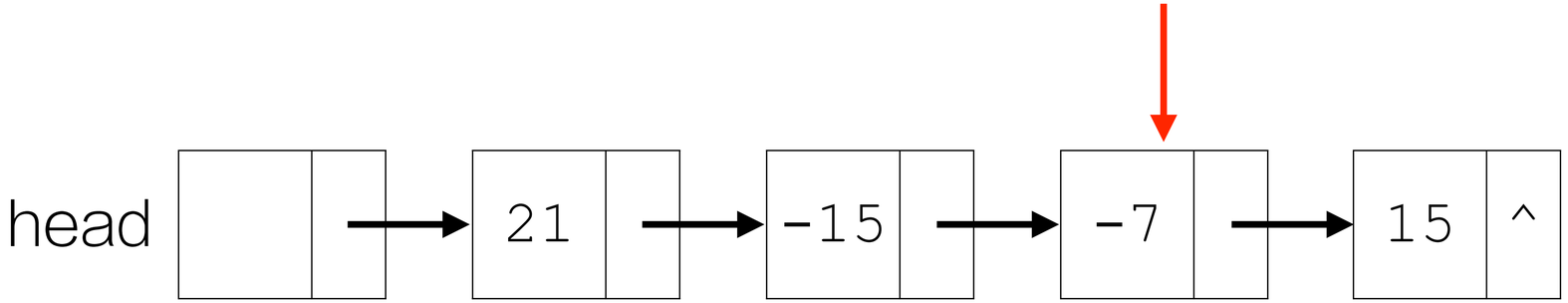

0
0
0
0
0
0
1
0
1
0
1
2
...
7
...
15
...
21

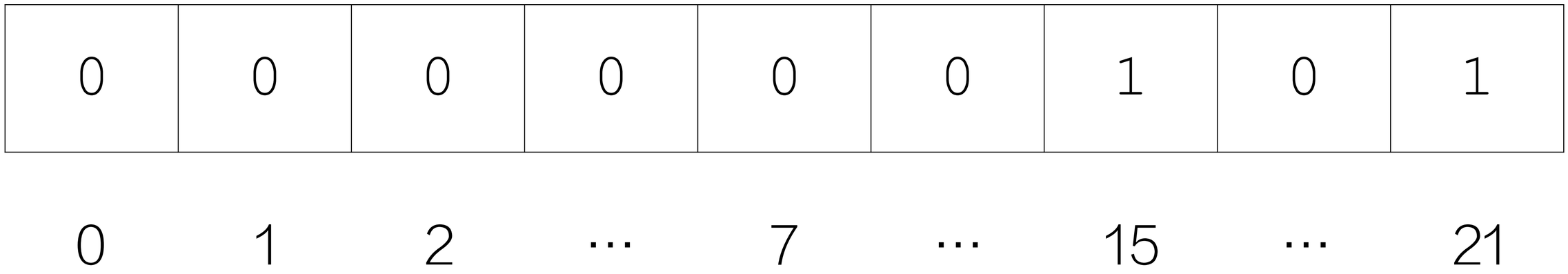

head
21
-15
-15
-7
15
^

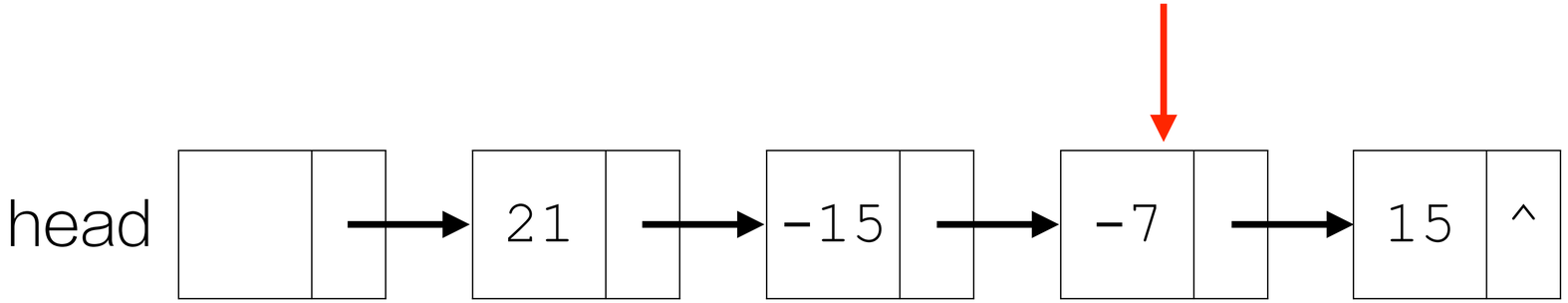

0
0
0
0
0
0
1
0
1
0
1
2
...
7
...
15
...
21

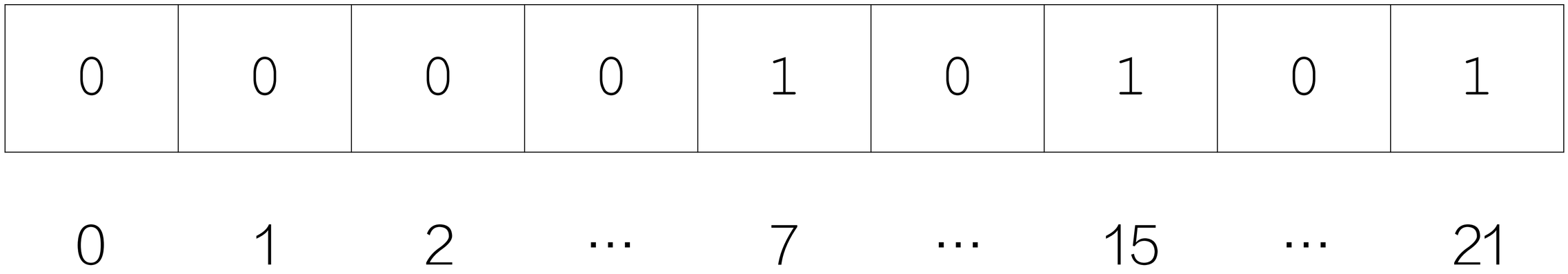

-15
head
21
-15
-7
15
^

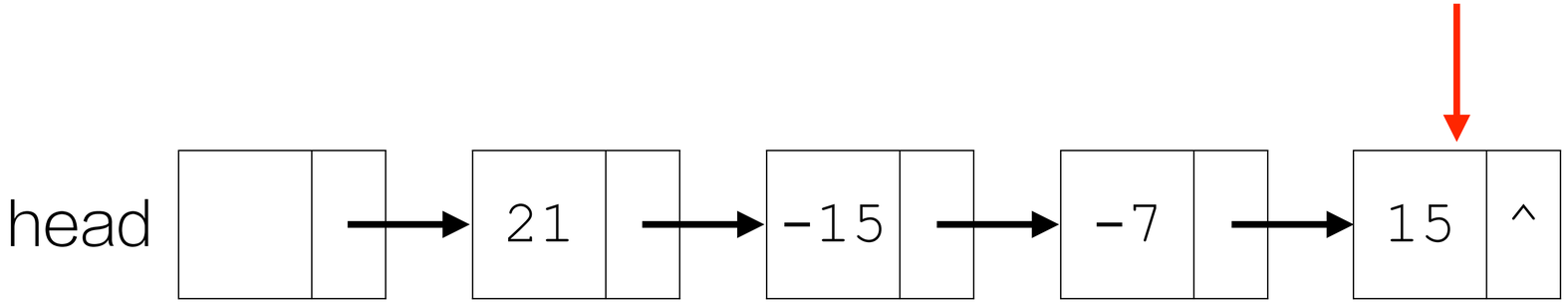

0
0
0
0
0
0
1
0
1
0
1
2
...
7
...
15
...
21

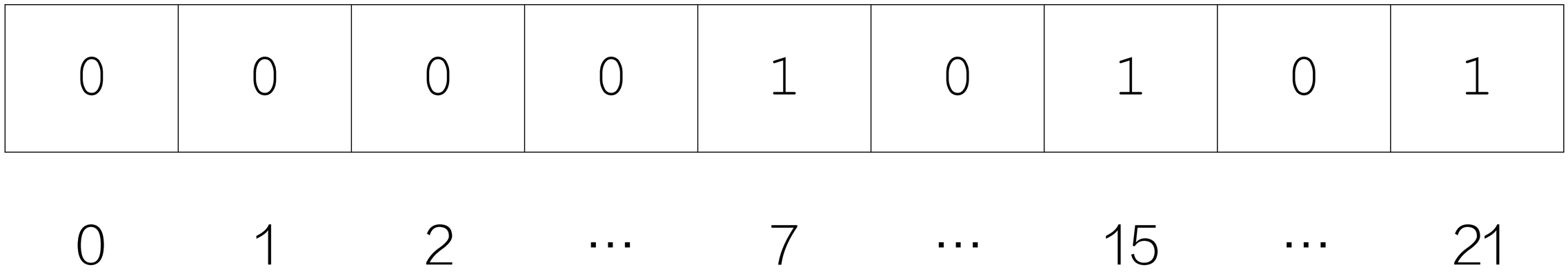

head
21
-15
-7
15
^

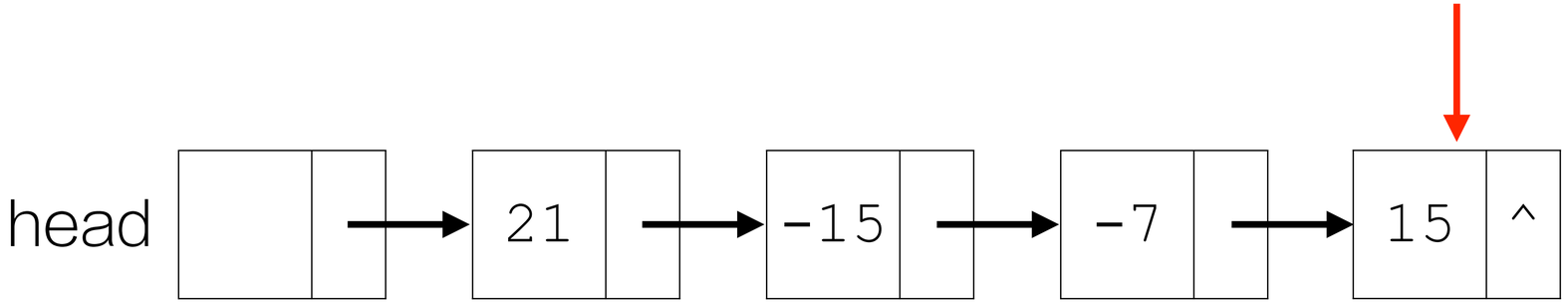

0
0
0
0
0
0
1
0
1
0
1
2
...
7
...
15
...
21

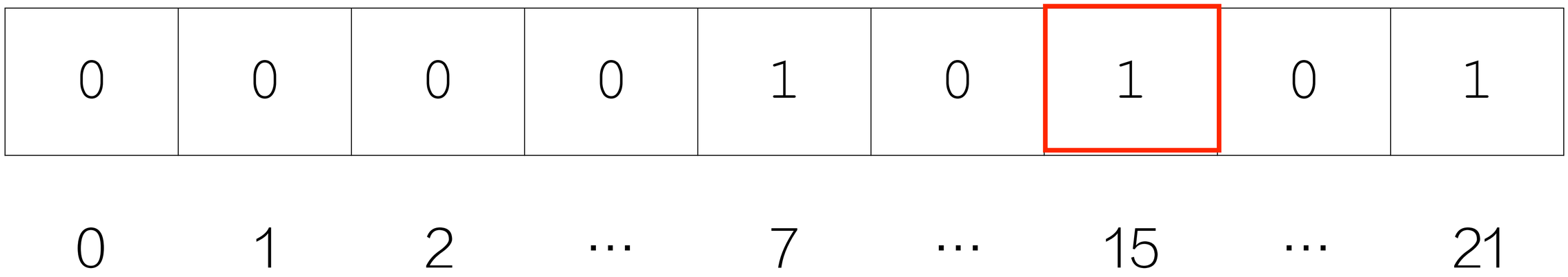

head
21
-15
-7
15
^


0
0
0
0
1
0
1
0
1
0
1
2
...
7
...
15
...
21

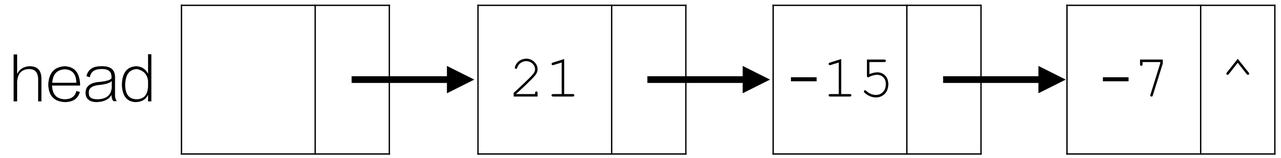

head
21
-15
-7
15
^

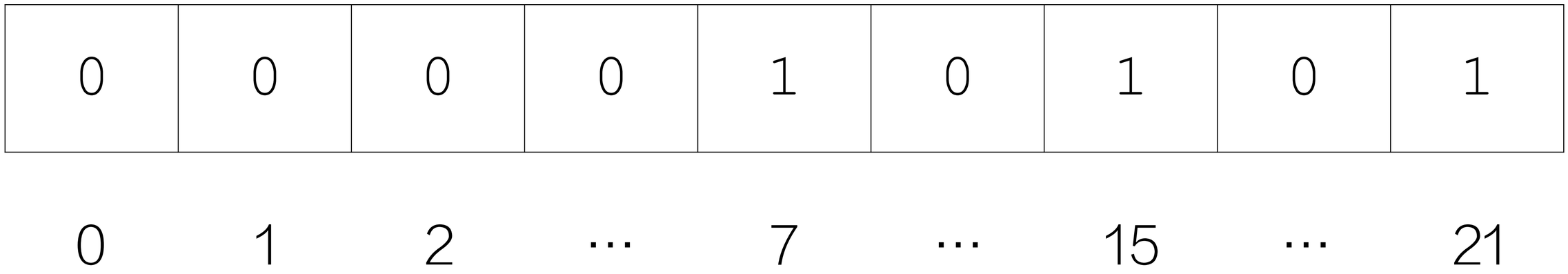

0
0
0
0
1
0
1
0
1
0
1
2
...
7
...
15
...
21


head
21
-15
-7
15
^

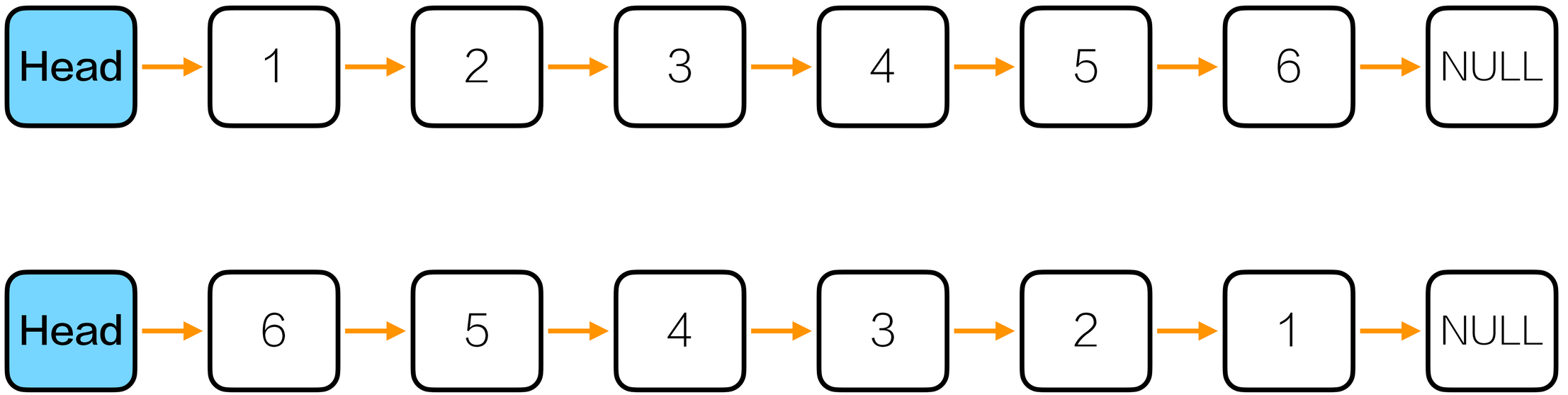

0
0
0
0
1
0
1
0
1
0
1
2
...
7
...
15
...
21

案例：线性表/链表/test11.c



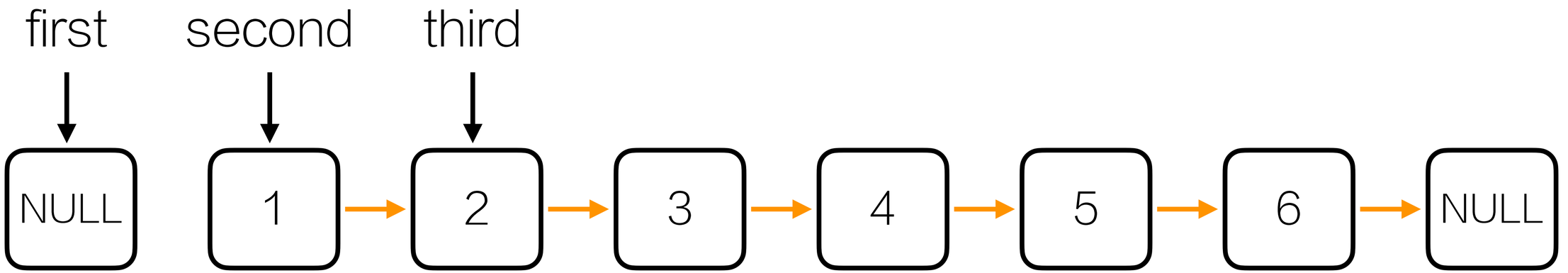


单链表应用 – 反转链表



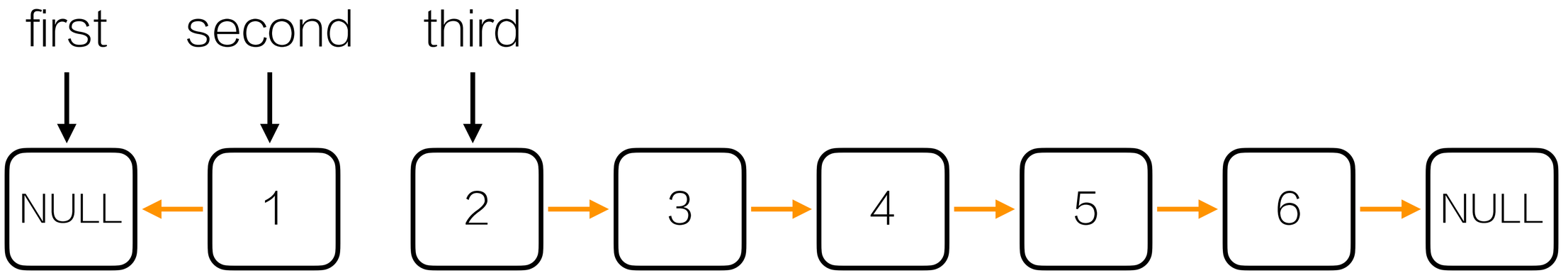
单链表应用 – 反转链表

案例：线性表/链表/test12.c



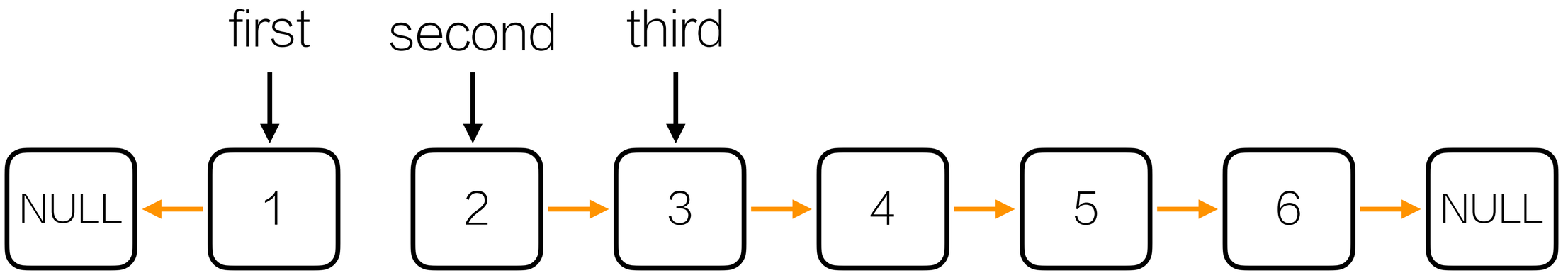
单链表应用 – 反转链表

案例：线性表/链表/test12.c



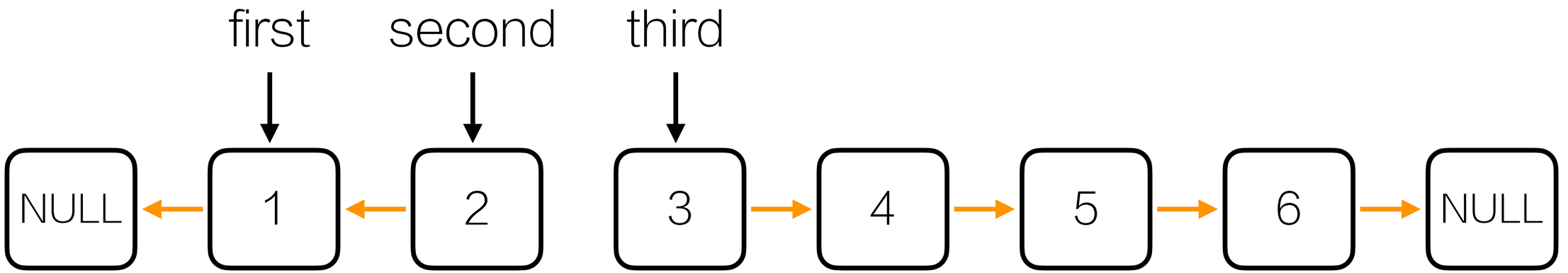
单链表应用 – 反转链表

案例：线性表/链表/test12.c



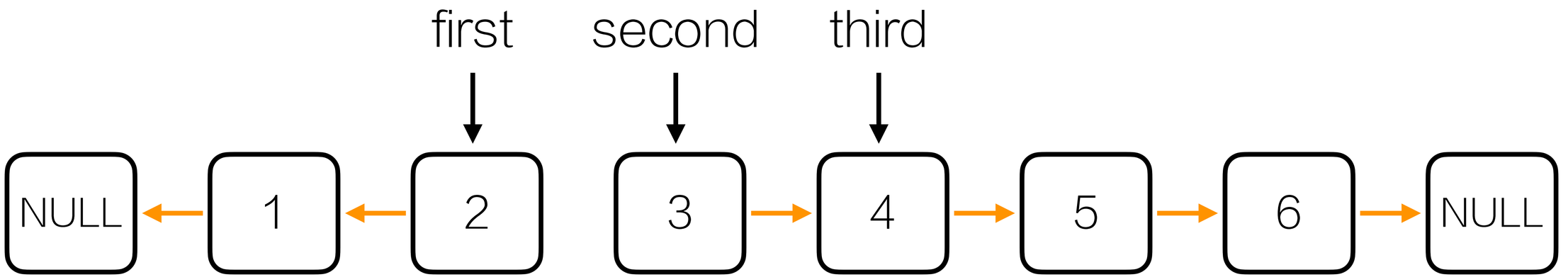
单链表应用 – 反转链表

案例：线性表/链表/test12.c



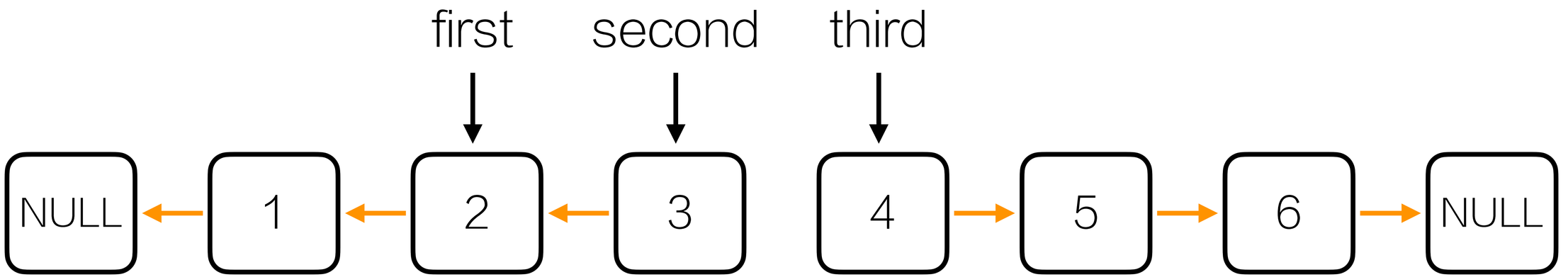
单链表应用 – 反转链表

案例：线性表/链表/test12.c



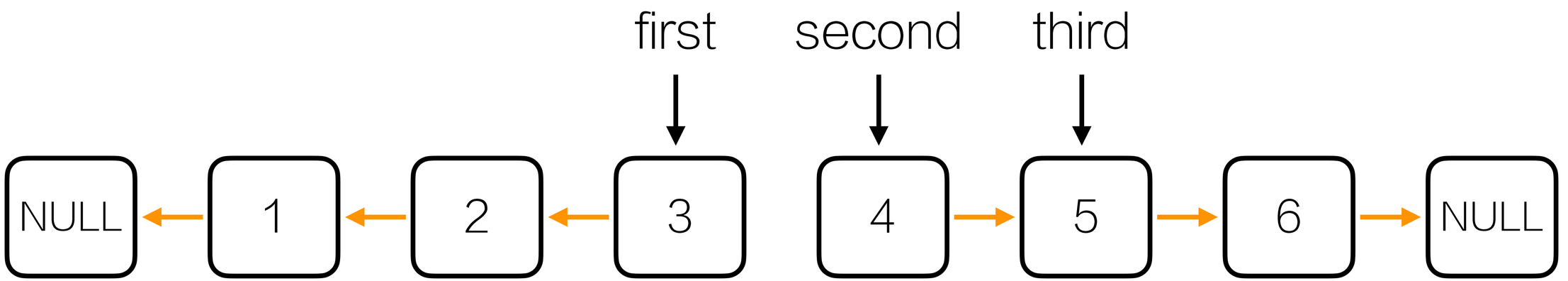
单链表应用 – 反转链表

案例：线性表/链表/test12.c



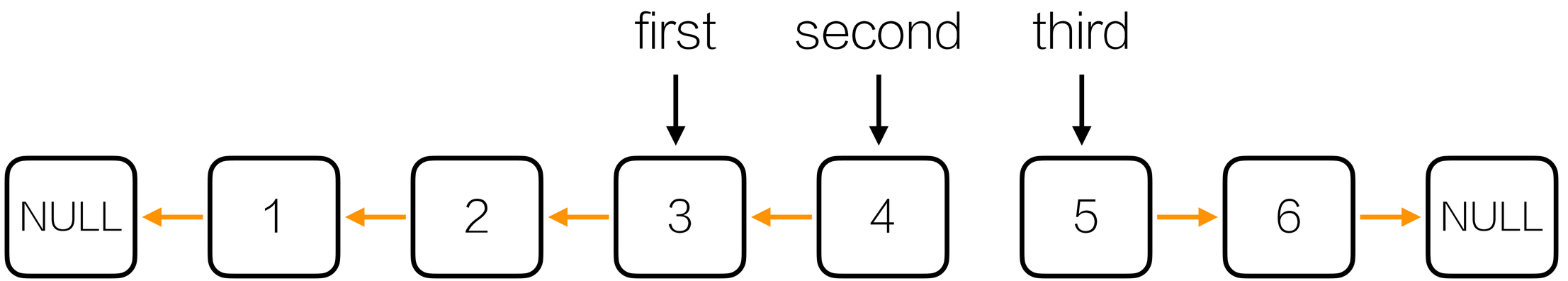
单链表应用 – 反转链表

案例：线性表/链表/test12.c



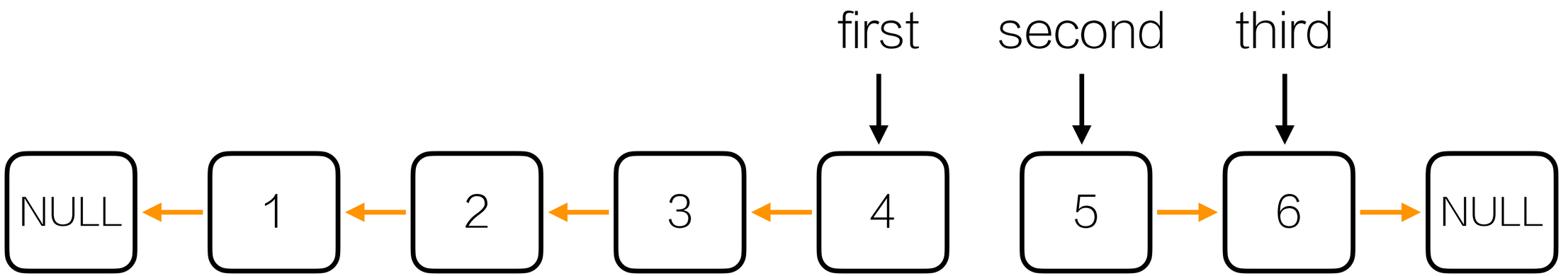
单链表应用 – 反转链表

案例：线性表/链表/test12.c



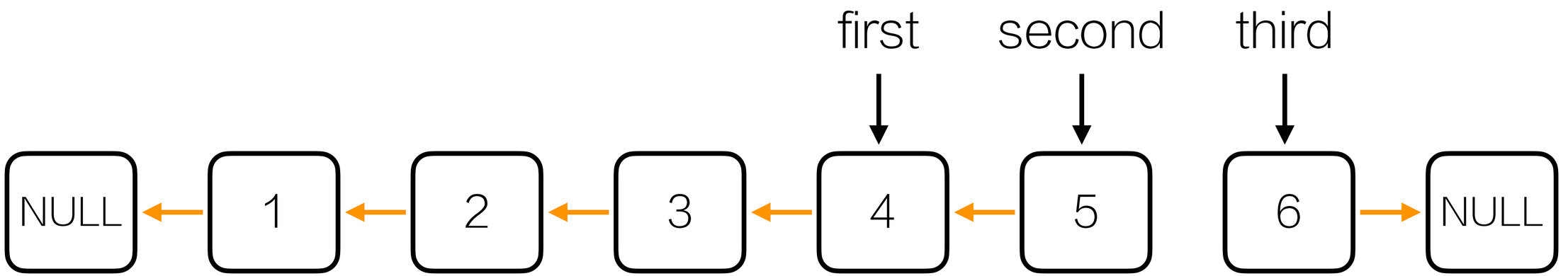
单链表应用 – 反转链表

案例：线性表/链表/test12.c



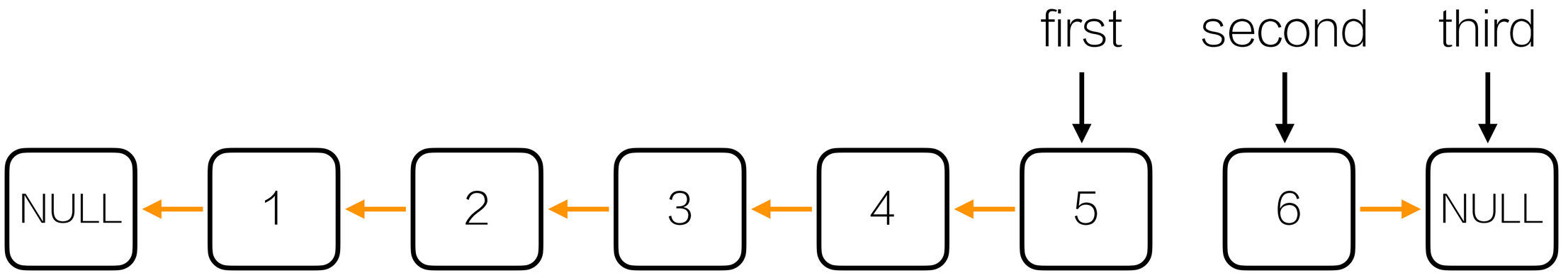
单链表应用 – 反转链表

案例：线性表/链表/test12.c



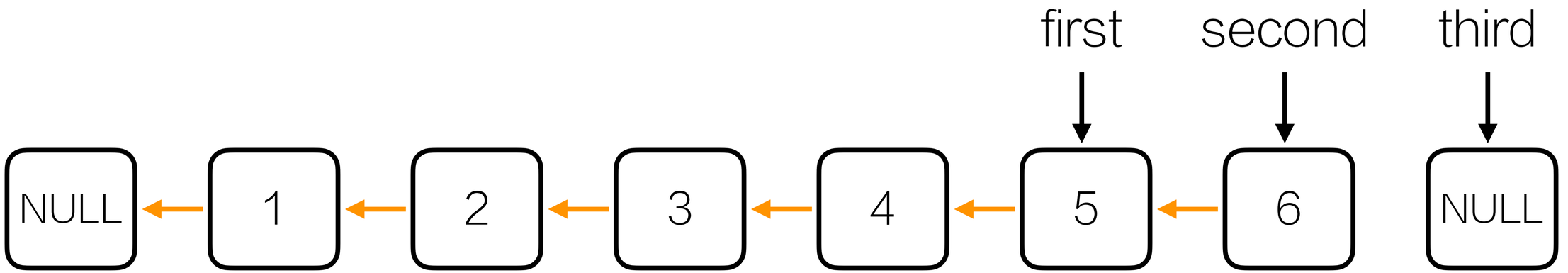
单链表应用 – 反转链表

案例：线性表/链表/test12.c



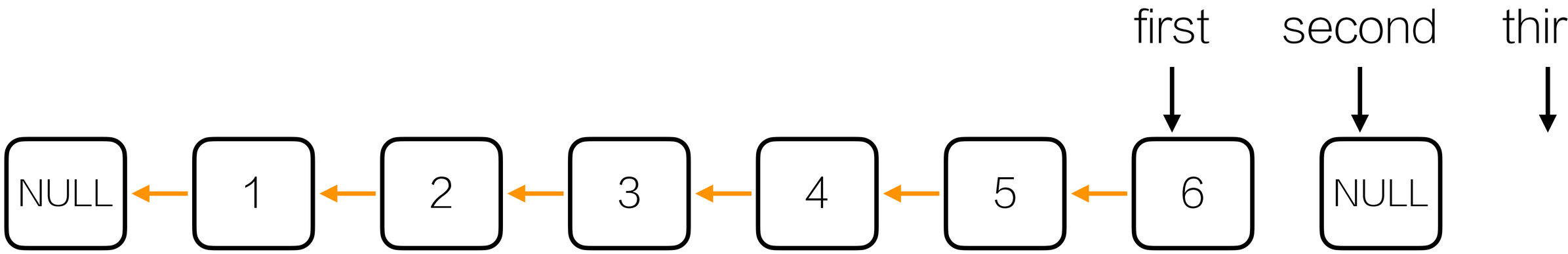
单链表应用 – 反转链表

案例：线性表/链表/test12.c



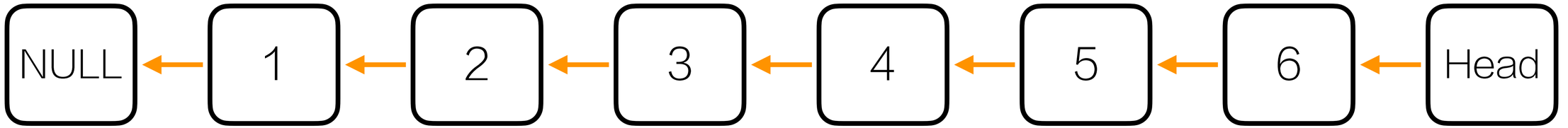
单链表应用 – 反转链表

案例：线性表/链表/test12.c



单链表应用 - 反转链表

案例：线性表/链表/test12.c



单链表应用 – 删除链表中间节点

案例：线性表/链表/test13.c



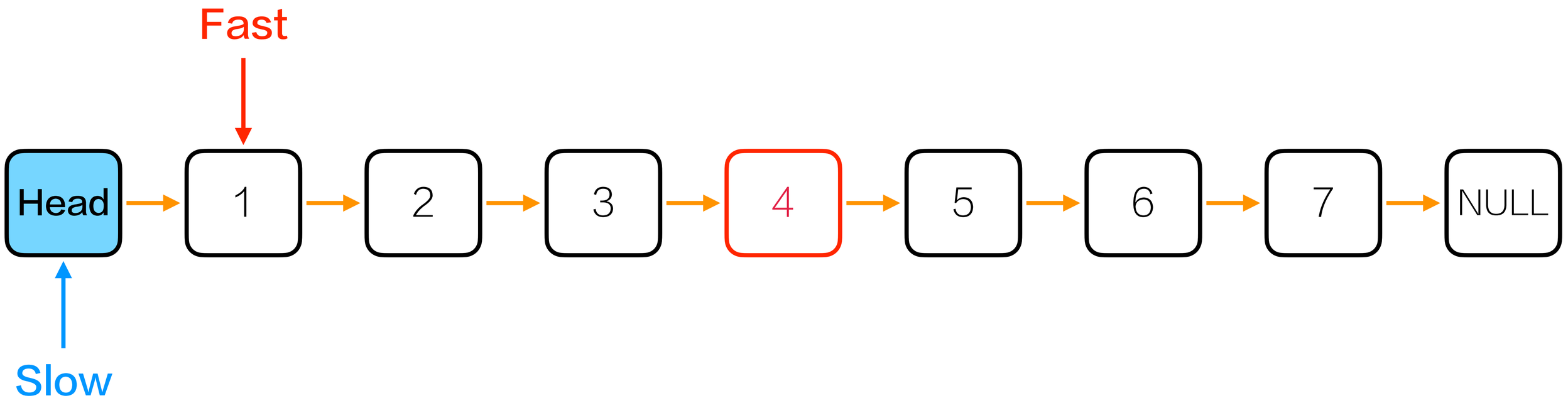
单链表应用 – 删除链表中间节点

案例：线性表/链表/test13.c



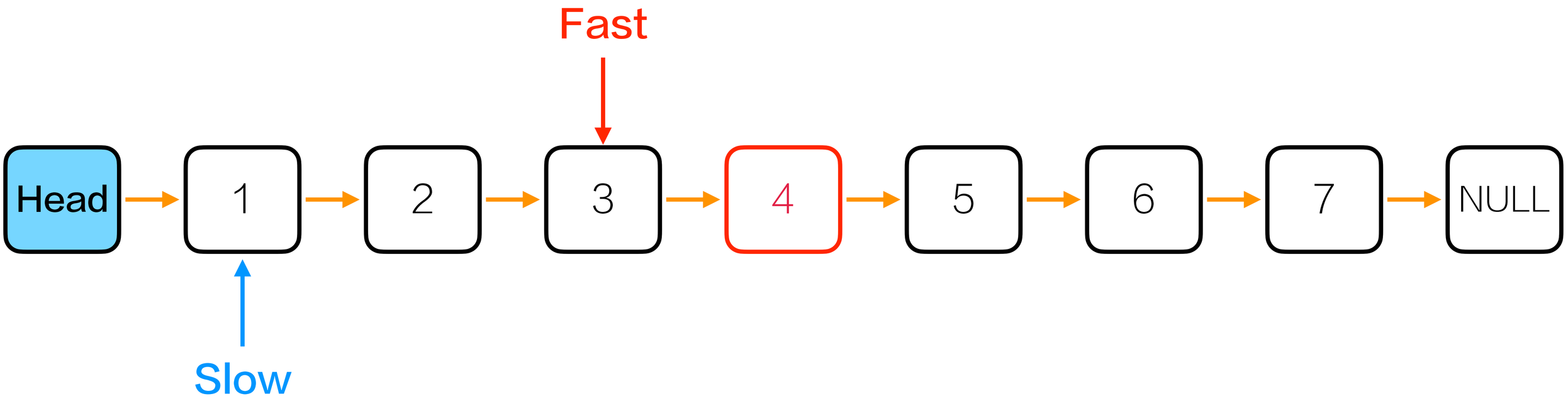
单链表应用 - 删除链表中间节点

案例：线性表/链表/test13.c



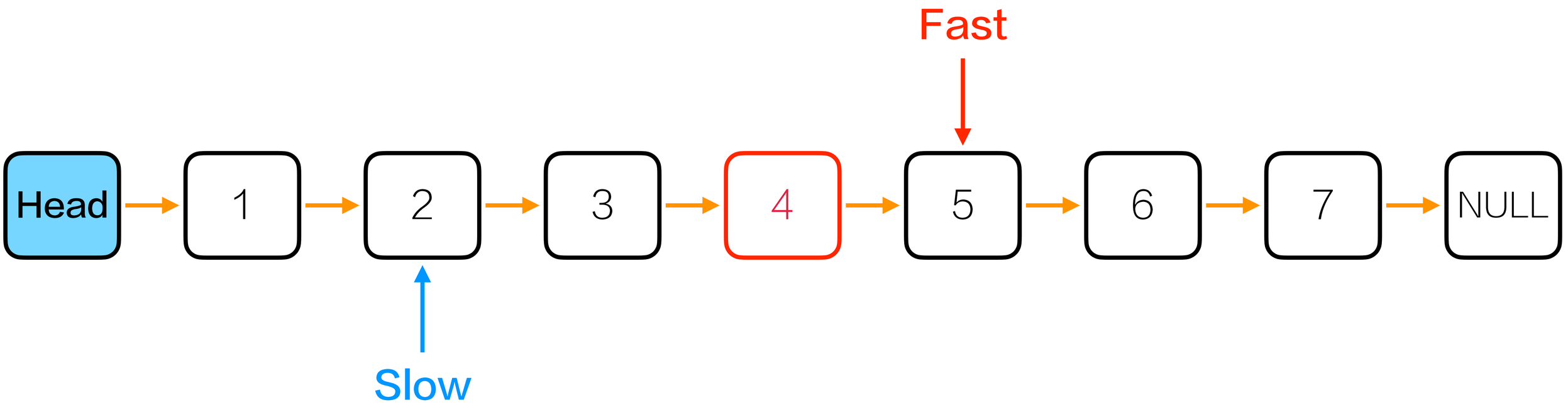
单链表应用 – 删除链表中间节点

案例：线性表/链表/test13.c



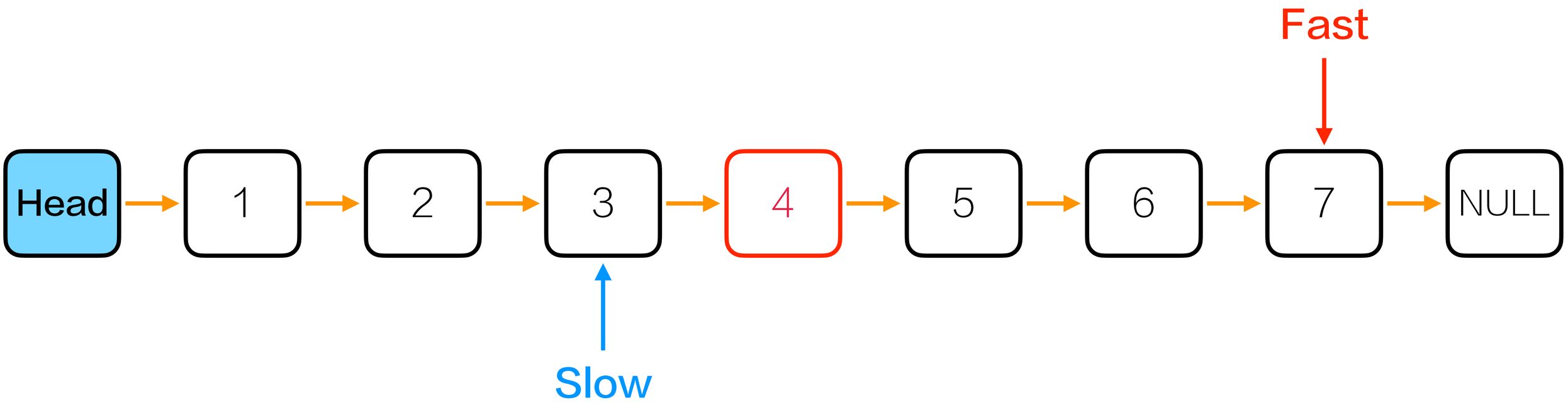
单链表应用 – 删除链表中间节点

案例：线性表/链表/test13.c



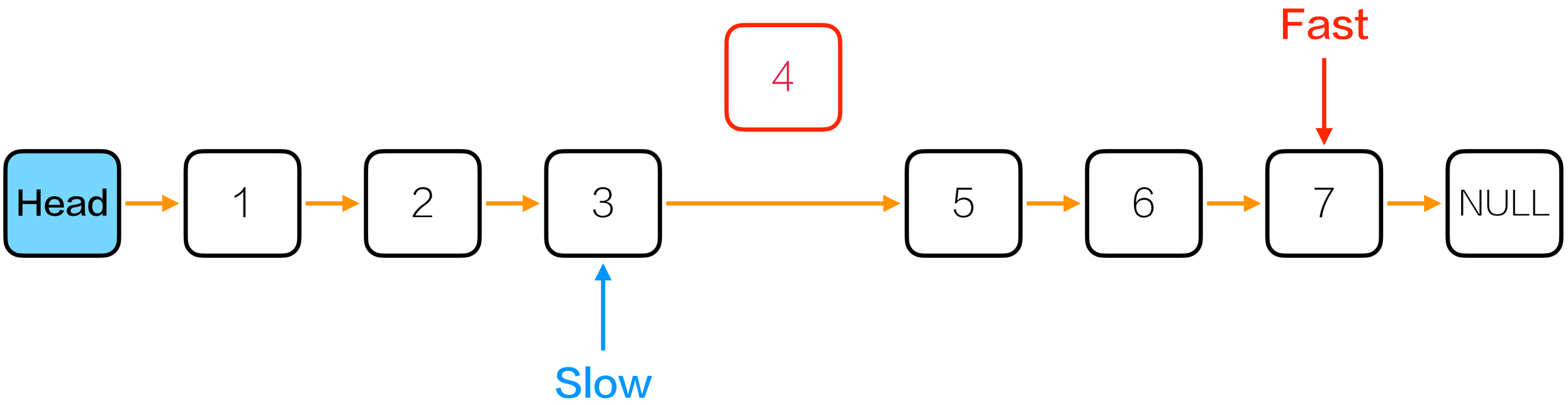
单链表应用 – 删除链表中间节点

案例：线性表/链表/test13.c



单链表应用 – 删除链表中间节点

案例：线性表/链表/test13.c



单链表应用 – 删除链表中间节点

案例：线性表/链表/test13.c

```
int delMiddleNode(Node *head)
{
    Node *fast = head->next;
    Node *slow = head;
    while(fast != NULL && fast->next != NULL)
    {
        fast = fast->next->next;
        slow = slow->next;
    }
    Node *q = slow->next;
    slow->next = q->next;
    free(q);
    return 1;
}
```

单链表应用

[2019]（ 13分 ）设线性表L=(a_1, a_2, a_3, … , a_{n-2}, a_{n-1}, a_n)采用带头节点的单链表保存，链表中的节点定义如下：

```
typedef struct node
{
    int data;
    struct node* next;
}NODE;
```

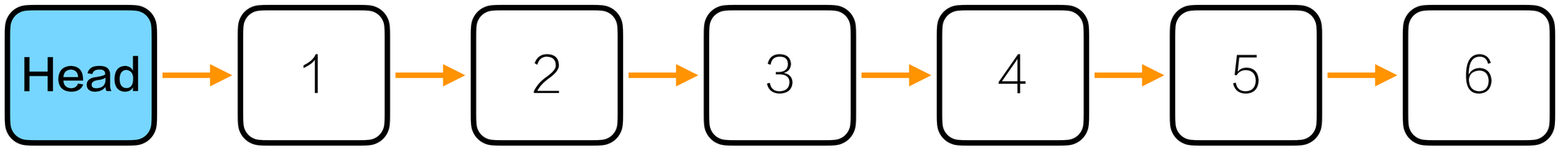
请设计一个空间复杂度为 O(1) 且时间上尽可能高效的算法，重新排列 L 中的各节点，得到线性表L’= (a_1, a_n, a_2, a_{n-1}, a_3, a_{n-2}, …)。要求：

1）给出算法的基本设计思想。

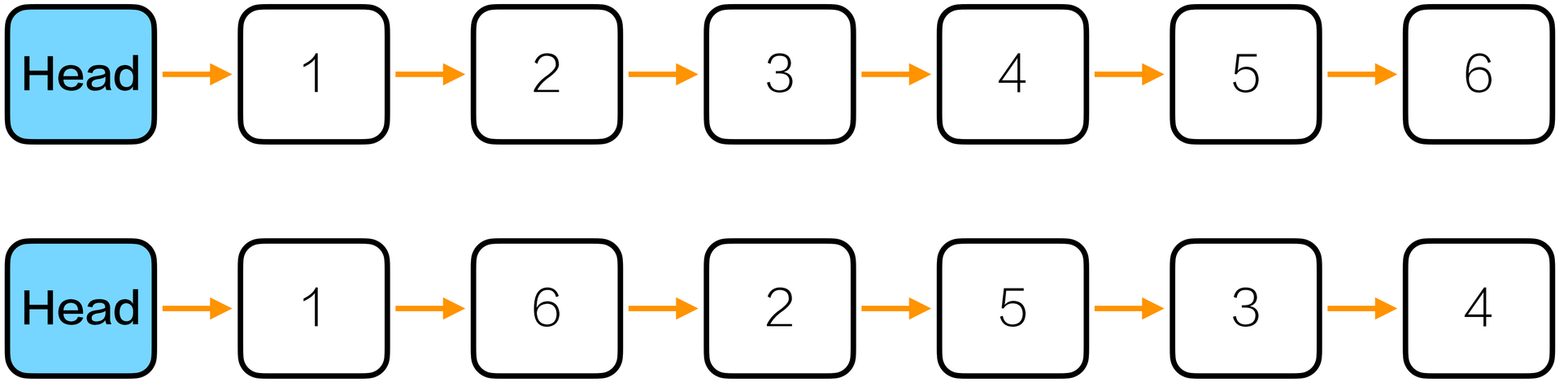
2）根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

3）说明你所设计算法的时间复杂度。

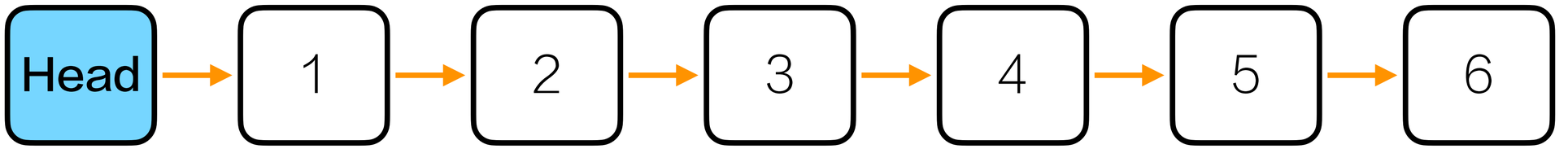
单链表应用



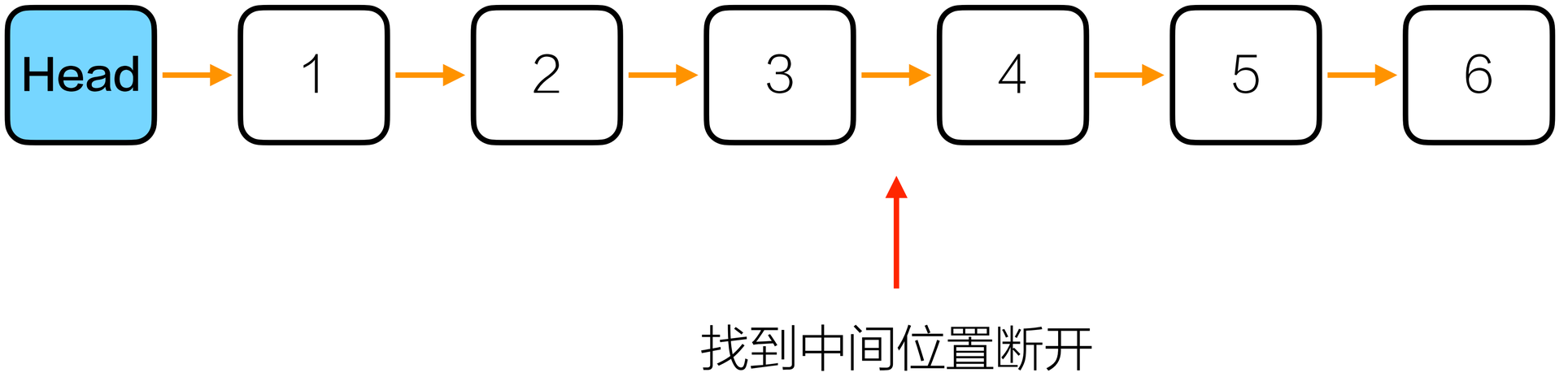
单链表应用



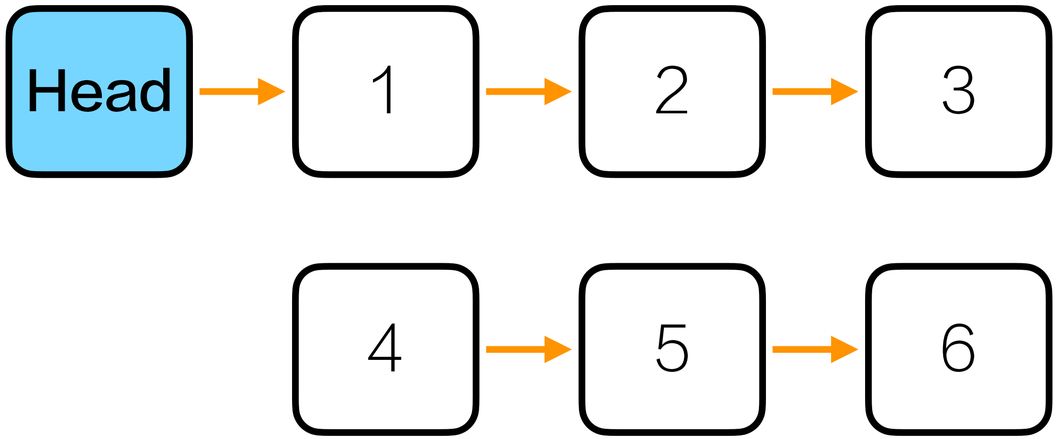
单链表应用



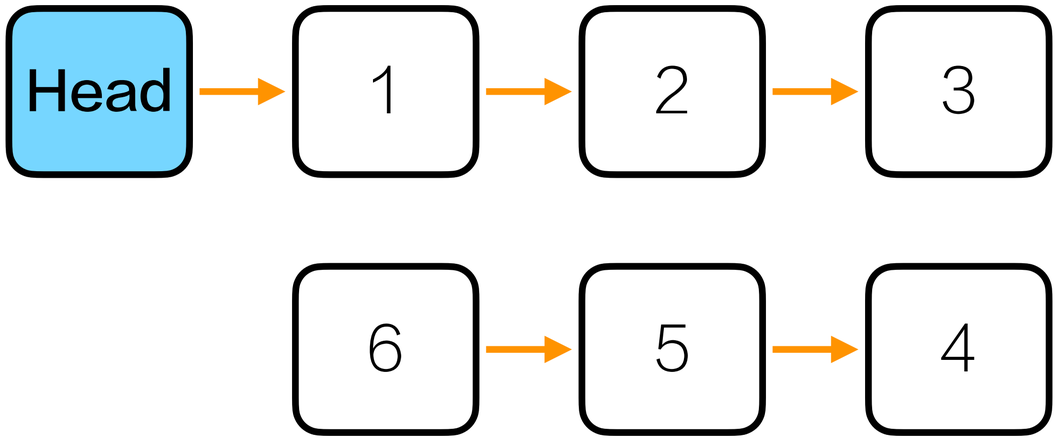
单链表应用



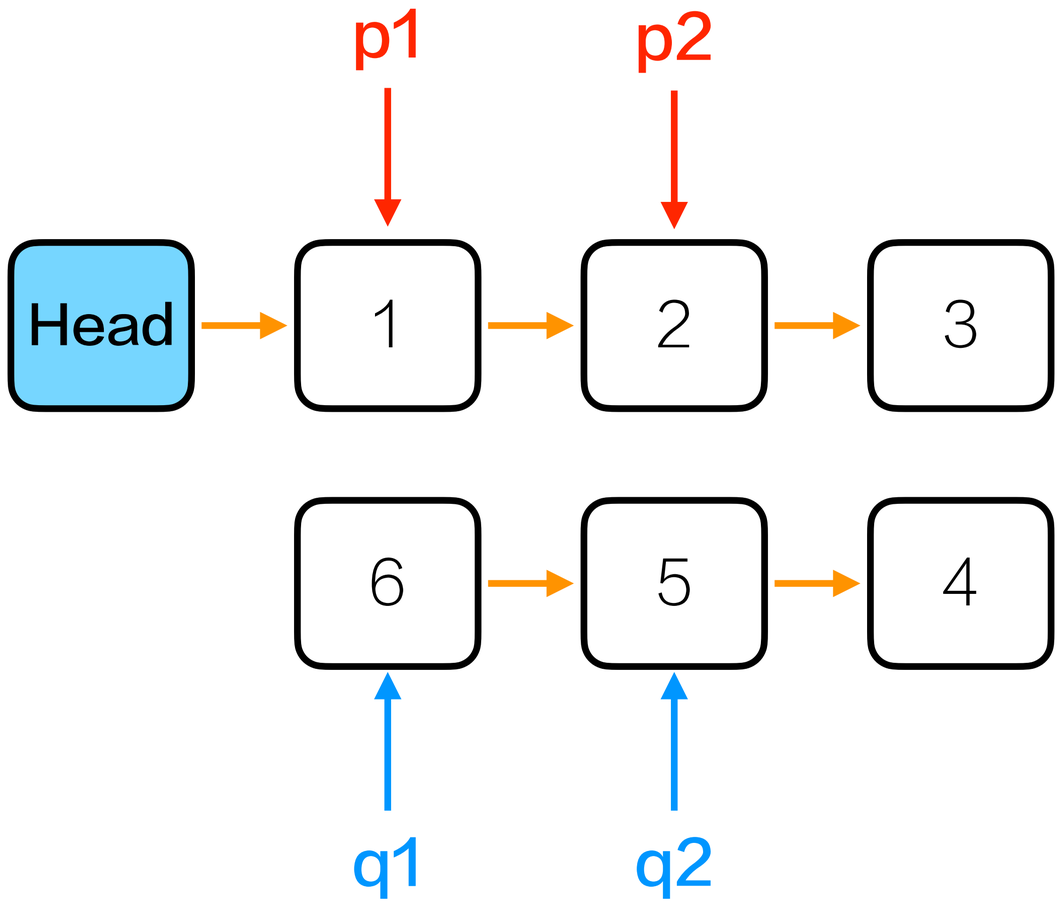
单链表应用



单链表应用

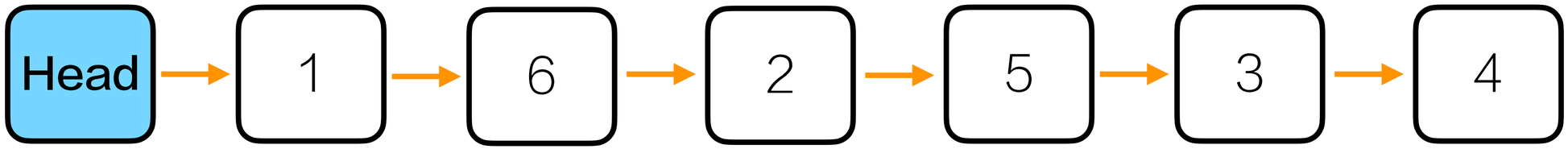


单链表应用



单链表应用

案例：线性表/链表/test14.c

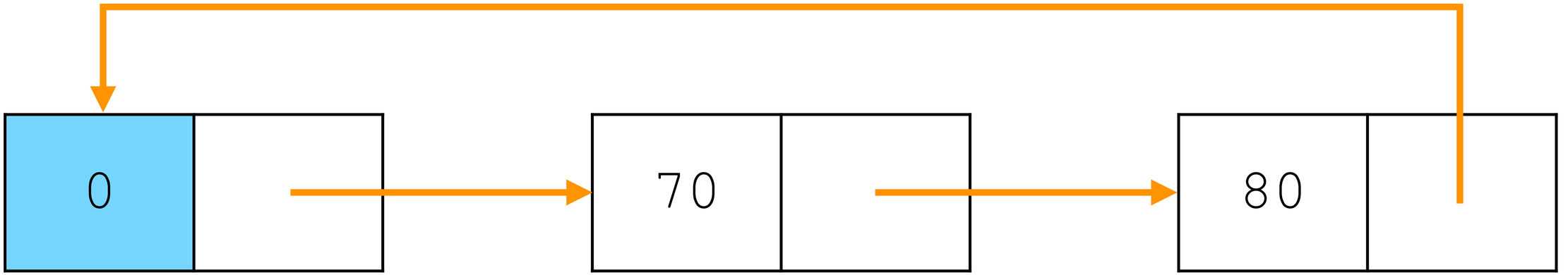


单链表有哪些局限?

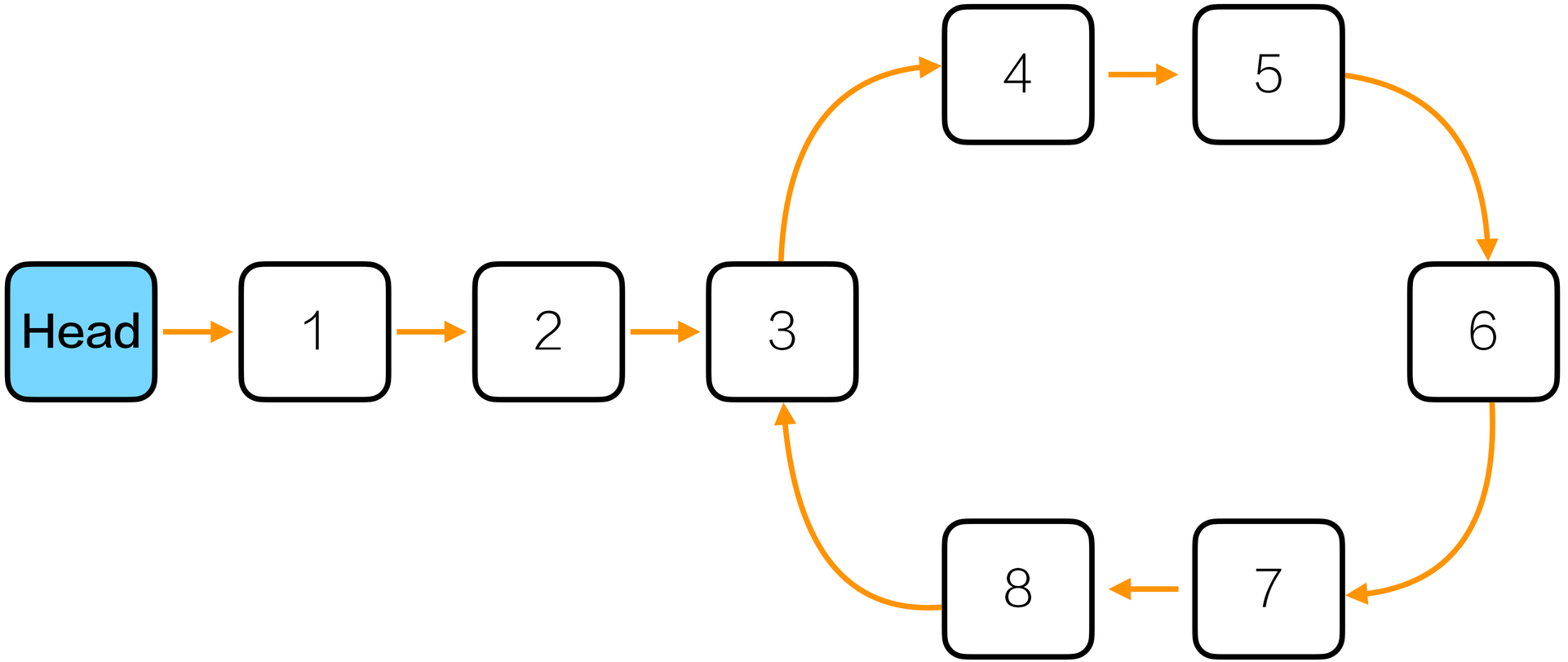
单向循环链表

循环链表（Circular Linked List）是另一种形式的链式存储结构。其特点是表中最后一个节点的指针域指向头节点，整个链表形成一个环。

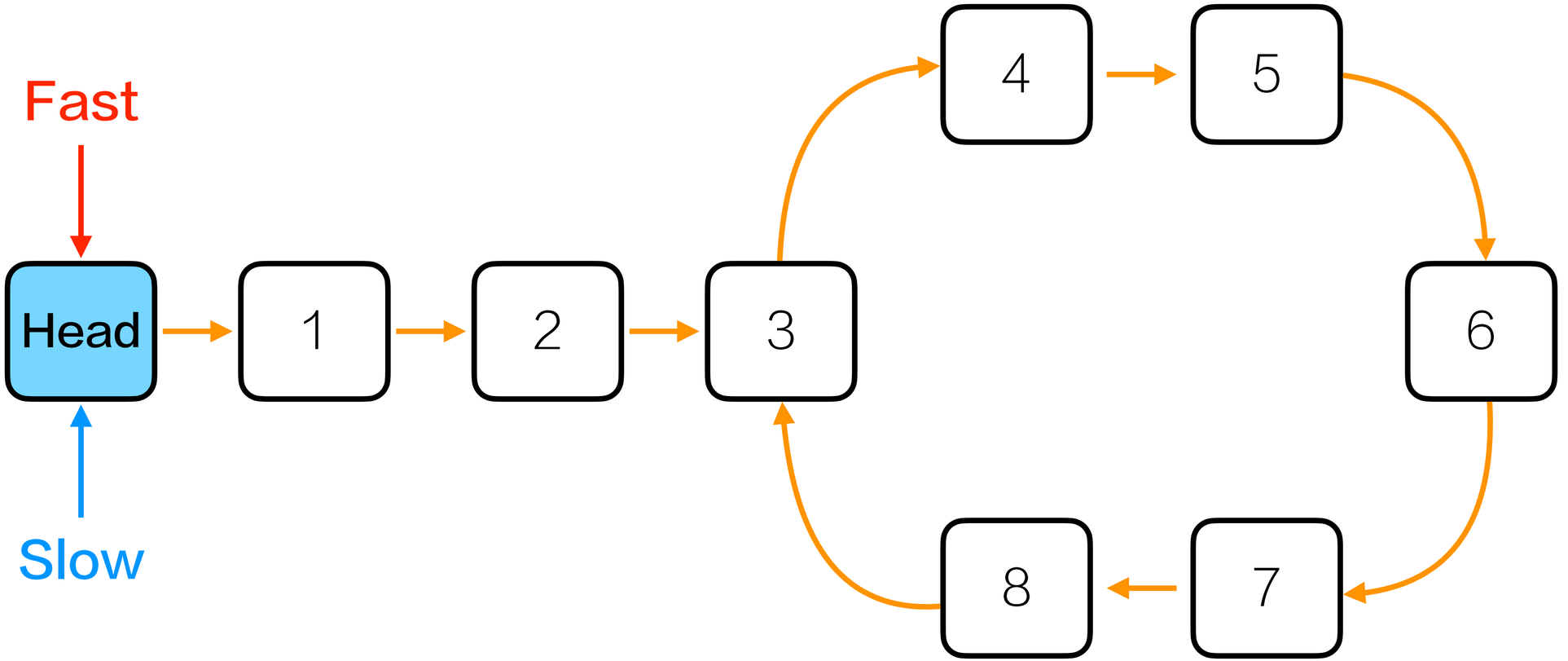
当链表遍历时，判别当前指针 p 是否指向表尾结点的终止条件不同。在单链表中，判别条件为 p!=NULL 或 p->next!=NULL，而循环单链表的判别条件为 p!=L 或 p->next!=L。



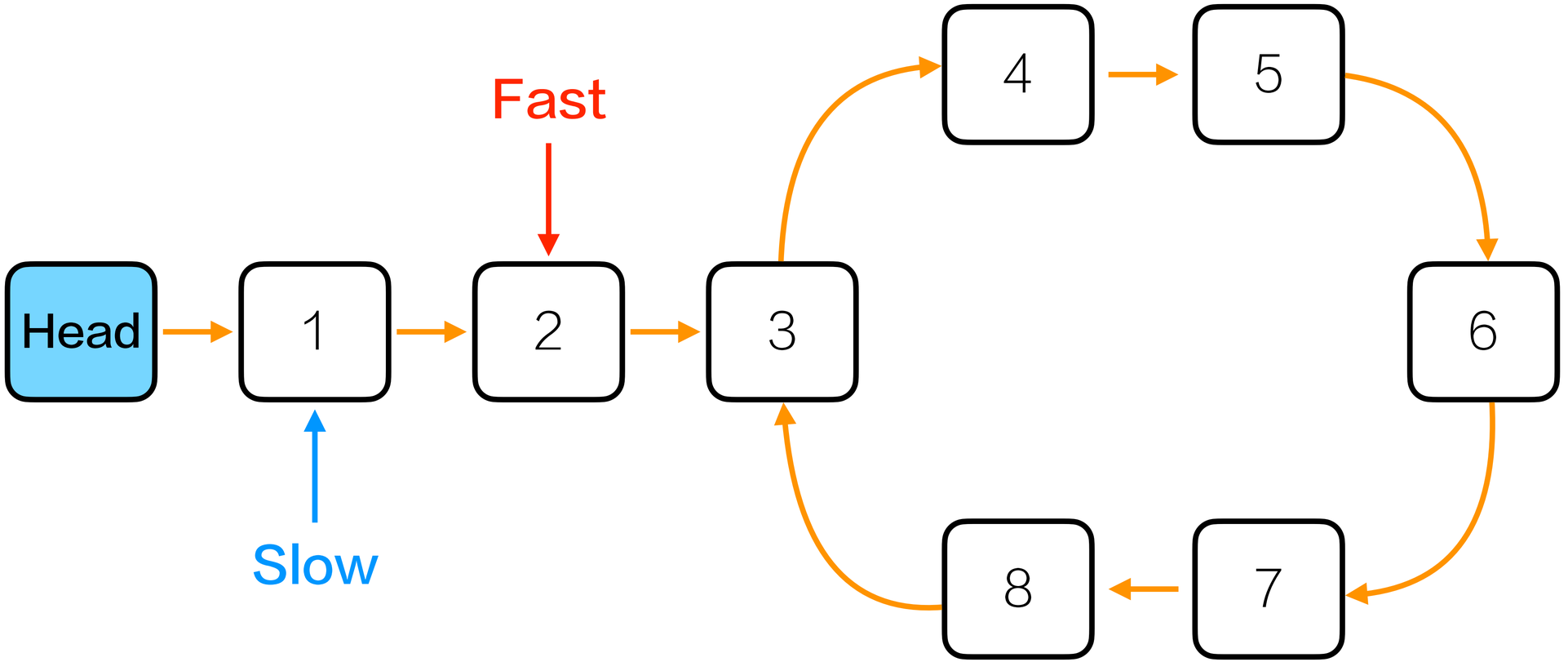
判断链表是否有环



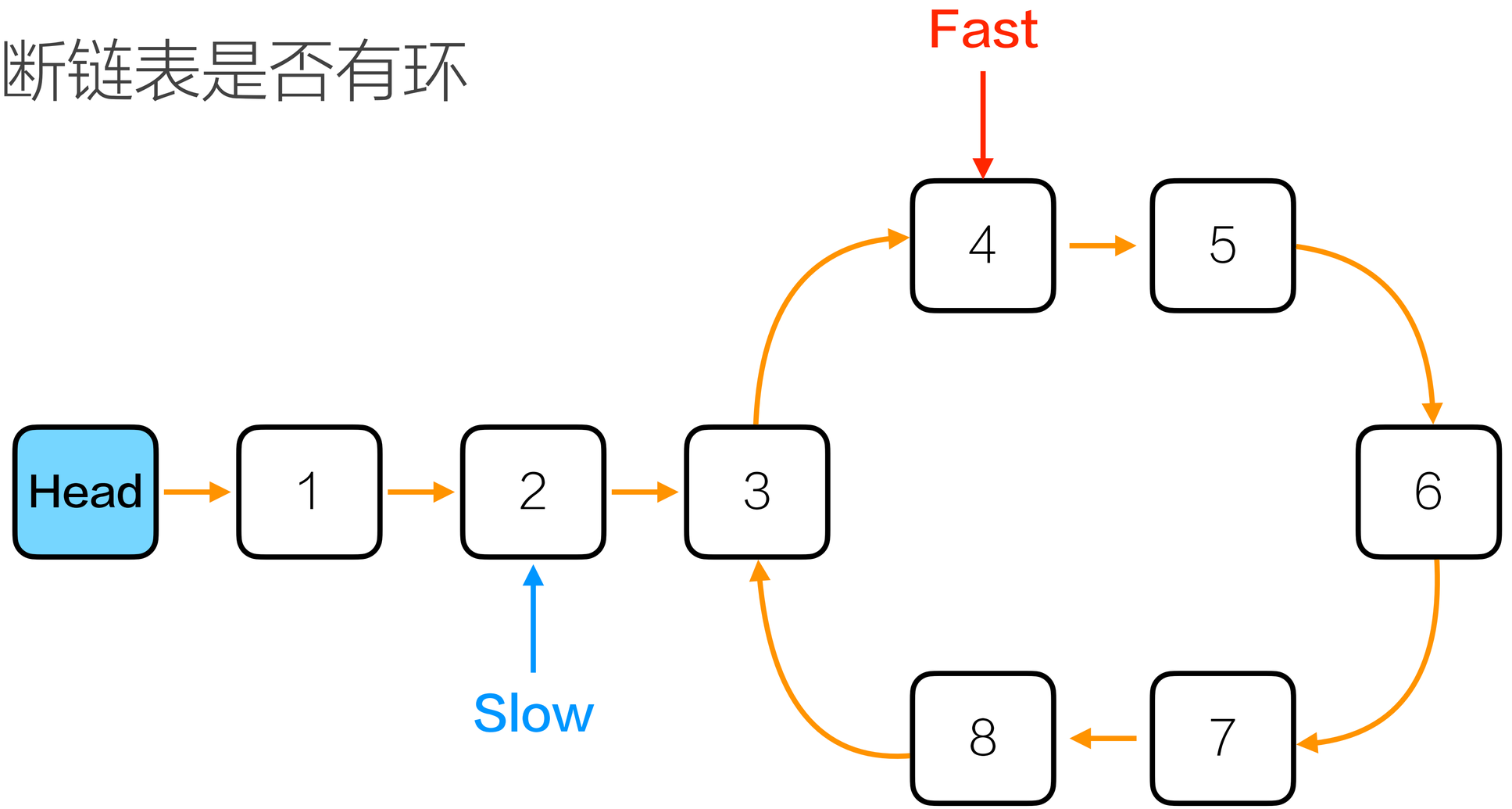
判断链表是否有环



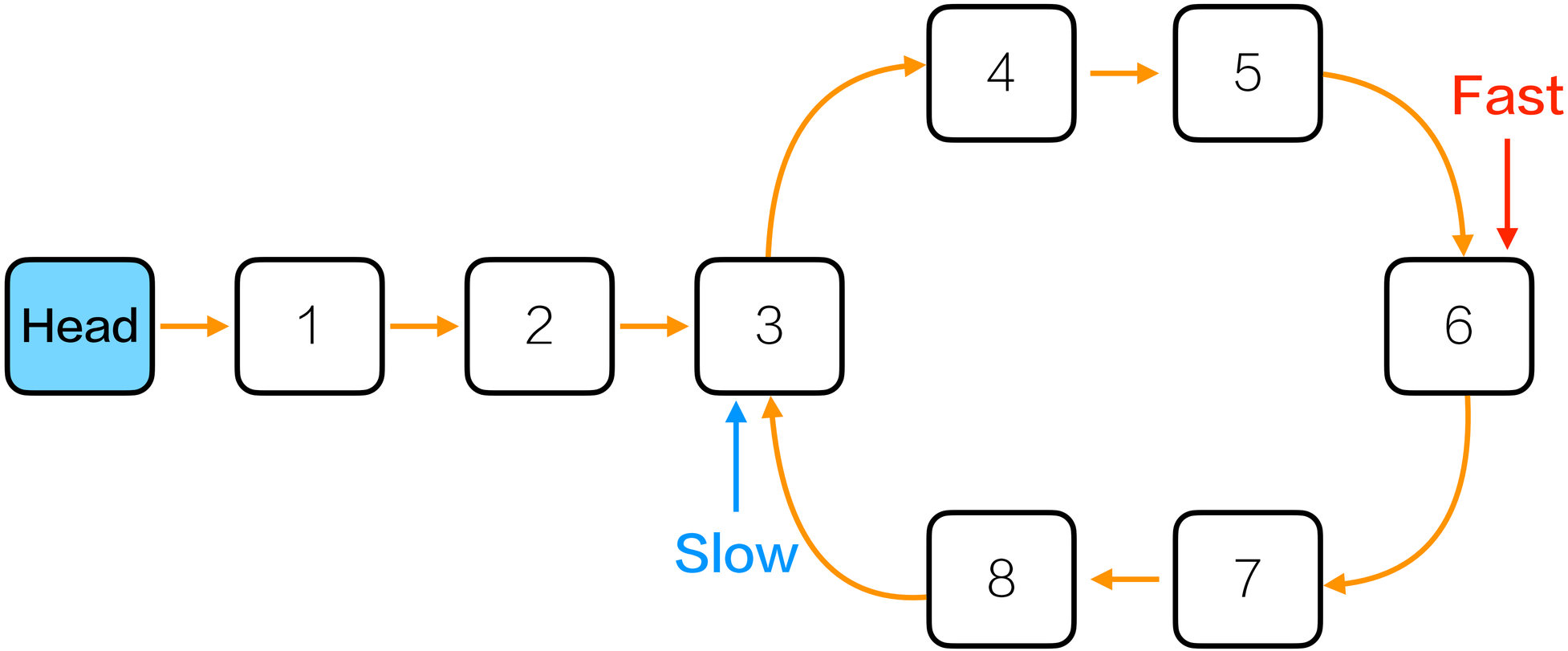
判断链表是否有环



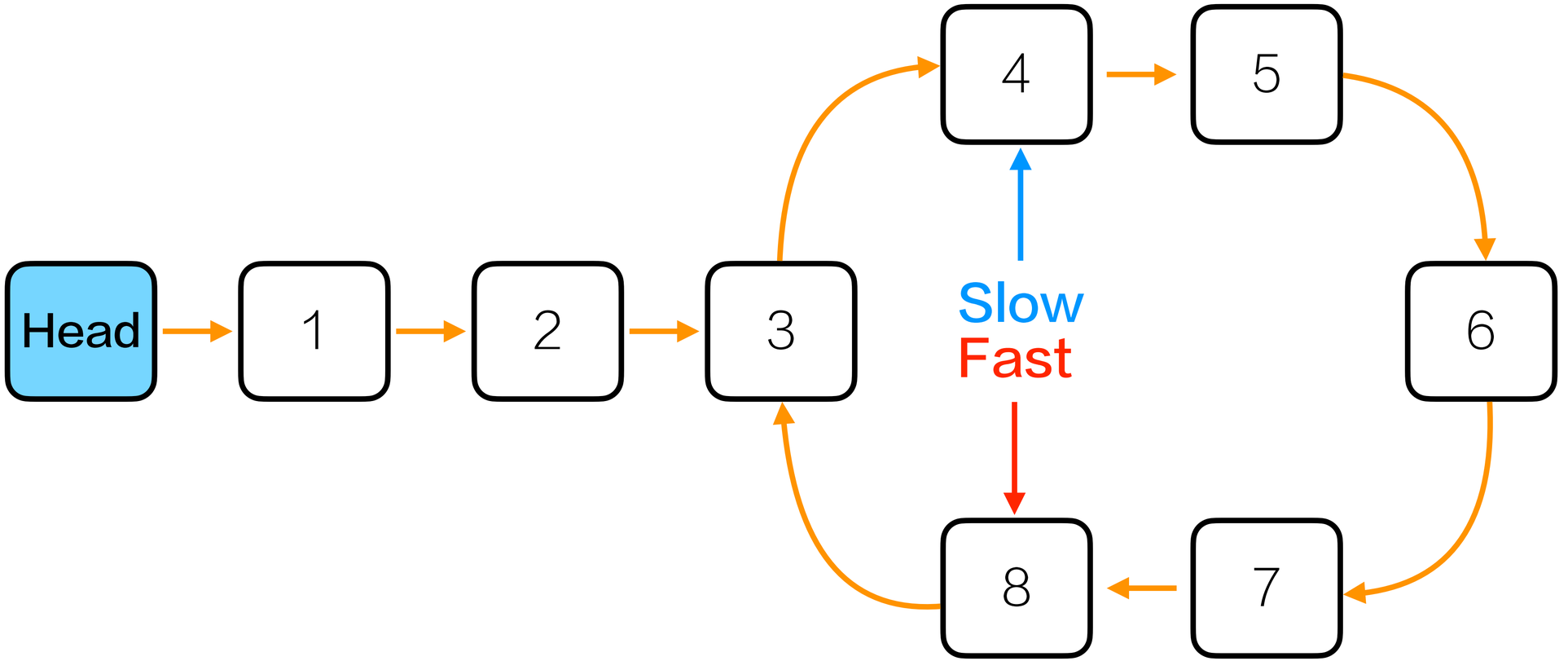
判断链表是否有环



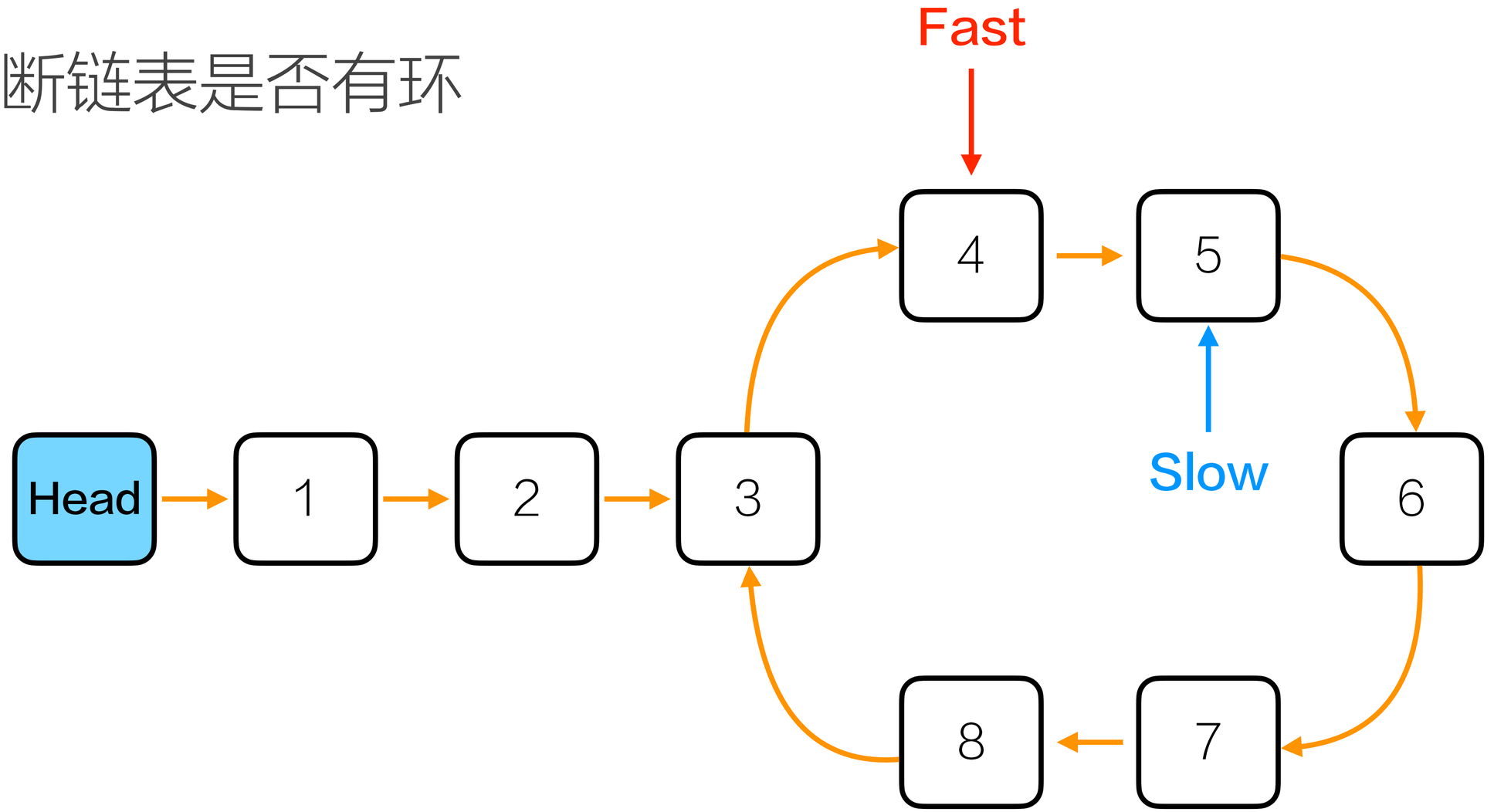
判断链表是否有环



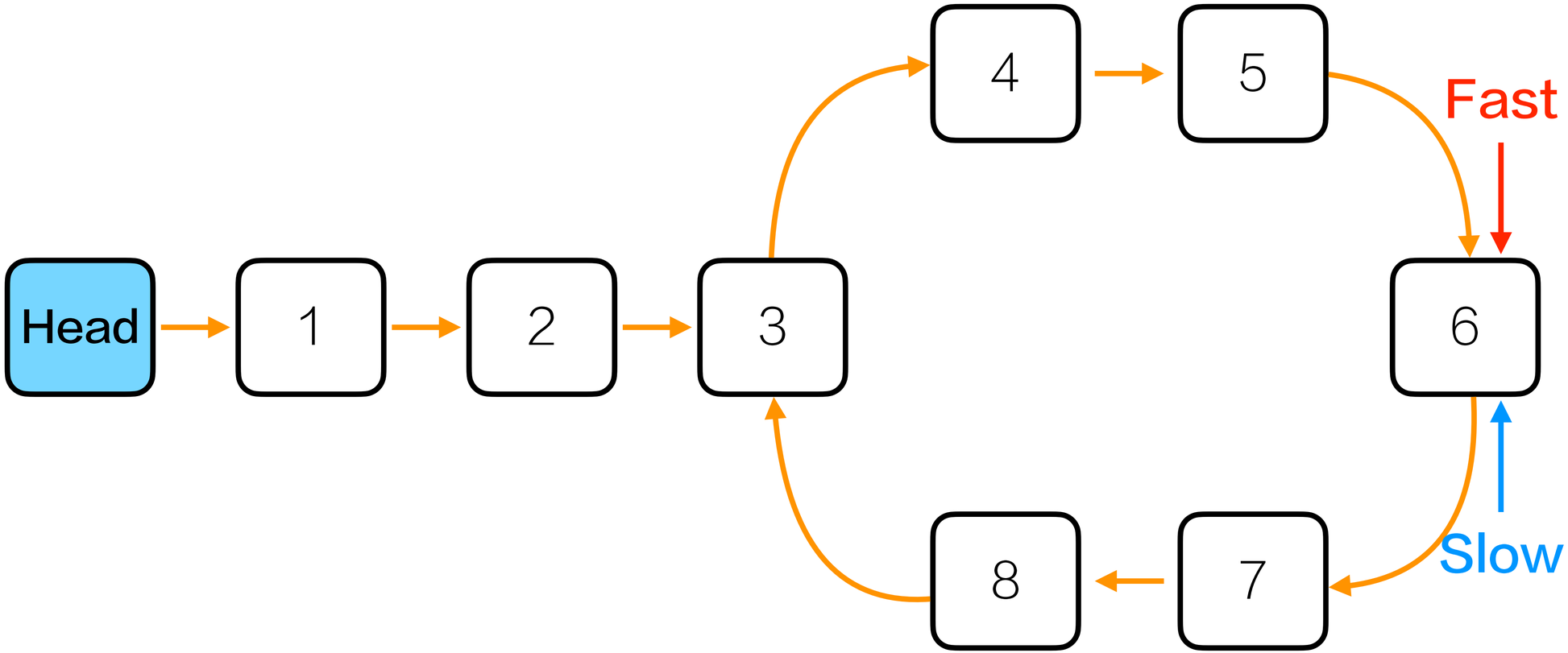
判断链表是否有环



判断链表是否有环



判断链表是否有环

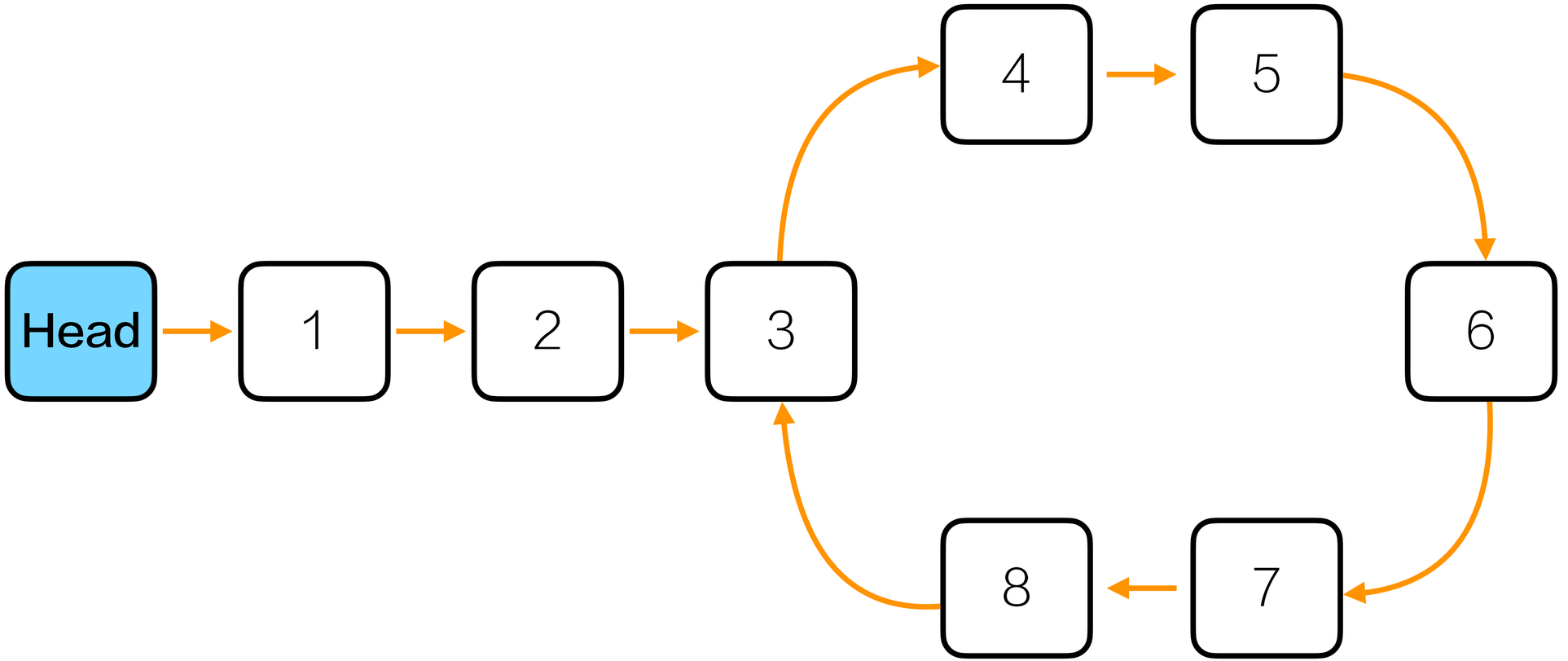


判断链表是否有环

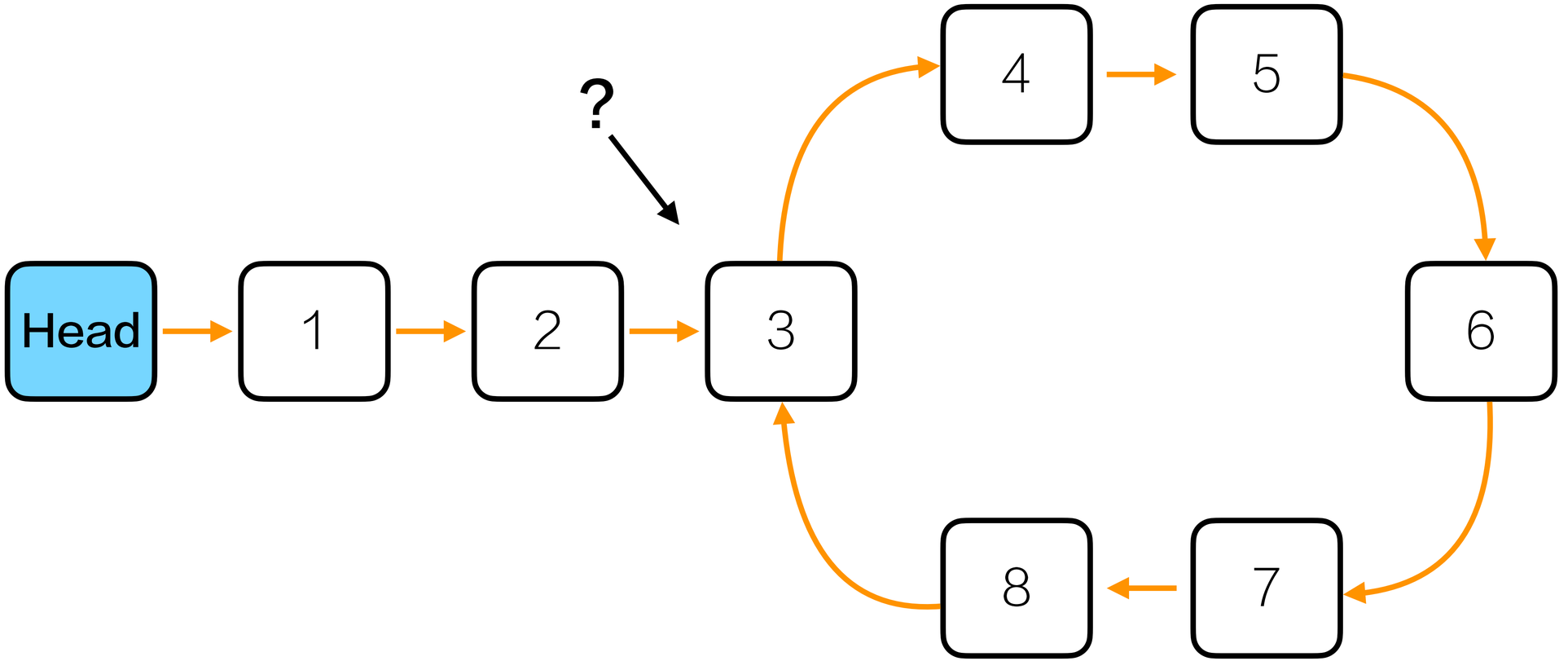
案例：线性表/链表/test15.c

```
int isCycle(Node *head)
{
    Node *fast = head;
    Node *slow = head;
    while(fast != NULL && fast->next != NULL)
    {
        fast = fast->next->next;
        slow = slow->next;
        if (fast == slow)
        {
            return 1;
        }
    }
    return 0;
}
```

链表有环入口在哪里?

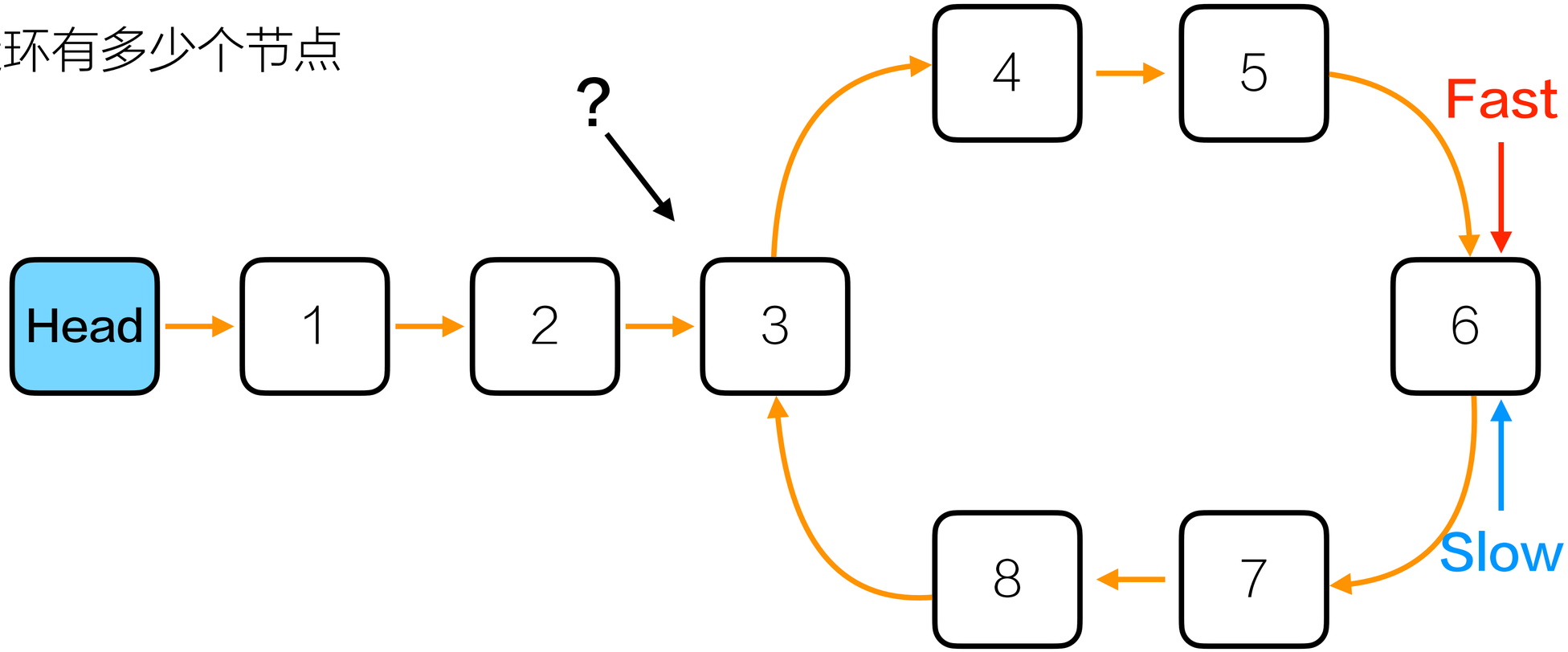


链表有环入口在哪里?



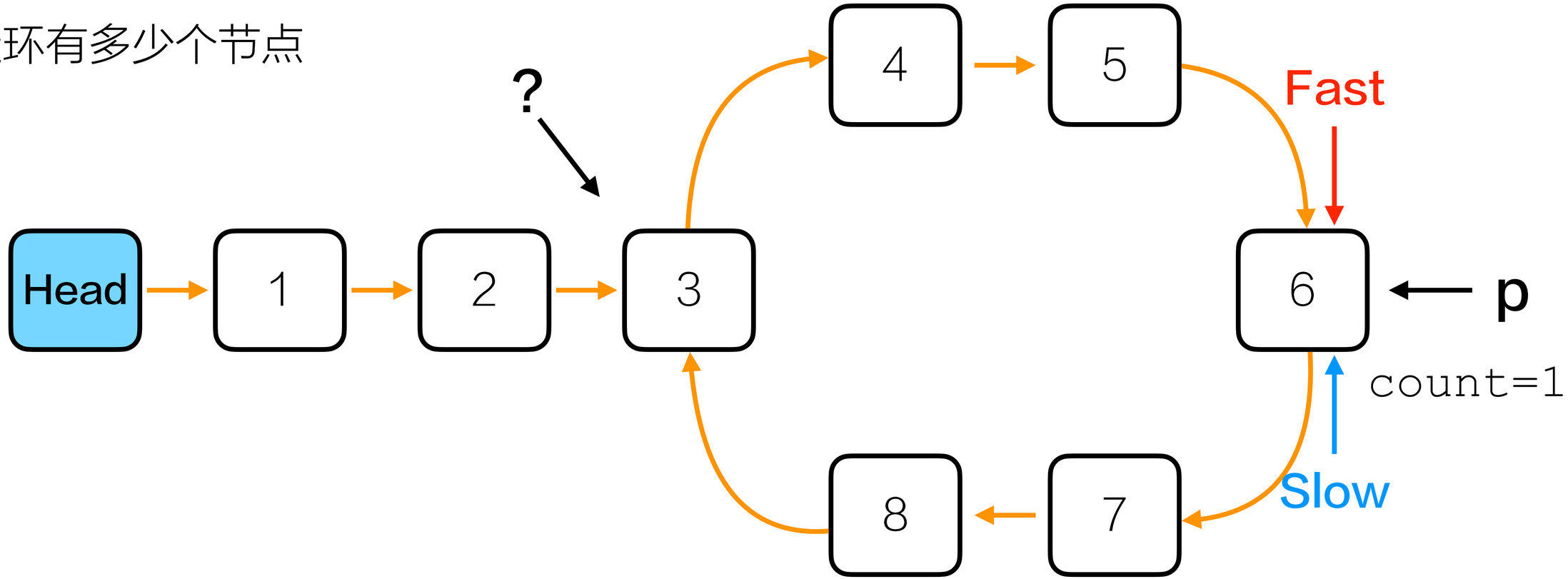
链表有环入口在哪里?

先确定环有多少个节点



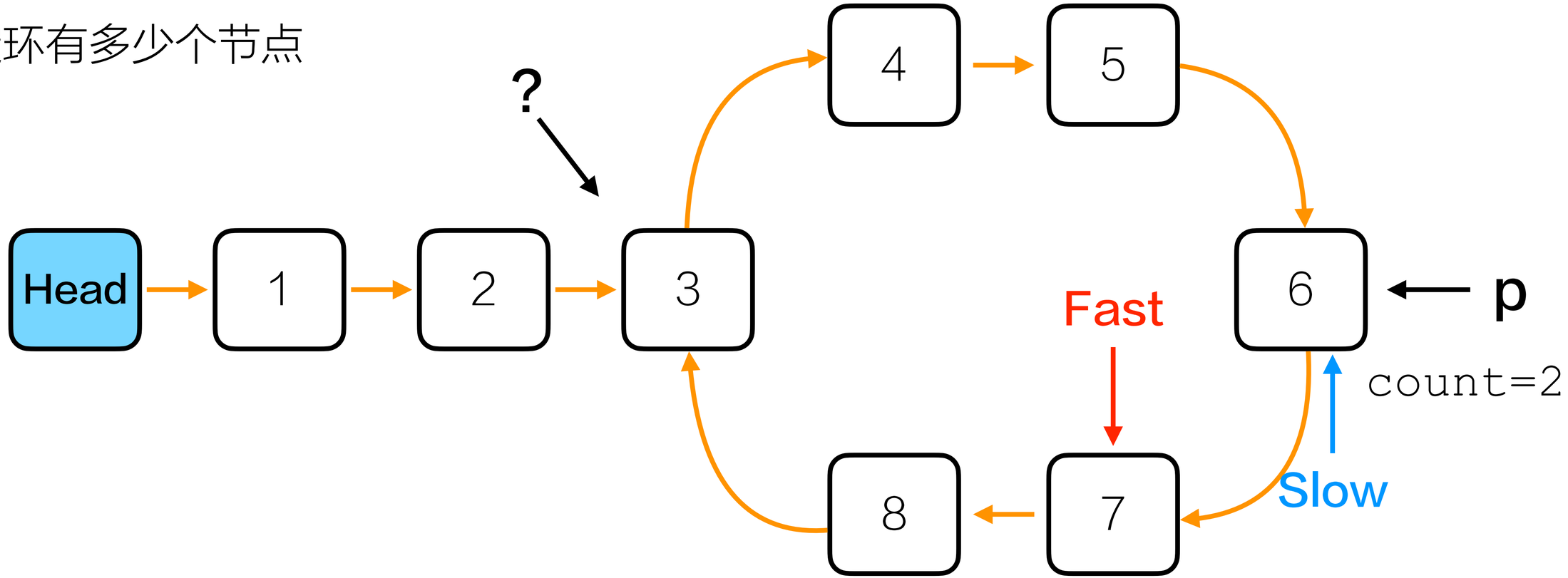
链表有环入口在哪里?

先确定环有多少个节点



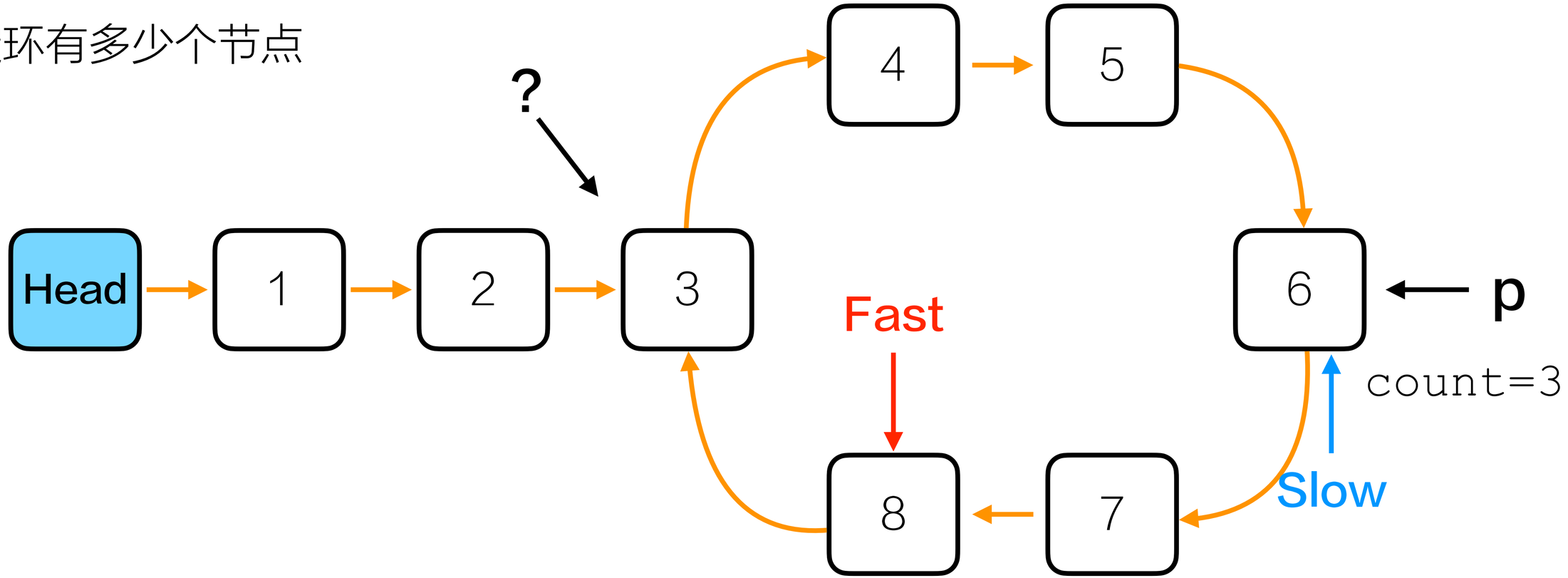
链表有环入口在哪里?

先确定环有多少个节点



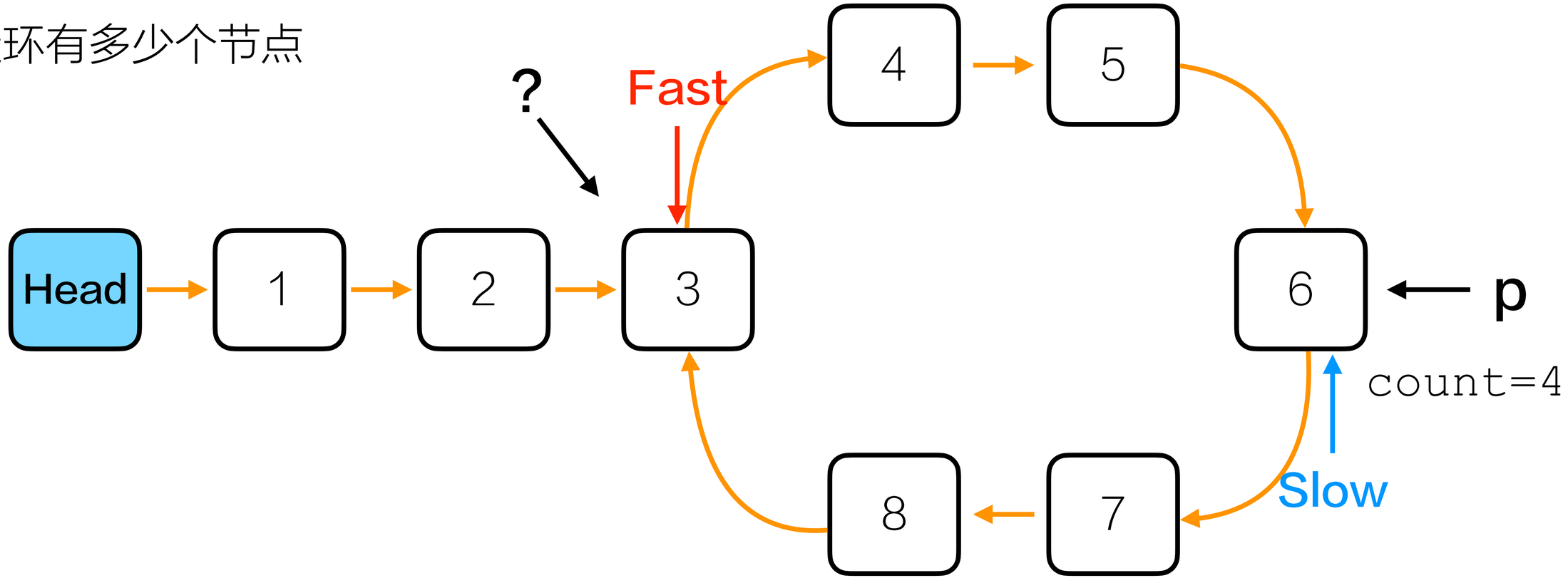
链表有环入口在哪里?

先确定环有多少个节点



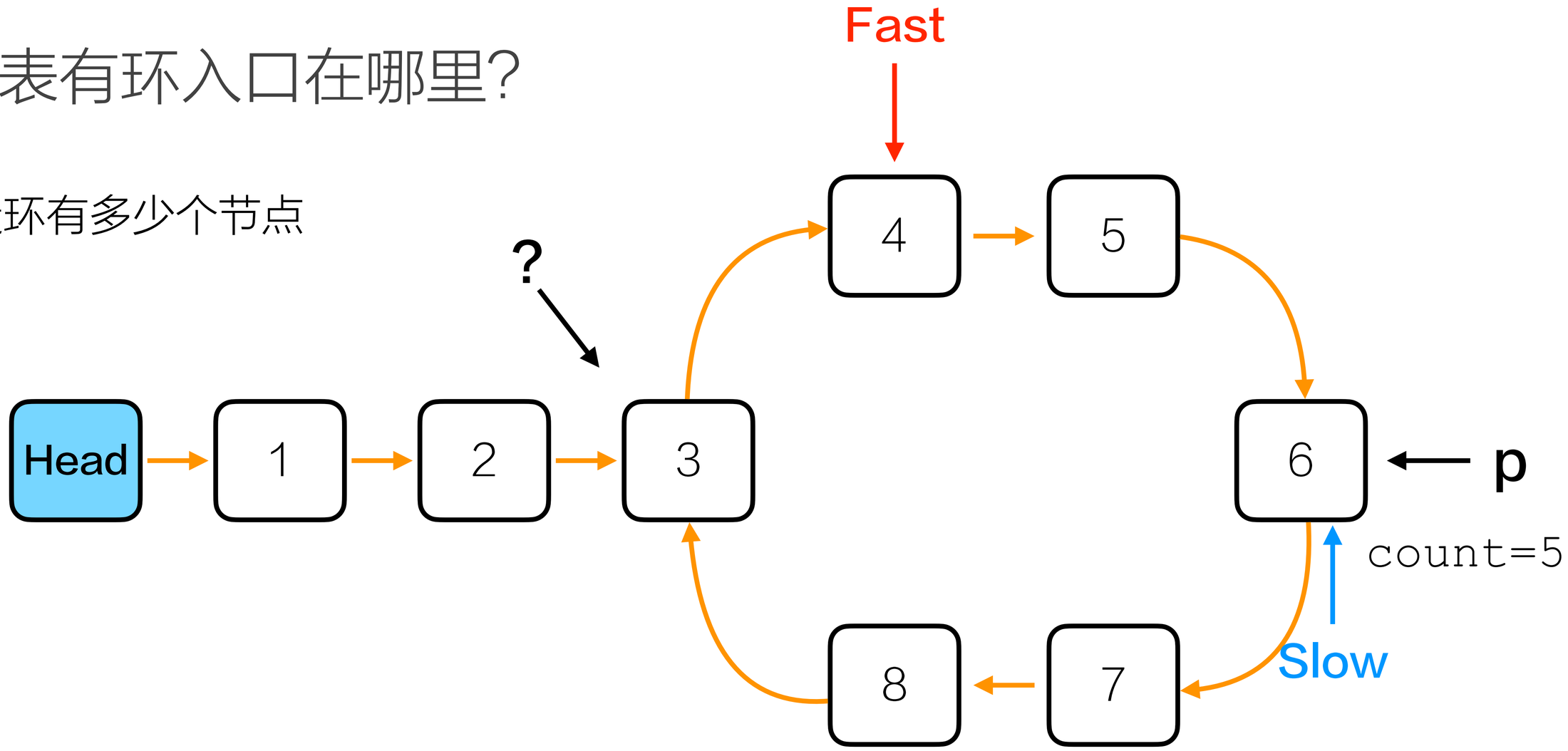
链表有环入口在哪里?

先确定环有多少个节点



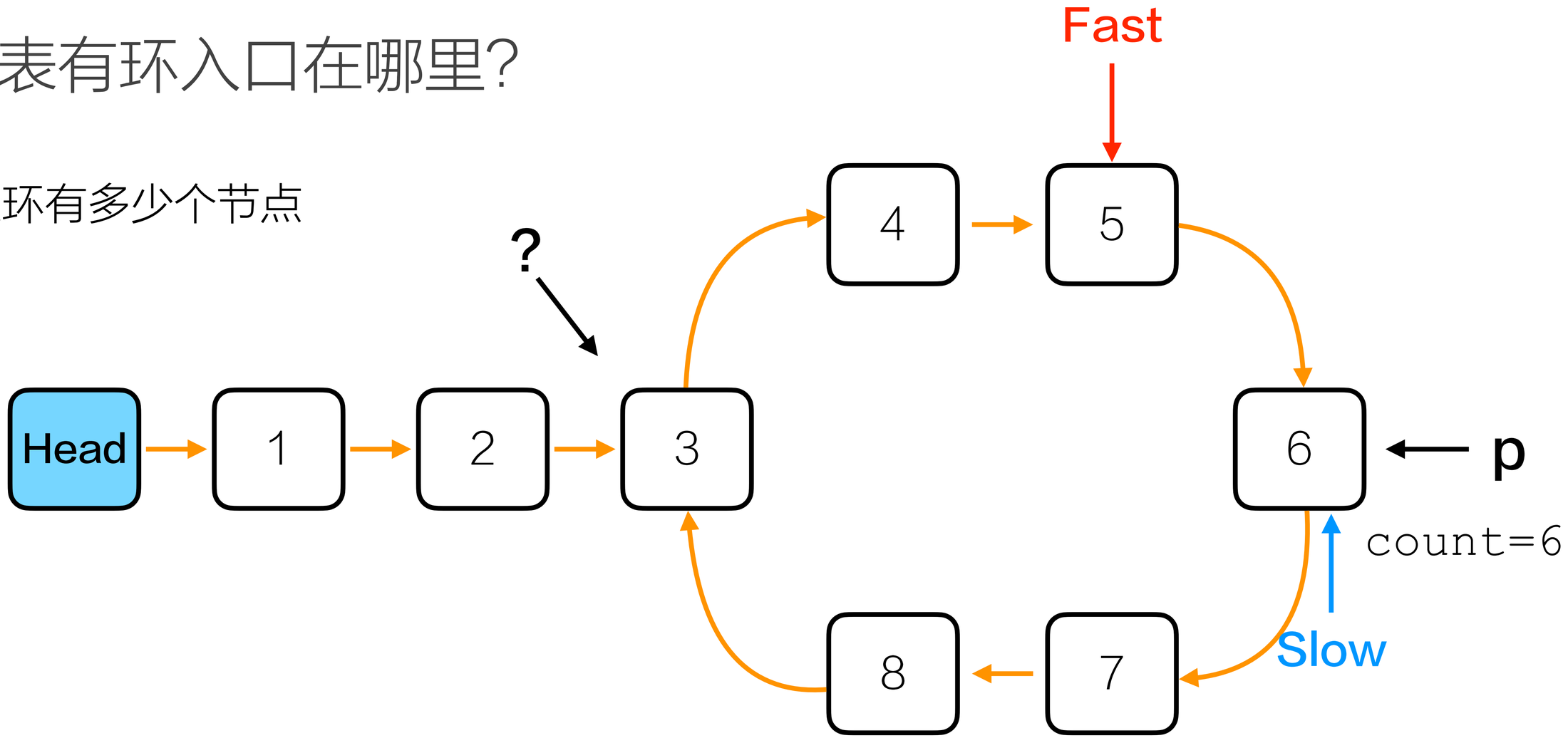
链表有环入口在哪里?

先确定环有多少个节点



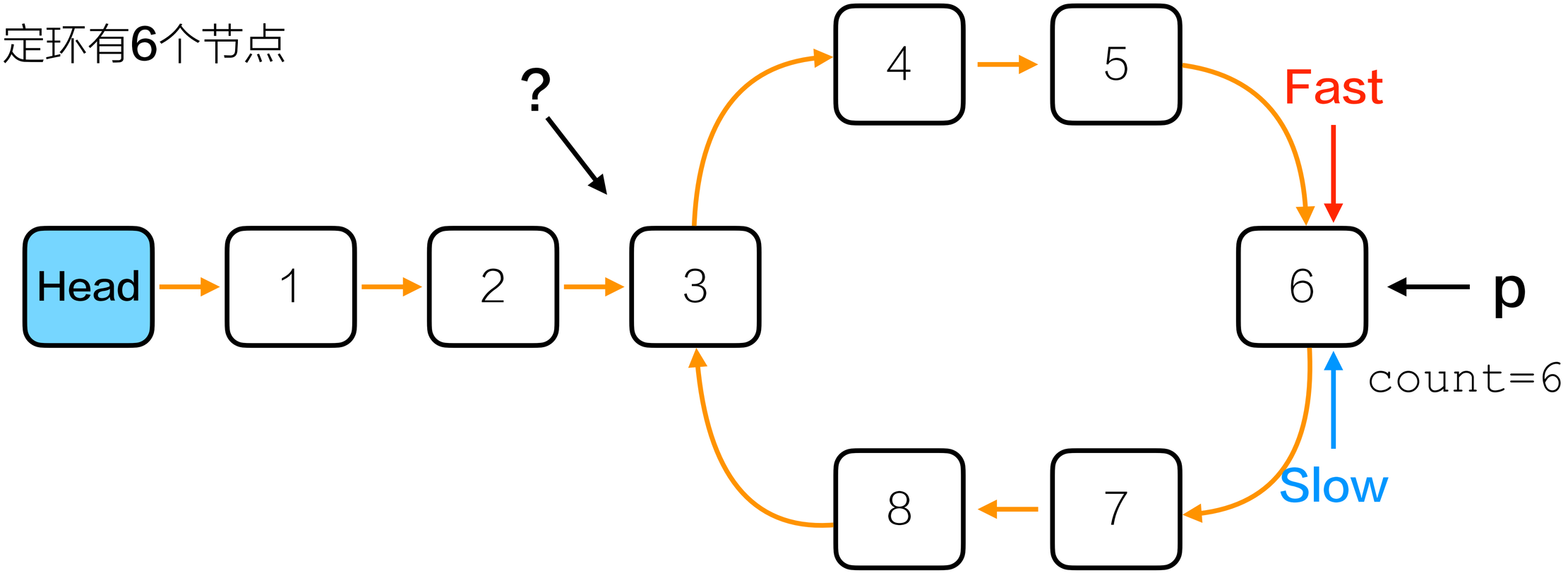
链表有环入口在哪里?

先确定环有多少个节点



链表有环入口在哪里?

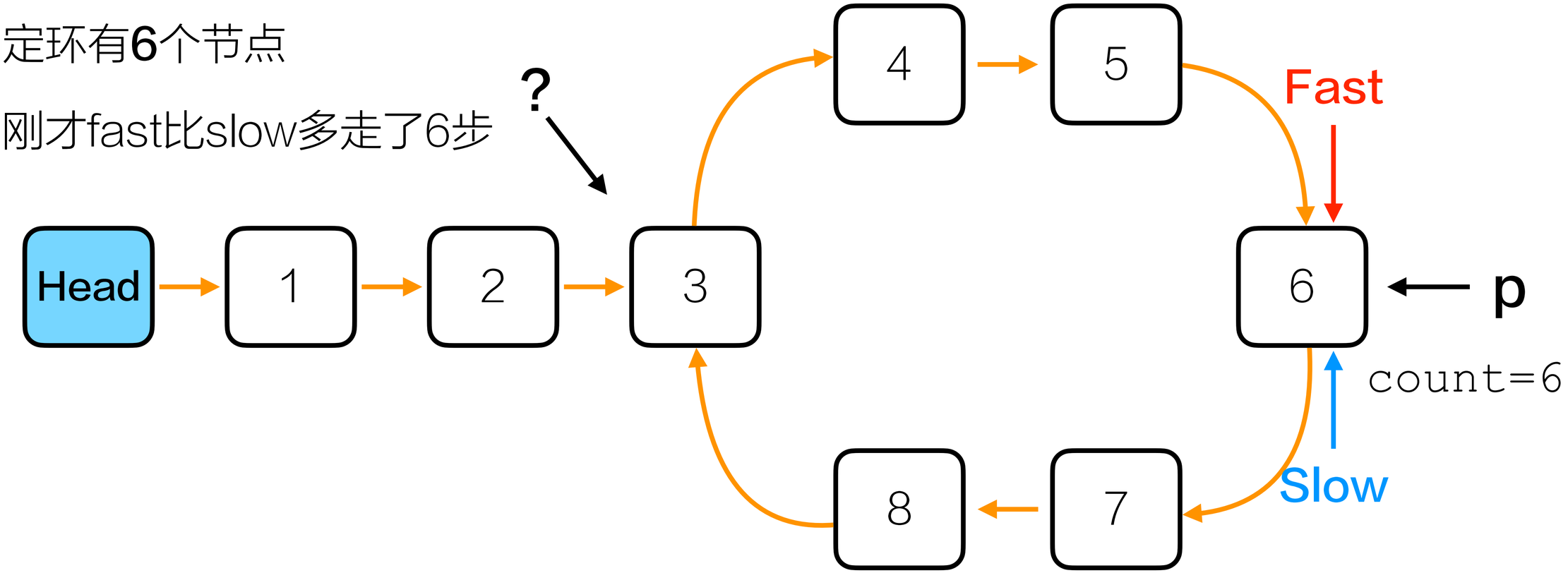
先确定环有6个节点



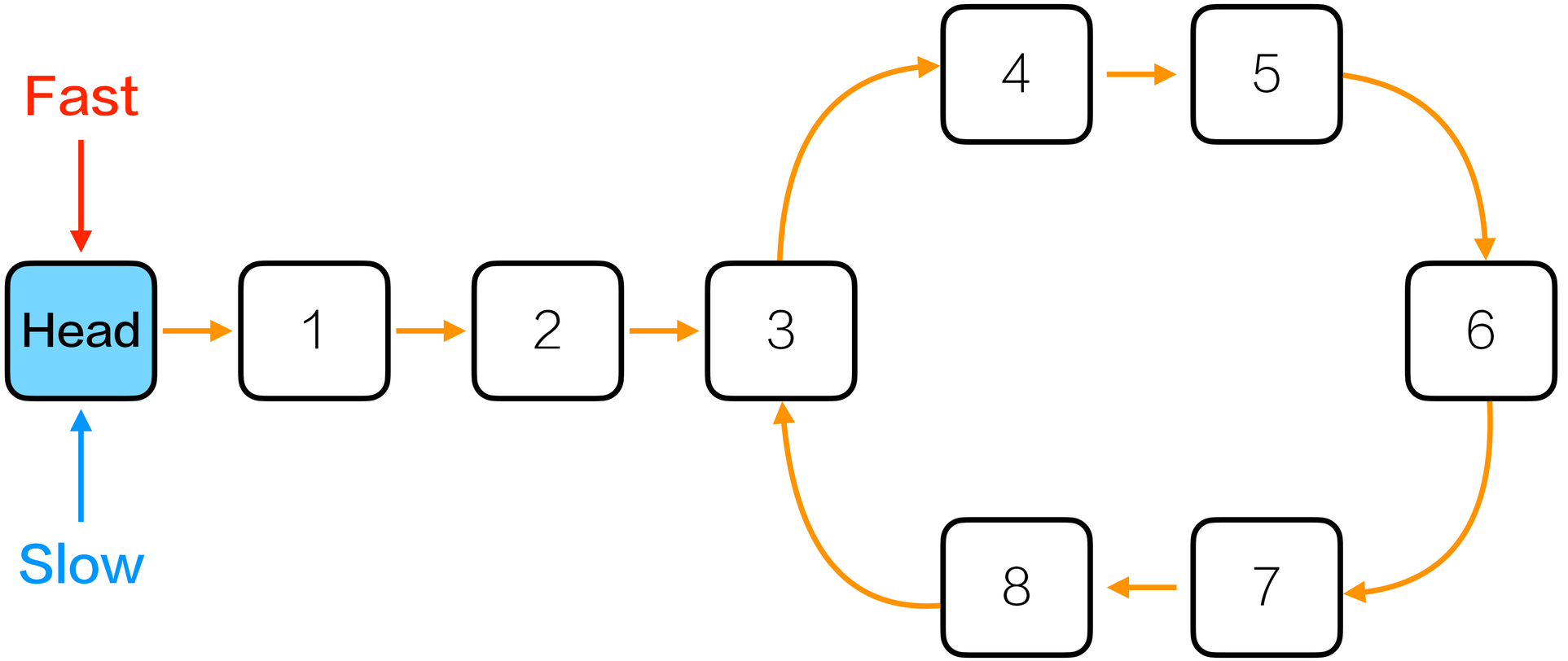
链表有环入口在哪里?

先确定环有6个节点

说明刚才fast比slow多走了6步

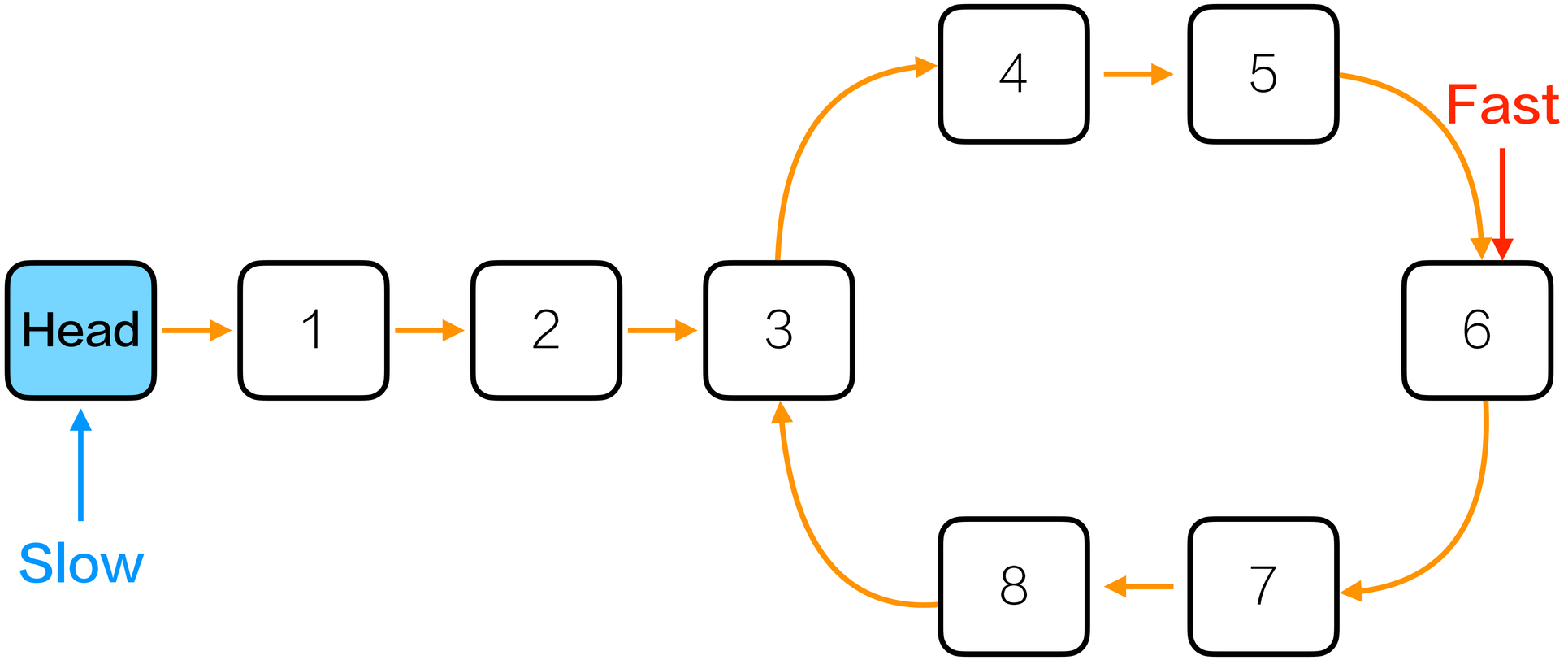


链表有环入口在哪里?



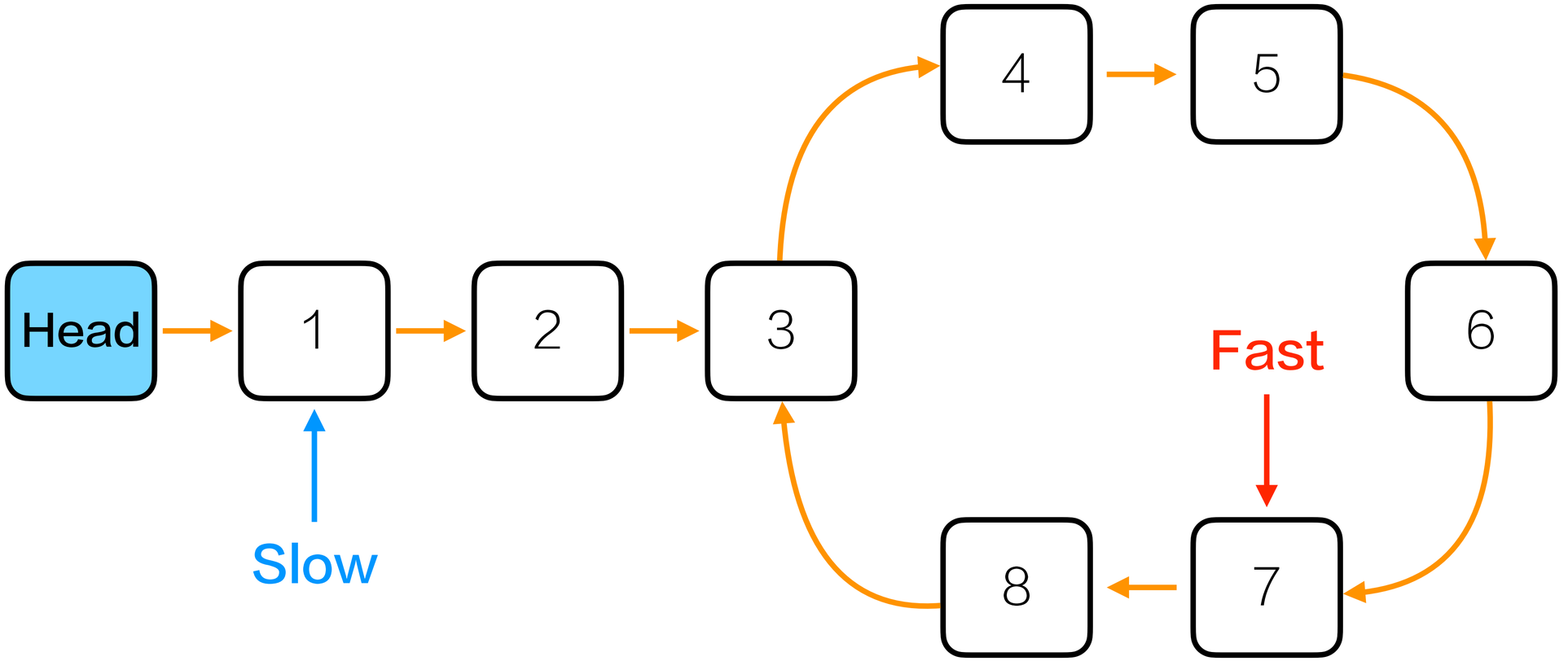
重新让fast与slow指向头节点，fast比slow先走6步，然后同步走。

链表有环入口在哪里?



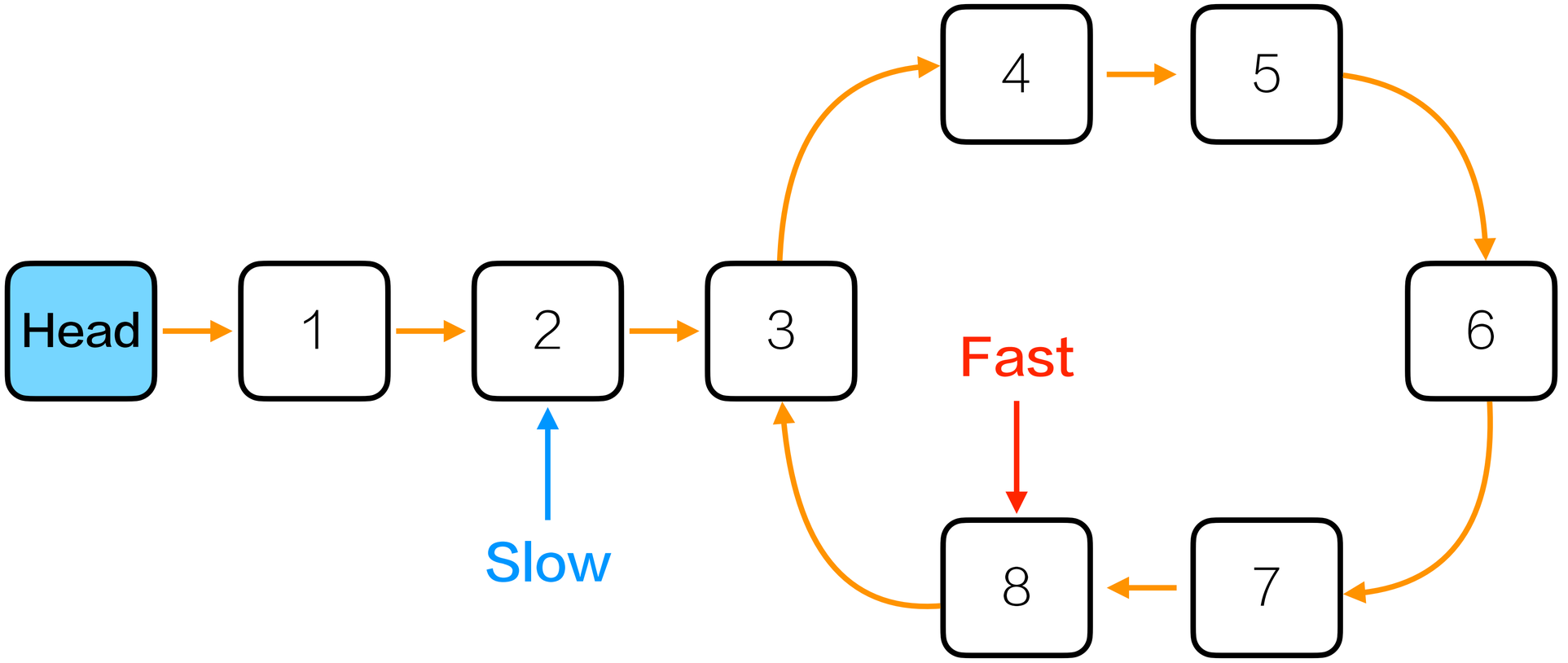
重新让fast与slow指向头节点，fast比slow先走6步，然后同步走。

链表有环入口在哪里?



重新让fast与slow指向头节点，fast比slow先走6步，然后同步走。

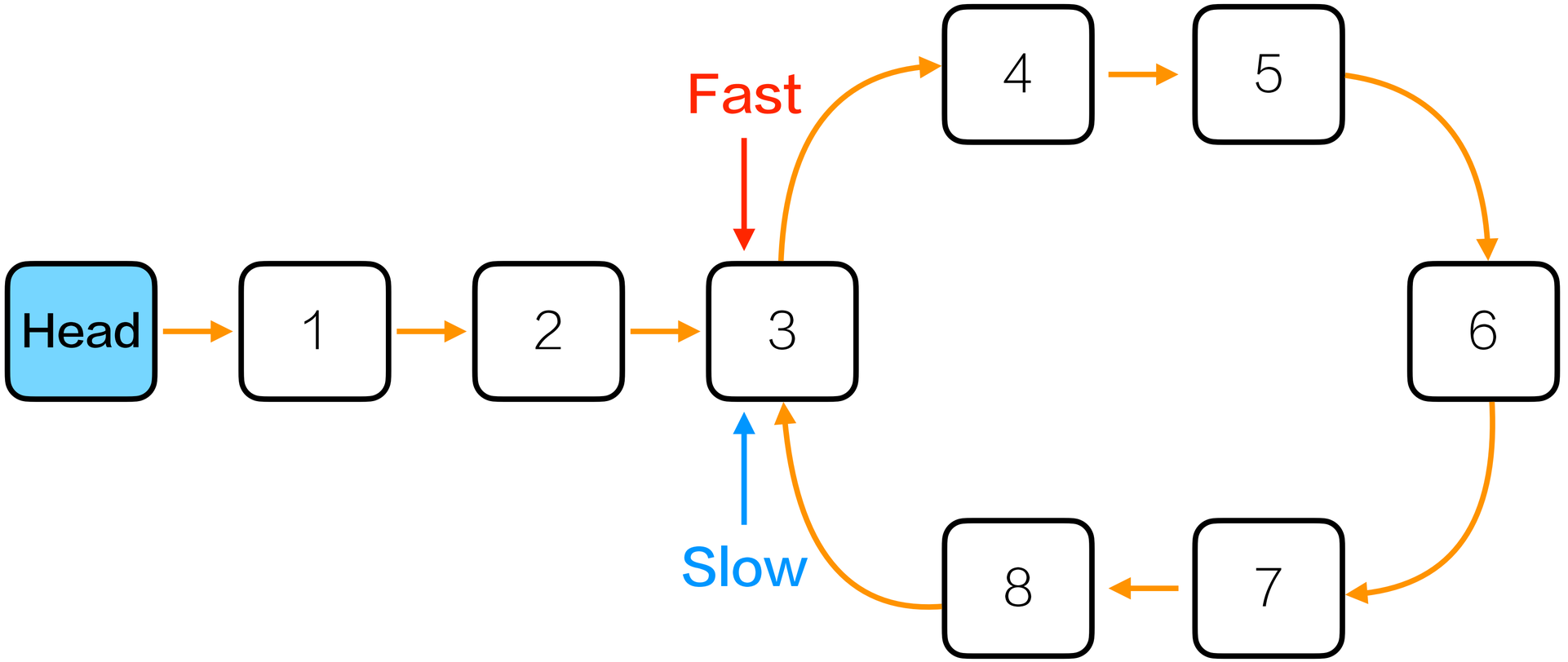
链表有环入口在哪里?



重新让fast与slow指向头节点，fast比slow先走6步，然后同步走。

链表有环入口在哪里?

案例：线性表/链表/test16.c



重新让fast与slow指向头节点，fast比slow先走6步，然后同步走。

双向链表

链式存储结构的节点中只有一个指示直接后继的指针域，由此，从某个结点出发只能顺指针向后寻查其他节点。若要寻查结点的直接前驱、则必须从表头指针出发。换句话说，在单链表中，查找直接后继的执行时间为 O(1)，而查找直接前驱的执行时间为 O(n) 。为克服单链表这种单向性的缺点，可利用双向链表（ Double Linked List ）。在双向链表的节点中有两个指针域，一个指向直接后继，另一个指向直接前驱。

双向链表

```
typedef int ElemType;

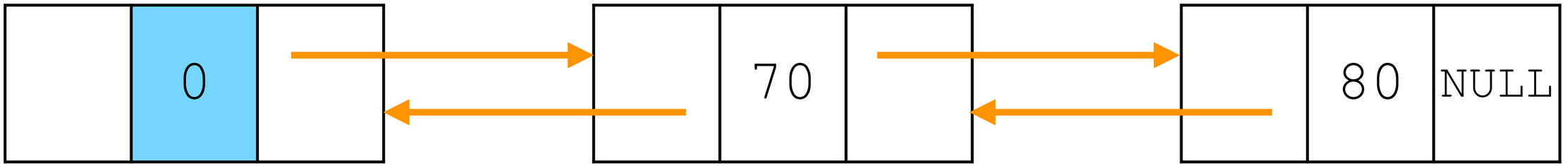

typedef struct node{
    ElemType data;
    struct node *prev, *next;
}Node;
```





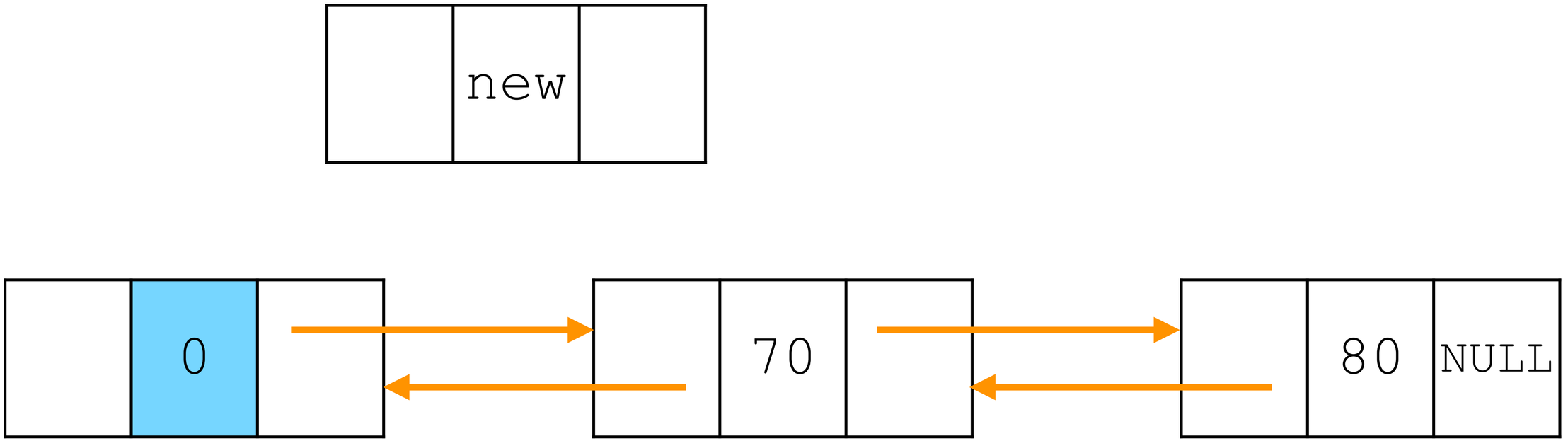
双向链表 – 头插法

案例：线性表/链表/test17.c



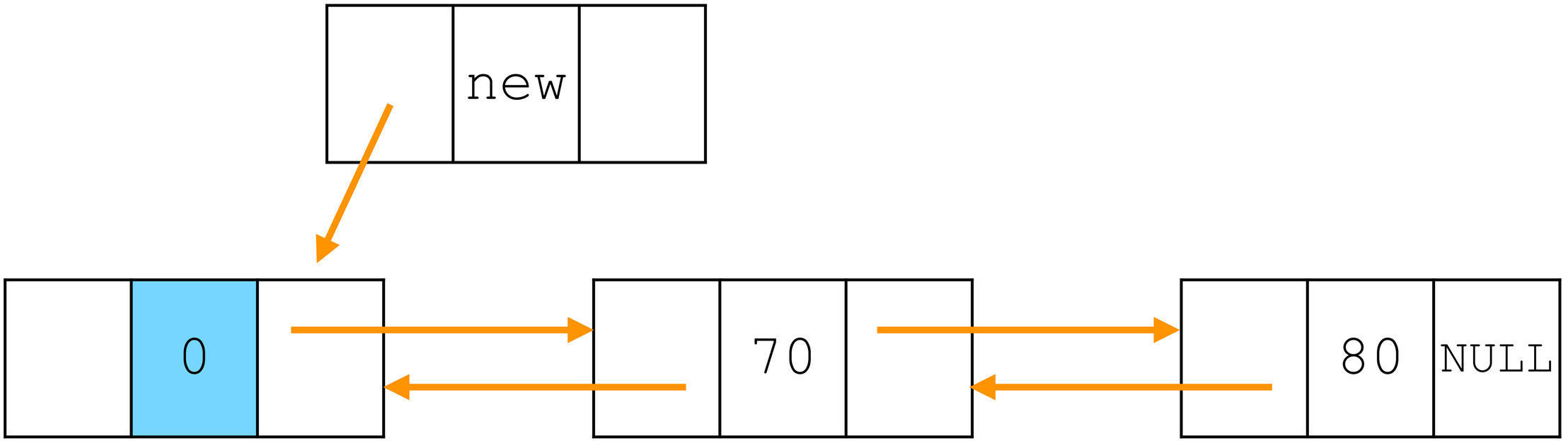
双向链表 – 头插法

案例：线性表/链表/test17.c



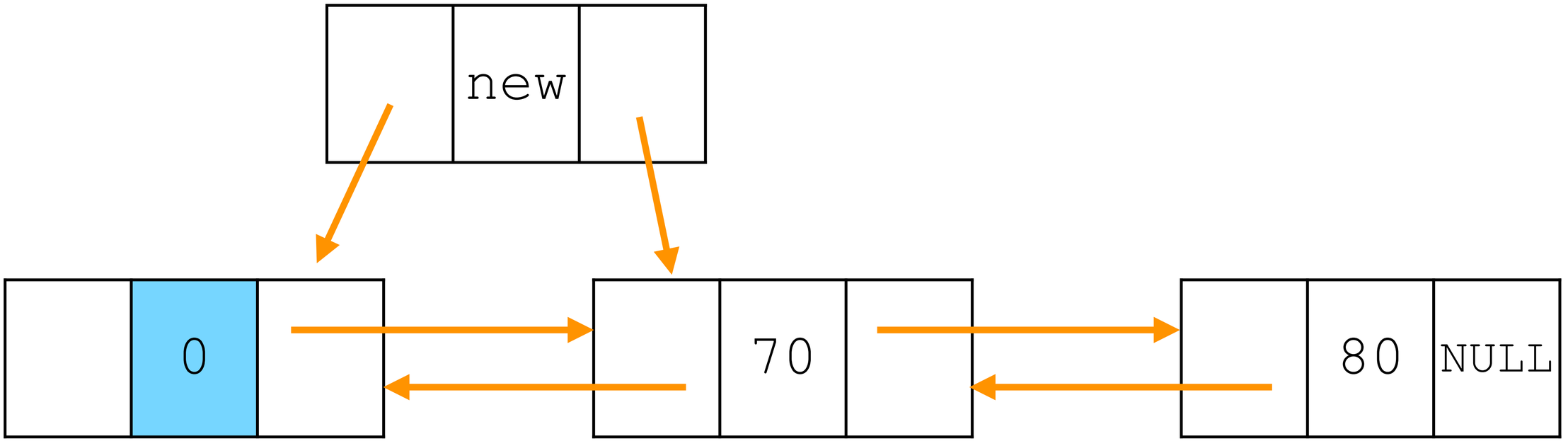
双向链表 – 头插法

案例：线性表/链表/test17.c



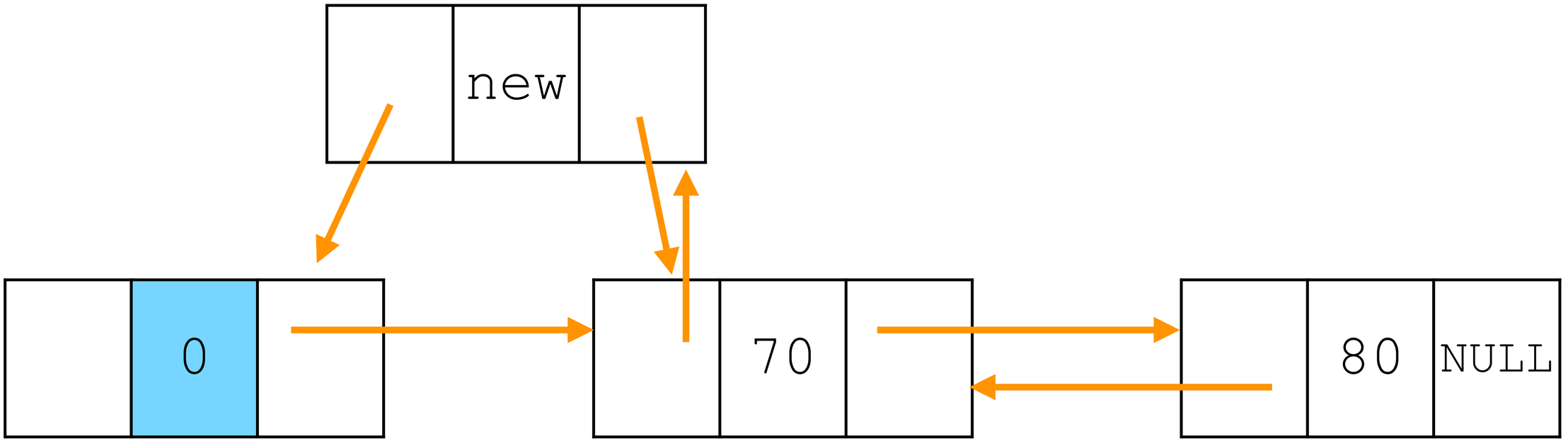
双向链表 – 头插法

案例：线性表/链表/test17.c



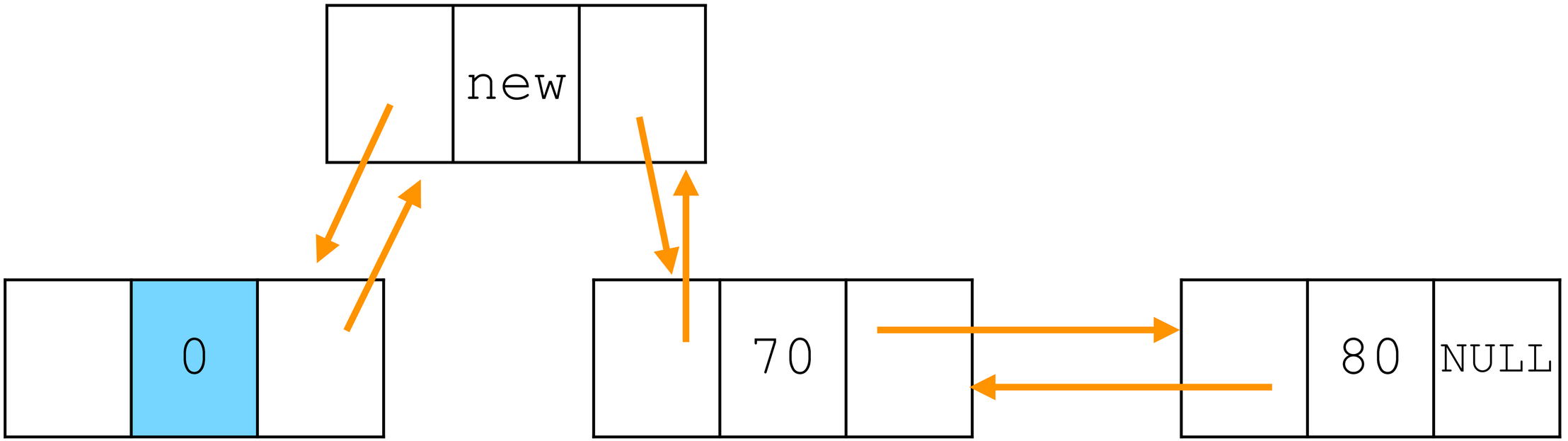
双向链表 – 头插法

案例：线性表/链表/test17.c



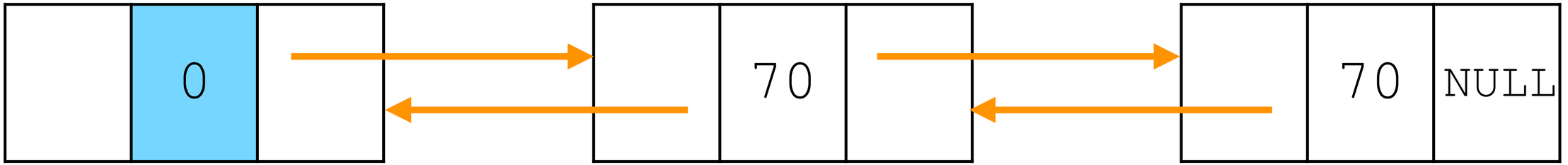
双向链表 – 头插法

案例：线性表/链表/test17.c



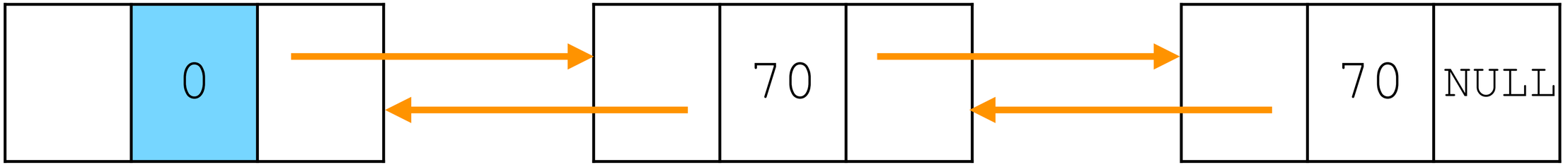
双向链表 - 尾插法

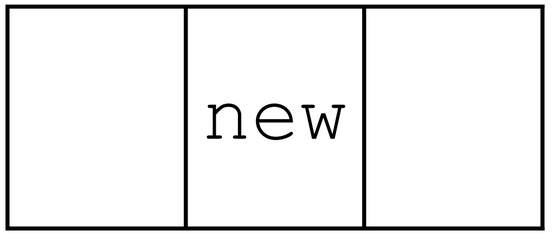
案例：线性表/链表/test17.c



双向链表 - 尾插法

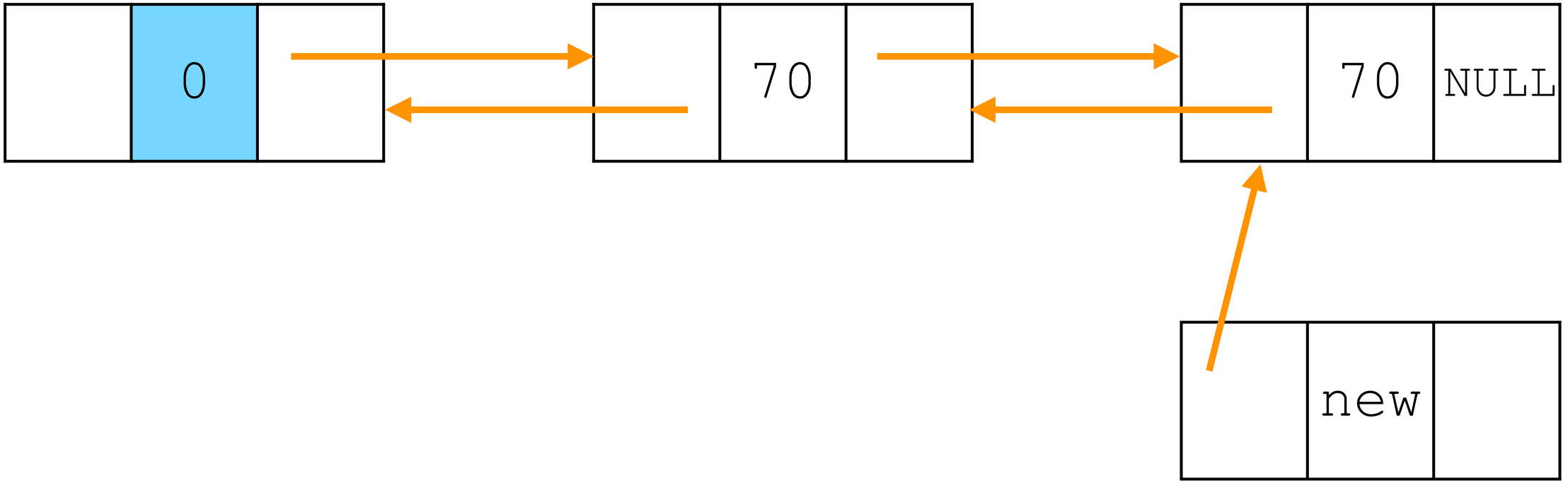
案例：线性表/链表/test17.c





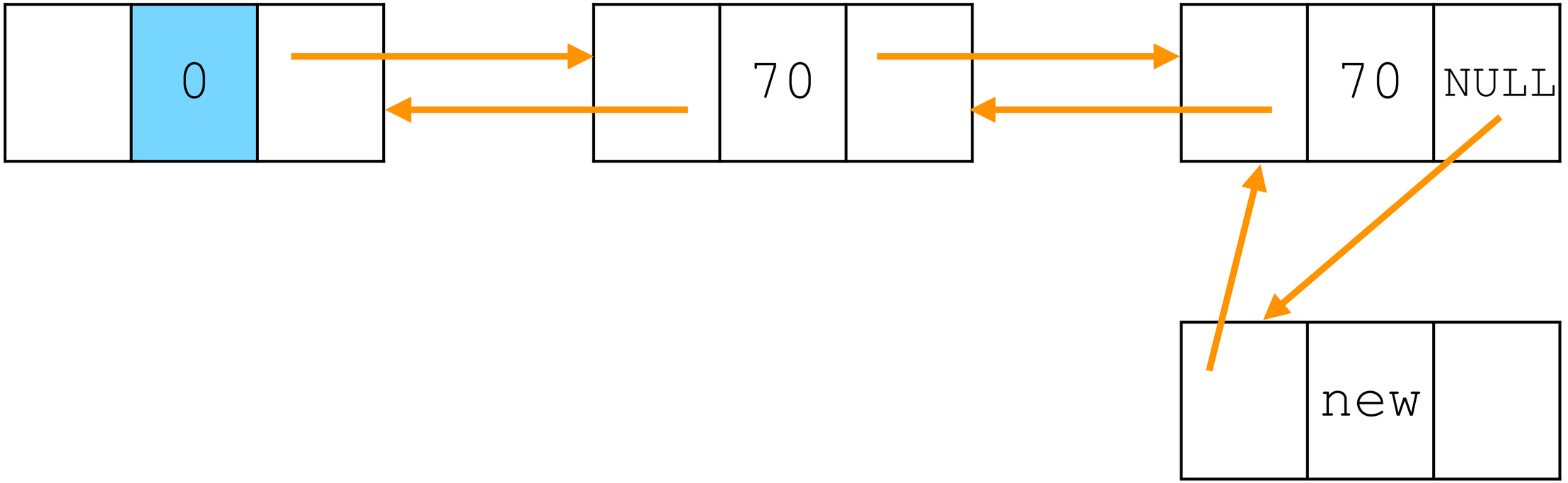
双向链表 - 尾插法

案例：线性表/链表/test17.c



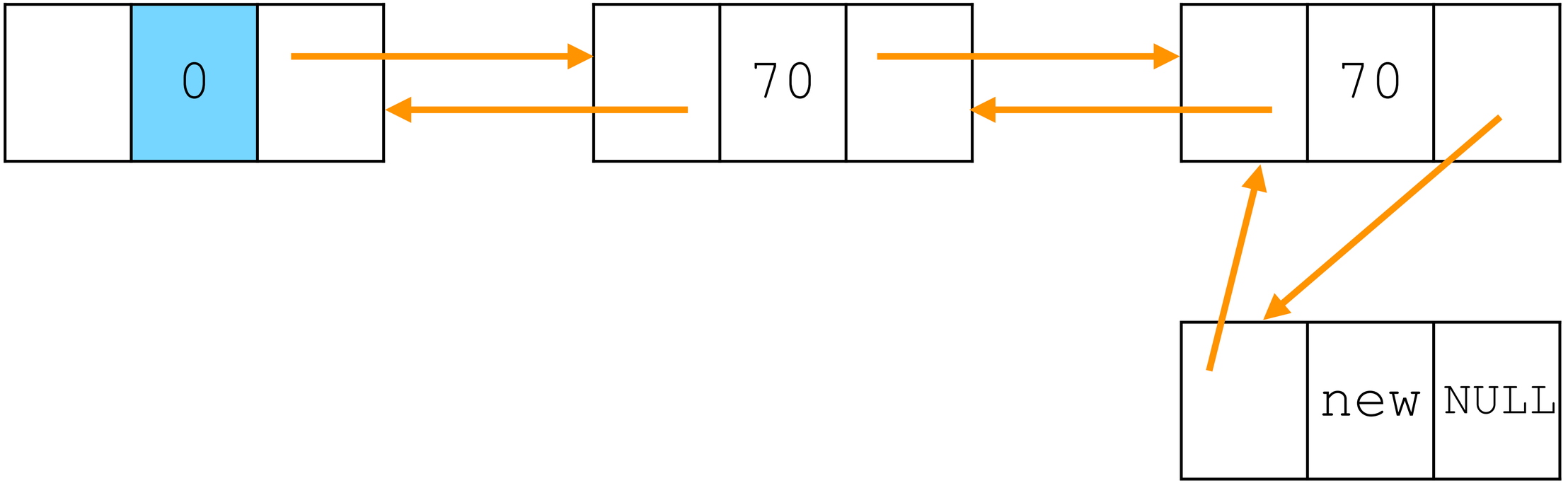
双向链表 – 尾插法

案例：线性表/链表/test17.c



双向链表 – 尾插法

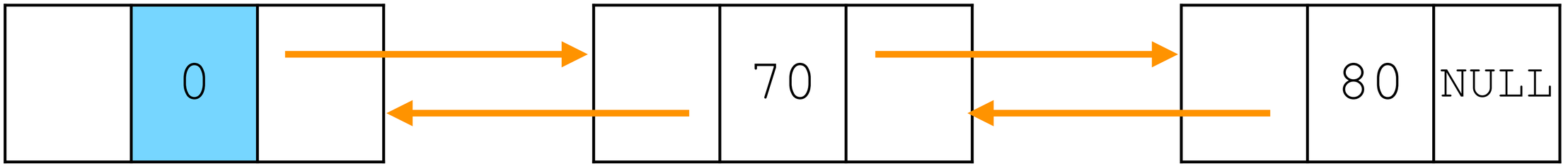
案例：线性表/链表/test17.c



双向链表 – 在指定位置插入数据

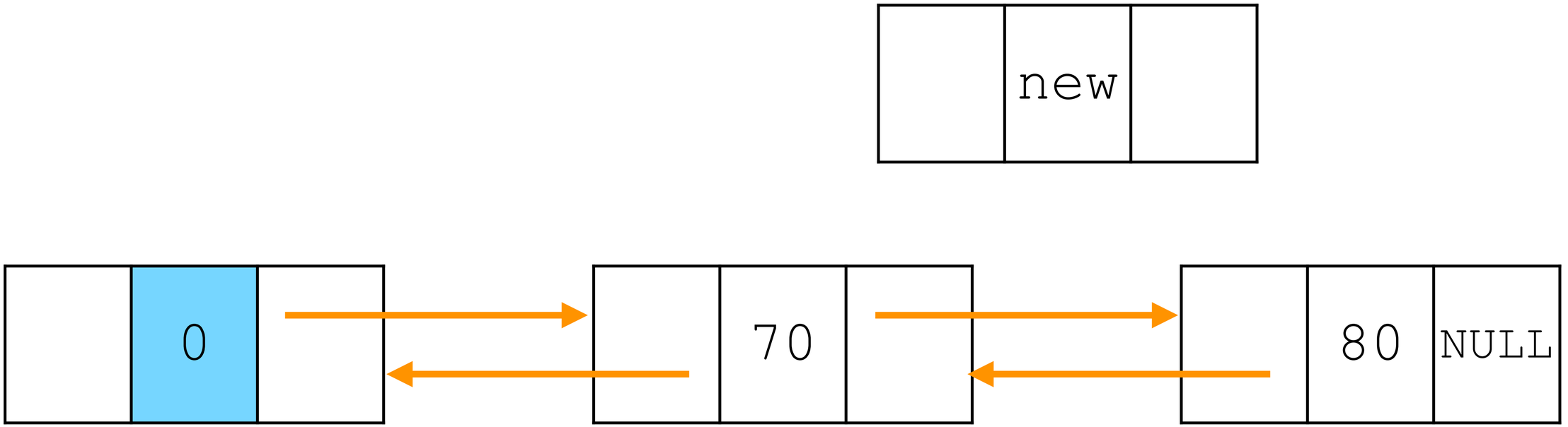
双向链表 – 在指定位置插入数据

案例：线性表/链表/test17.c



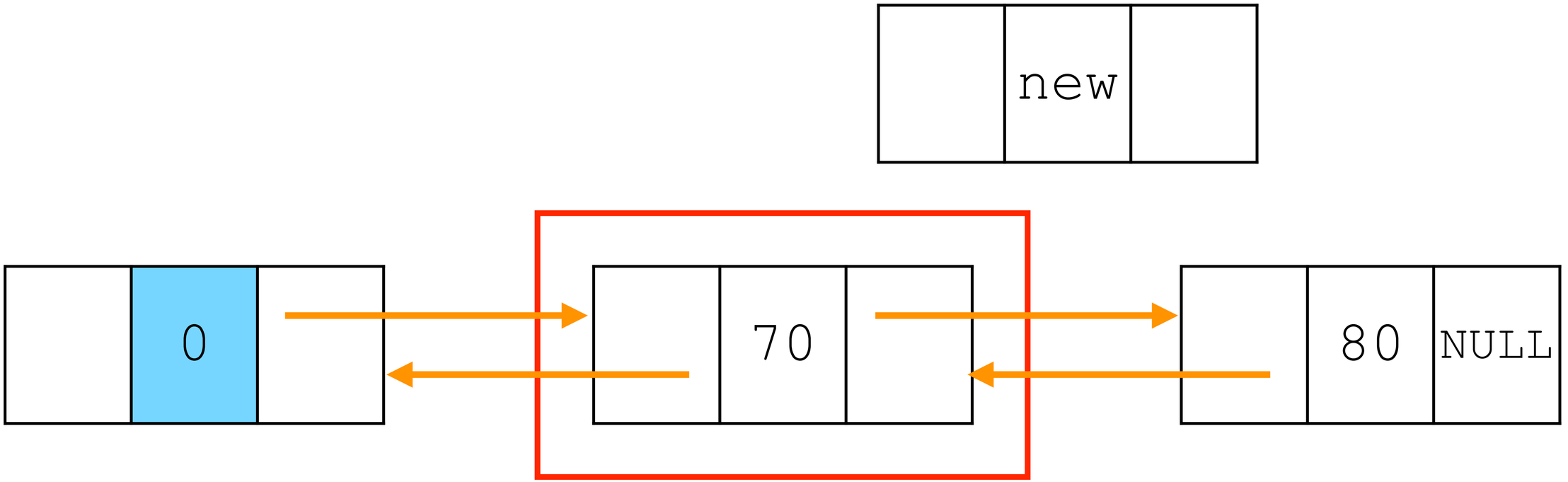
双向链表 – 在指定位置插入数据

案例：线性表/链表/test17.c



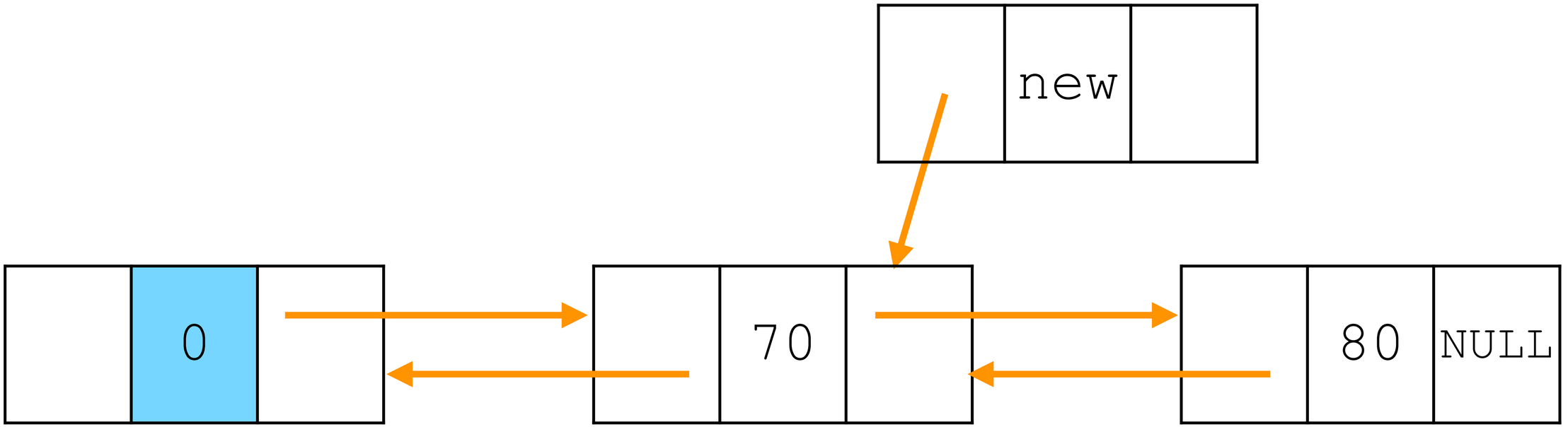
双向链表 – 在指定位置插入数据

案例：线性表/链表/test17.c



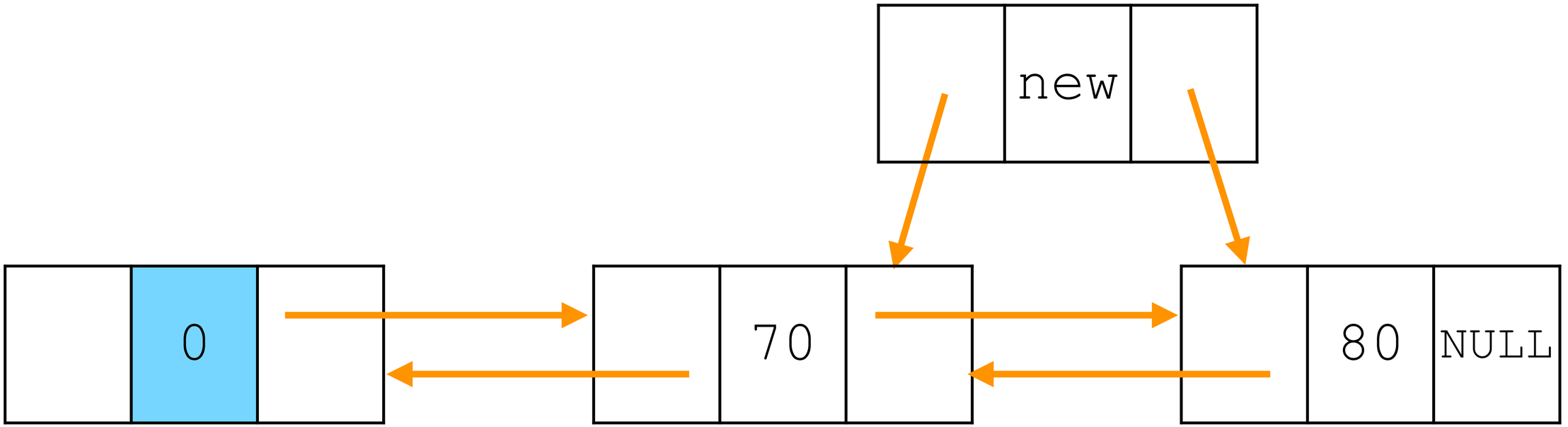
双向链表 – 在指定位置插入数据

案例：线性表/链表/test17.c



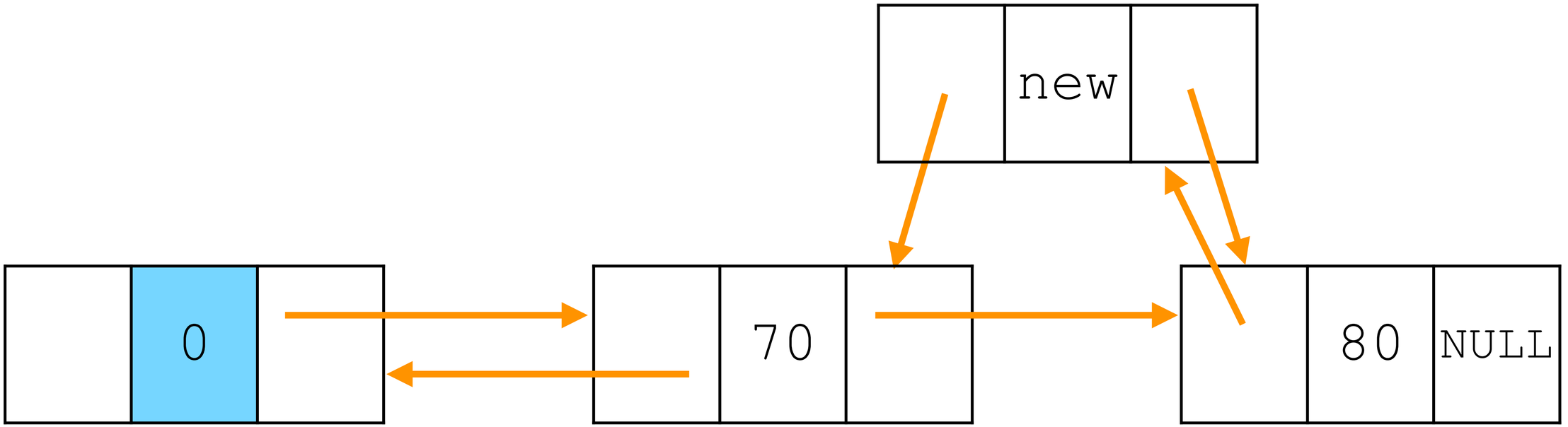
双向链表 – 在指定位置插入数据

案例：线性表/链表/test17.c



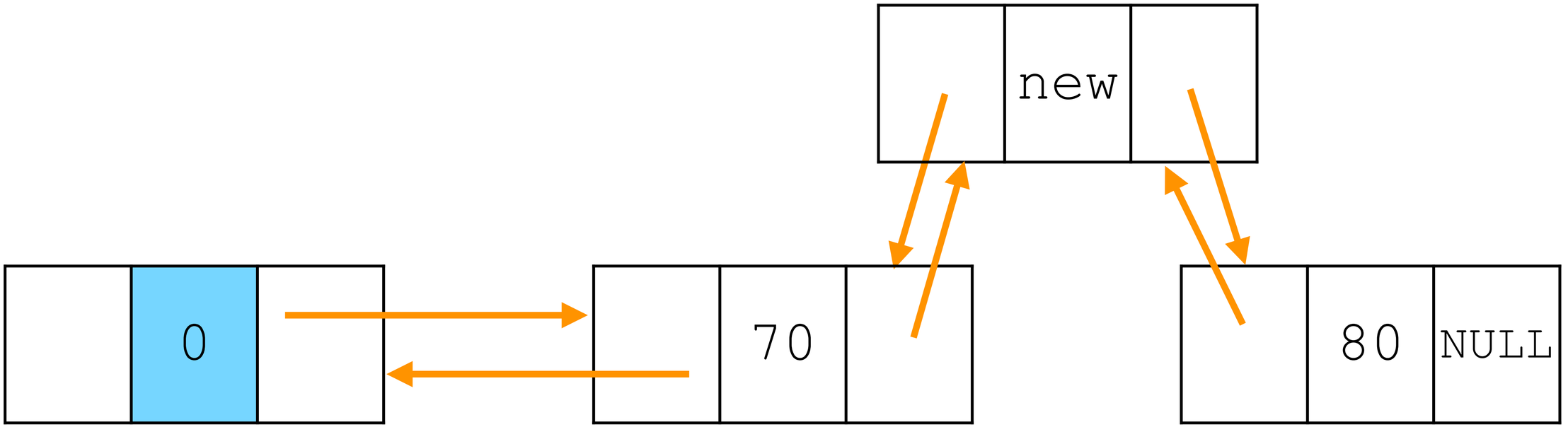
双向链表 – 在指定位置插入数据

案例：线性表/链表/test17.c



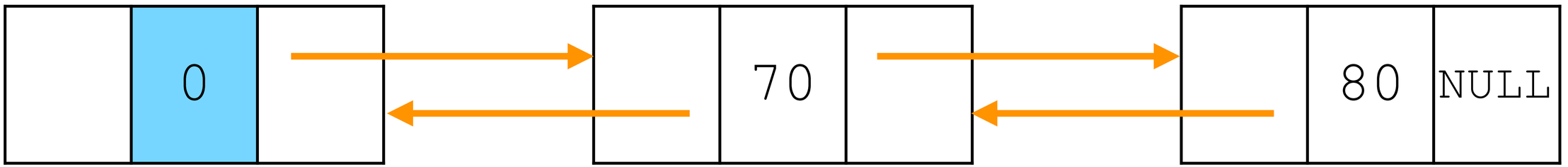
双向链表 – 在指定位置插入数据

案例：线性表/链表/test17.c



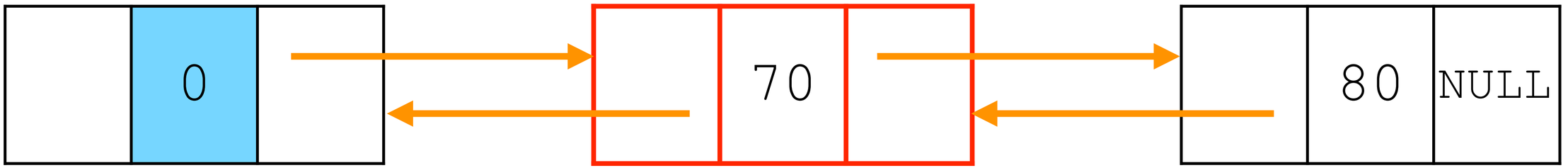
双向链表 – 删除节点

案例：线性表/链表/test17.c



双向链表 – 删除节点

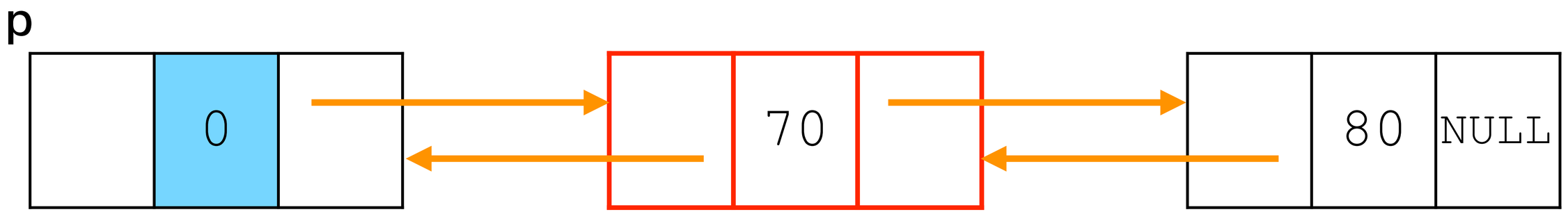
案例：线性表/链表/test17.c



双向链表 – 删除节点

案例：线性表/链表/test17.c

找到要删除节点的前置节点 p

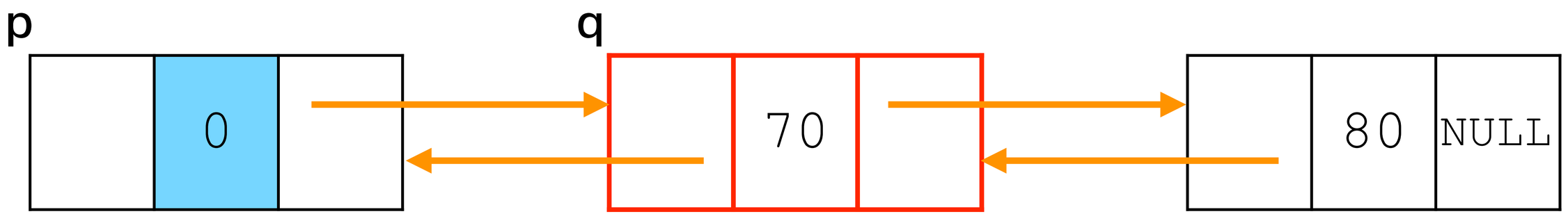


双向链表 – 删除节点

案例：线性表/链表/test17.c

找到要删除节点的前置节点 p

用指针 q 记录要删除的节点



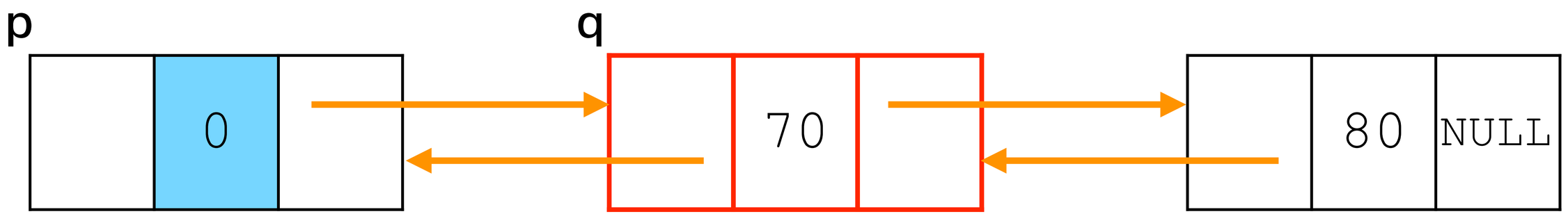
双向链表 – 删除节点

案例：线性表/链表/test17.c

找到要删除节点的前置节点 p

用指针 q 记录要删除的节点

通过改变 p 的后继节点实现删除



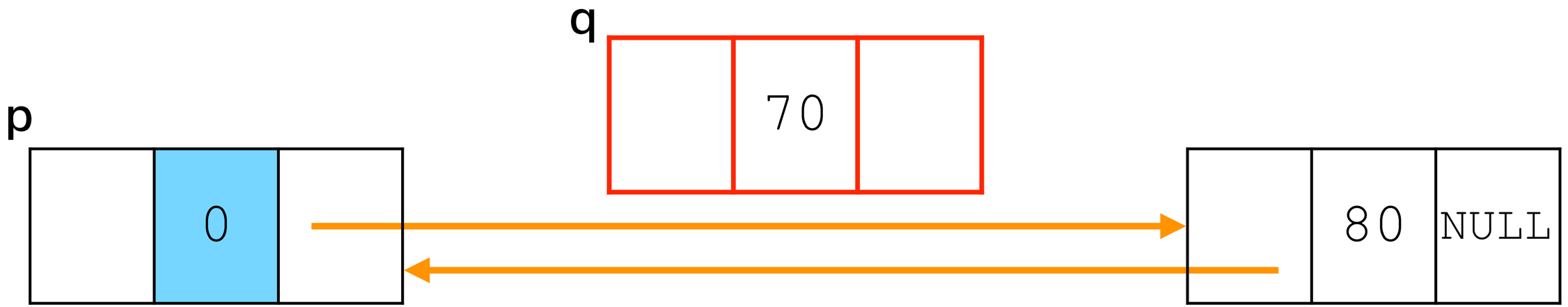
双向链表 – 删除节点

案例：线性表/链表/test17.c

找到要删除节点的前置节点 p

用指针 q 记录要删除的节点

通过改变 p 的后继节及要删除节点的下一个节点的前驱实现删除



双向链表 – 删除节点

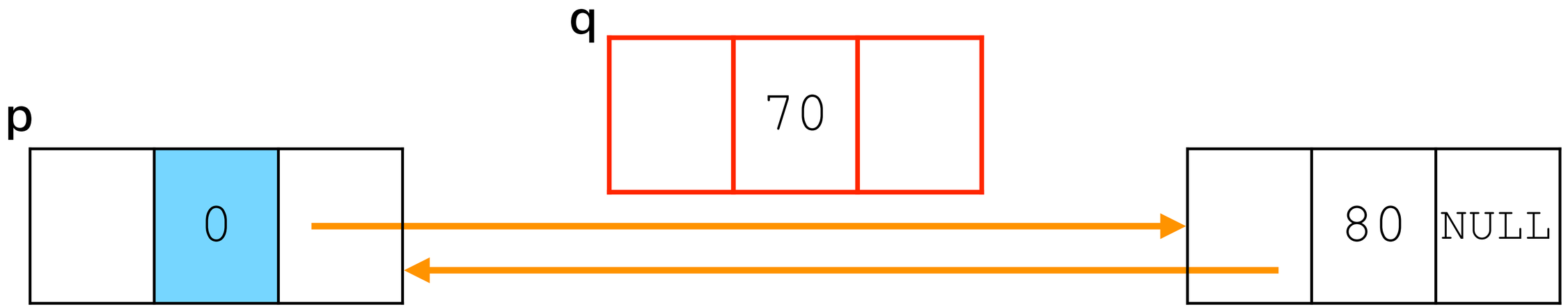
案例：线性表/链表/test17.c

找到要删除节点的前置节点 p

用指针 q 记录要删除的节点

通过改变 p 的后继节及要删除节点的下一个节点的前驱实现删除

释放删除节点的空间



顺序表 VS 链表

比较项目		存储结构	
		顺序表	链表
空间	存储空间	预先分配，会出现闲置或溢出现象	动态分配，不会出现存储空间闲置或溢出现象
	存储密度	不用为表示节点间的逻辑关系而增加额外的存储，存储密度等于 1	需要借助指针来体现元素间的逻辑关系，存储密度小于 1
时间	存取元素	随机存取，按位置访问元素的时间复杂度为 O(1)	顺序存取，按位置访问元素时间复杂度为 O(n)
	插入、删除	平均移动约表中一半元素，时间复杂度为 O(n)	不需要移动元素，确定插入、删除位置后，时间复杂度为O(1)
适用情况		1) 表长变化不大，且能事先确定变化的范围 2) 很少进行插入或删除操作，经常按元素位置序号访问数据元素	1) 长度变化较大 2) 频繁进行插入或删除操作

链表题

[2016] 已知表头元素为 c 的单链表在内存中的存储状态如下表所示。

地址	元素	链接地址
1000H	a	1010H
1004H	b	100CH
1008H	c	1000H
100CH	d	NULL
1010H	e	1004H
1014H		

现将f存放于1014H处并插入到链表中，若f在逻辑上位于a和e之间，则a，e，f，的“链接地址”依次是_____。

A. 1010H, 1014H, 1004H

B. 1010H, 1004H, 1014H

C. 1014H, 1010H, 1004H

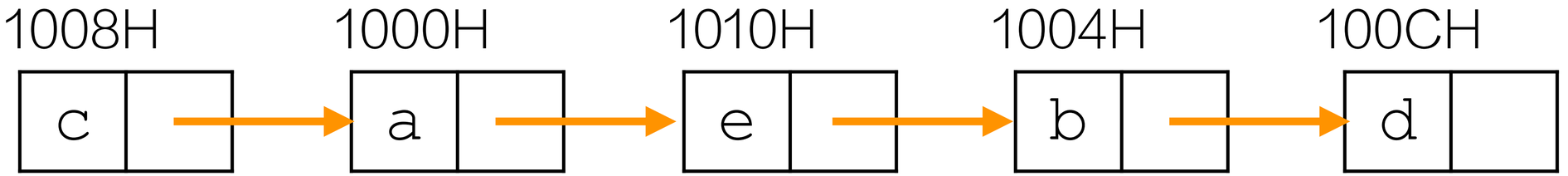
D. 1014H, 1004H, 1010H

链表题

[2016] 已知表头元素为 c 的单链表在内存中的存储状态如下表所示。

地址	元素	链接地址
1000H	a	1010H
1004H	b	100CH
1008H	c	1000H
100CH	d	NULL
1010H	e	1004H
1014H		

现将f存放于1014H处并插入到链表中，若f在逻辑上位于a和e之间，则a，e，f，的“链接地址”依次是_____。



链表题

[2016] 已知表头元素为 c 的单链表在内存中的存储状态如下表所示。

地址	元素	链接地址
1000H	a	1010H
1004H	b	100CH
1008H	c	1000H
100CH	d	NULL
1010H	e	1004H
1014H		

现将f存放于1014H处并插入到链表中，若f在逻辑上位于a和e之间，则a，e，f，的“链接地址”依次是_____。

A. 1010H, 1014H, 1004H

B. 1010H, 1004H, 1014H

B. 1014H, 1010H, 1004H

D. 1014H, 1004H, 1010H

链表题

[2016] 已知一个带有表头节点的双向循环链表L，节点结构为

prev	data	next

其中，prev 和 next 分别指向其直接前驱和直接后继节点的指针。现要删除指针 p 所指向的节点。正确的语句序列是_____。

A. p->next->prev = p->prev; p->prev->next = p->prev; free(p);

B. p->next->prev = p->next; p->prev->next = p->next; free(p);

C. p->next->prev = p->next; p->prev->next = p->prev; free(p);

D. p->next->prev = p->prev; p->prev->next = p->next; free(p);

链表题

[2016] 已知一个带有表头节点的双向循环链表L，节点结构为

prev	data	next

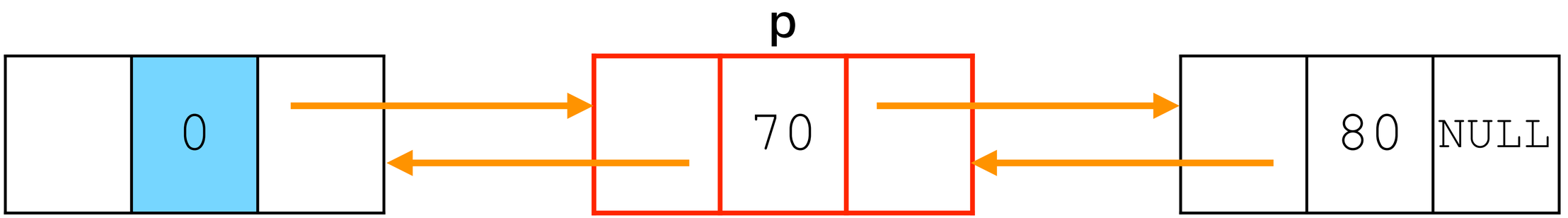
其中，prev 和 next 分别指向其直接前驱和直接后继节点的指针。现要删除指针 p 所指向的节点。正确的语句序列是_____。

A. p->next->prev = p->prev; p->prev->next = p->prev; free(p);

B. p->next->prev = p->next; p->prev->next = p->next; free(p);

C. p->next->prev = p->next; p->prev->next = p->prev; free(p);

D. p->next->prev = p->prev; p->prev->next = p->next; free(p);



链表题

[2016] 已知一个带有表头节点的双向循环链表L，节点结构为

prev	data	next

其中，prev 和 next 分别指向其直接前驱和直接后继节点的指针。现要删除指针 p 所指向的节点。正确的语句序列是_____。

A. p->next->prev = p->prev; p->prev->next = p->prev; free(p);

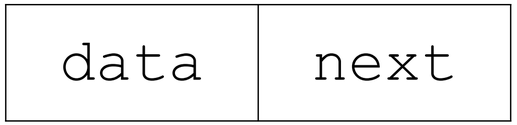
B. p->next->prev = p->next; p->prev->next = p->next; free(p);

C. p->next->prev = p->next; p->prev->next = p->prev; free(p);

D. p->next->prev = p->prev; p->prev->next = p->next; free(p);

链表题

[2021] 已知头指针 h 指向一个带头节点的非空单循环链表，节点结构为：



其中，next 是指向直接后继节点的指针，p 是尾指针，q 是临时指针。现要删除该链表的第一个元素，正确的语句序列是_____。

A. h->next=h->next->next; q = h->next; free(q);

B. q = h->next; h->next = h->next->next; free(q);

C. q = h->next; h->next = q-> next; if(p!=q)p=h; free(q);

D. q = h->next; h->next = q->next; if(p==q)p=h; free(q);

链表题

[2021] 已知头指针 h 指向一个带头节点的非空单循环链表，节点结构为：

data	next

其中，next 是指向直接后继节点的指针，p 是尾指针，q 是临时指针。现要删除该链表的第一个元素，正确的语句序列是_____。

A. h->next=h->next->next; q = h->next; free(q);

B. q = h->next; h->next = h->next->next; free(q);

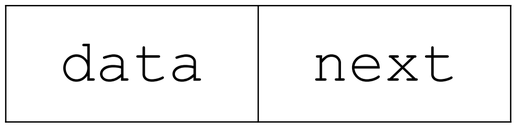
C. q = h->next; h->next = q-> next; if(p!=q)p=h; free(q);

D. q = h->next; h->next = q->next; if(p==q)p=h; free(q);



链表题

[2021] 已知头指针 h 指向一个带头节点的非空单循环链表，节点结构为：



其中，next 是指向直接后继节点的指针，p 是尾指针，q 是临时指针。现要删除该链表的第一个元素，正确的语句序列是_____。

A. h->next=h->next->next; q = h->next; free(q);

B. q = h->next; h->next = h->next->next; free(q);

C. q = h->next; h->next = q-> next; if(p!=q)p=h; free(q);

D. q = h->next; h->next = q->next; if(p==q)p=h; free(q);